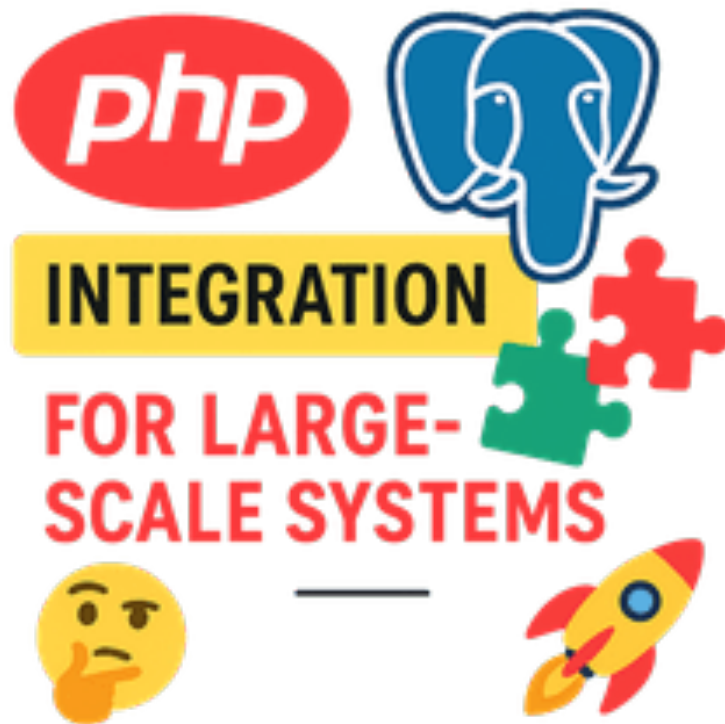




PHP и PostgreSQL Интеграция для крупных систем

Модуль 1 - Введение в интеграцию PHP и PostgreSQL

Урок 1 - Установка и настройка PostgreSQL

Шаг 01:

Миссия 1

Программист Иван впервые устанавливает PostgreSQL на сервер. Какой пакет чаще всего используется для установки PostgreSQL в Linux (Ubuntu/Debian)?

Подсказка: Вспомните стандартный пакетный менеджер apt и название пакета, связанного с postgres.

Ответ:

- postgresql
- mysql-server
- sqlite3
- php-pgsql

Шаг 02:

Миссия 2

Разработчица Анна решила проверить, запущен ли сервис PostgreSQL на её машине. Какую команду она должна использовать?

Подсказка: Думайте про systemctl — он управляет службами в Linux.

Ответ:

- **systemctl status postgresql**
- `psql --status`
- `service mysql status`
- `pg_status`

Шаг 03:

Миссия 3

Младший разработчик Пётр установил PostgreSQL, но не знает, как подключиться к консоли базы данных. Какой инструмент ему нужен?

Подсказка: Это стандартная утилита для работы с PostgreSQL из терминала.

Ответ:

- **psql**
- `mysql`
- `sqlite`
- `pgadmin-desktop`

Шаг 04:

Миссия 4

Сисадмин Олег хочет войти в PostgreSQL под пользователем по умолчанию. Какое имя пользователя создаётся вместе с установкой?

Подсказка: Этот пользователь совпадает с названием самой СУБД.

Ответ:

- **postgres**
- `root`
- `admin`
- `dbuser`

Шаг 05:

Миссия 5

Разработчик Кирилл готовится подключить PHP к PostgreSQL. Какое расширение PHP отвечает за взаимодействие с PostgreSQL?

Подсказка: В названии модуля присутствуют буквы pg.

Ответ:

- **pdo_pgsql**
- `pdo_mysql`
- `mysqli`
- `pdo_sqlite`

Шаг 06:

Миссия 6

Программистка Мария установила PostgreSQL. Она хочет, чтобы сервер запускался автоматически после перезагрузки. Какой командой это включается?

Подсказка: Используйте `systemctl` и вспомните слово `enable`.

Ответ:

- **systemctl enable postgresql**
- **service start postgresql**
- **pg_start --auto**
- **systemctl reload postgresql**

Шаг 07:

Миссия 7

 Задача: Настройки подключения к PostgreSQL

Сопоставьте параметр конфигурации с его назначением.

Подсказка: Думайте о том, какой параметр отвечает за IP, какой — за порт, а какой — за пользователей.

Ответ:

- **listen_addresses** — Указывает, на каких IP-адресах PostgreSQL будет слушать соединения.
- **port** — Определяет номер порта для подключения.
- **pg_hba.conf** — Файл, где задаются правила доступа пользователей.
- **max_connections** — Ограничивает количество одновременных подключений.

Шаг 08:

Миссия 8

 Задача: Разработчик хочет понять роли в PostgreSQL

Сопоставьте роль с её описанием.

Подсказка: Подумайте, какая роль суперпользовательская, а какая подходит для простого чтения.

Ответ:

- **postgres** — Суперпользователь, созданный по умолчанию.
- **readonly** — Роль для пользователей, которые могут только читать данные.
- **readwrite** — Роль с правами на чтение и запись.
- **replication** — Роль для настройки репликации базы данных.

Шаг 09:

Миссия 9

 Задача: Конфигурационные файлы PostgreSQL

Сопоставьте файл с его назначением.

Подсказка: Один отвечает за логику доступа, другой за параметры сервера.

Ответ:

- **postgresql.conf** — Основной файл конфигурации сервера PostgreSQL.
- **pg_hba.conf** — Файл для настройки правил аутентификации клиентов.
- **pg_ident.conf** — Файл для сопоставления системных пользователей с ролями PostgreSQL.
- **pg_log** — Каталог, куда сохраняются журналы работы PostgreSQL.

Шаг 10:

Миссия 10

 Задача: Типовые команды psql

Сопоставьте команду с её действием.

Подсказка: Все команды начинаются с обратного слэша.

Ответ:

- **\l** — Показать список всех баз данных.
- **\c** — Подключиться к выбранной базе данных.
- **\dt** — Вывести список таблиц в текущей базе.
- **\du** — Показать всех пользователей и роли.

Шаг 11:

Миссия 11

 Задача: Сопоставьте утилиту с её назначением

Подсказка: `pg_dump` и `pg_restore` обычно используют вместе.

Ответ:

- **pg_dump** — Создание резервной копии базы данных.
- **pg_restore** — Восстановление базы данных из дампа.
- **createdb** — Создание новой базы данных.
- **dropdb** — Удаление базы данных.

Шаг 12:

Миссия 12

 Задача: Сопоставьте параметр настройки с его целью оптимизации

Подсказка: `work_mem` и `shared_buffers` часто упоминаются при тюнинге.

Ответ:

- **shared_buffers** — Определяет объем памяти для кэширования страниц данных.
- **work_mem** — Используется для операций сортировки и хеширования.
- **maintenance_work_mem** — Применяется при операциях `VACUUM` и `CREATE INDEX`.
- **checkpoint_timeout** — Интервал между контрольными точками записи на диск.

Шаг 13:

Миссия 13

Расставьте шаги установки PostgreSQL в правильном порядке:

Подсказка: Сначала добавляется репозиторий, потом обновляются пакеты, далее установка и запуск.

Ответ:

- Добавить официальный репозиторий PostgreSQL.
- Обновить список пакетов `apt`.
- Установить пакет `postgresql`.
- Запустить и проверить службу PostgreSQL.

Шаг 14:

Миссия 14

Расположите шаги подключения PHP к PostgreSQL в правильной последовательности:

Подсказка: Сначала ставим расширение, потом настраиваем `php.ini`, а затем проверяем соединение.

Ответ:

- Установить расширение pdo_pgsql.
- Настроить php.ini для загрузки расширения.
- Создать PHP-скрипт с подключением к PostgreSQL.
- Проверить выполнение скрипта через браузер или CLI.

 Шаг 15:

Миссия 15

Расположите шаги создания новой базы данных и пользователя:

Подсказка: Сначала создаётся пользователь, потом база, затем выдаются права.

Ответ:

- Создать нового пользователя.
- Создать новую базу данных.
- Назначить права пользователю на базу.
- Проверить подключение под новым пользователем.

 Шаг 16:

Миссия 16

Разработчица Елена настраивает удалённое подключение к PostgreSQL. Расставьте действия в нужном порядке:

Подсказка: Нужно изменить listen_addresses, затем pg_hba.conf, а потом перезапустить сервер.

Ответ:

- Открыть postgresql.conf и изменить listen_addresses.
- Отредактировать pg_hba.conf, добавив IP клиента.
- Перезапустить службу PostgreSQL.
- Проверить подключение с удалённой машины.

 Урок 2 - Как PHP взаимодействует с PostgreSQL

 Шаг 01:

Миссия 1

Программист Андрей подключает PHP к PostgreSQL. Какой драйвер используется в связке с PDO?

Подсказка: Название драйвера начинается с pdo_ и указывает на PostgreSQL.

Ответ:

- pdo_pgsql
- pdo_mysql
- pdo_sqlite
- pgsql_connect

 Шаг 02:

Миссия 2

Разработчица Катя хочет установить соединение с PostgreSQL через PHP. Какая функция используется для этого в процедурном стиле?

Подсказка: Она начинается с pg_ и напоминает слово connect.

Ответ:

- **pg_connect**
- pg_query
- pg_exec
- pdo_pgsql_connect

🔮 **Шаг 03:**

Миссия 3

Программист Владислав хочет выполнить SQL-запрос через PDO. Какая функция применяется для подготовки запроса?

Подсказка: PDO всегда работает с подготовкой через prepare.

Ответ:

- **prepare**
- executeQuery
- pg_query
- runSQL

🔮 **Шаг 04:**

Миссия 4

Разработчица Света подключилась к PostgreSQL через PHP. Какой метод PDO она должна использовать для выполнения подготовленного запроса?

Подсказка: После prepare всегда идёт вызов метода execute().

Ответ:

- **execute**
- run
- start
- query

🔮 **Шаг 05:**

Миссия 5

Программист Денис пишет код на PHP. Какая функция вернёт количество строк, затронутых последним SQL-запросом?

Подсказка: В PDO для этого используют rowCount().

Ответ:

- **rowCount**
- numRows
- countRows
- rowsAffected

🔮 **Шаг 06:**

Миссия 6

📖 **Задача:** Основные функции работы с PostgreSQL в PHP

Сопоставьте функцию с её назначением.

Подсказка: Вспомните, что одна функция подключает, другая выполняет запрос, третья получает строку.

Ответ:

- **pg_connect** — Создание соединения с сервером PostgreSQL.
- **pg_query** — Выполнение SQL-запроса.
- **pg_fetch_assoc** — Получение строки результата в виде ассоциативного массива.
- **pg_close** — Закрытие соединения с сервером.

Шаг 07:

Миссия 7

 Задача: Работа с PDO

Сопоставьте метод PDO с его действием.

Подсказка: Один метод готовит, другой выполняет, а третий получает результат.

Ответ:

- **prepare** — Подготовка SQL-запроса.
- **execute** — Выполнение подготовленного запроса.
- **fetch** — Получение одной строки результата.
- **fetchAll** — Получение всех строк результата.

Шаг 08:

Миссия 8

 Задача: Обработка ошибок

Сопоставьте тип обработки ошибок в PDO с его описанием.

Подсказка: Вспомните три режима: тихий, предупреждающий и исключения.

Ответ:

- **ERRMODE_SILENT** — Ошибки не выводятся, нужно проверять вручную.
- **ERRMODE_WARNING** — Ошибки выводятся как предупреждения.
- **ERRMODE_EXCEPTION** — Ошибки вызывают исключение.
- **ERRMODE_DEFAULT** — Поведение по умолчанию (обычно SILENT).

Шаг 09:

Миссия 9

 Задача: Результаты запросов

Сопоставьте метод получения данных с его описанием.

Подсказка: `fetchColumn` возвращает одно значение, `fetchAll` возвращает много.

Ответ:

- **fetch** — Возвращает одну строку результата.
- **fetchAll** — Возвращает все строки результата.
- **fetchColumn** — Возвращает одно значение из строки результата.
- **fetchObject** — Возвращает строку результата как объект.

Шаг 10:

Миссия 10

 Задача: Плейсхолдеры в PDO

Сопоставьте вид плейсхолдера с примером использования.

Подсказка: Есть именованные (:name) и позиционные (?).

Ответ:

- **Именованный плейсхолдер** — `INSERT INTO users(name) VALUES(:name)`
- **Позиционный плейсхолдер** — `INSERT INTO users(name) VALUES(?)`
- **Именованный плейсхолдер с несколькими параметрами** — `UPDATE users SET age = :age WHERE name = :name`
- **Позиционный плейсхолдер в WHERE** — `SELECT * FROM users WHERE id = ?`

Шаг 11:

Миссия 11

 Задача: Отличия `pg_query` и `pg_prepare/pg_execute`

Сопоставьте функцию с её назначением.

Подсказка: Одна выполняет сразу, другая готовит запрос.

Ответ:

- **`pg_query`** — Выполняет SQL-запрос без подготовки.
- **`pg_prepare`** — Готовит SQL-запрос с плейсхолдерами.
- **`pg_execute`** — Выполняет ранее подготовленный запрос.
- **`pg_fetch_row`** — Извлекает строку результата как массив с индексами.

Шаг 12:

Миссия 12

Расставьте шаги работы с PDO при выполнении запроса в правильном порядке:

Подсказка: Сначала создаётся объект PDO, потом готовится запрос, затем выполняется.

Ответ:

- Создать объект PDO с параметрами подключения.
- Подготовить SQL-запрос через `prepare`.
- Выполнить запрос через `execute`.
- Получить результат с помощью `fetch` или `fetchAll`.

Шаг 13:

Миссия 13

Расставьте шаги подключения с помощью `pg_connect` в правильной последовательности:

Подсказка: Сначала строка подключения, затем `pg_connect`, потом запрос и закрытие.

Ответ:

- Сформировать строку подключения.
- Подключиться через `pg_connect`.
- Выполнить запрос через `pg_query`.
- Закрыть соединение через `pg_close`.

Шаг 14:

Миссия 14

Расставьте шаги обработки ошибок при работе с PDO:

Подсказка: Включаем режим ошибок, выполняем запрос, обрабатываем исключение.

Ответ:

- Создать объект PDO.

- Установить `ERRMODE_EXCEPTION`.
- Выполнить SQL-запрос.
- Поймать исключение в блоке `try/catch`.

🔧 Шаг 15:

Миссия 15

Расставьте шаги выполнения подготовленного запроса с параметрами:

Подсказка: Подготовка, привязка значений, выполнение.

Ответ:

- Подготовить SQL-запрос с плейсхолдерами.
- Привязать значения к параметрам.
- Выполнить запрос через `execute`.
- Извлечь данные через `fetch` или `fetchAll`.

🔧 Шаг 16:

Миссия 16

Расставьте шаги интеграции PHP и PostgreSQL для простого веб-приложения:

Подсказка: Сначала соединение, потом запрос, затем отображение данных.

Ответ:

- Подключиться к PostgreSQL (PDO или `pg_connect`).
- Подготовить и выполнить SQL-запрос.
- Извлечь результаты в массив или объект.
- Отобразить данные в HTML-странице.

📖 Урок 3 - Основы работы с PDO и PgSQL в PHP

🔧 Шаг 01:

Миссия 1

Александр пишет PHP-скрипт, который должен подключиться к PostgreSQL через PDO. Какой правильный DSN он должен использовать?

Подсказка: В DSN указывается драйвер, хост и база данных.

Ответ:

- **`pgsql:host=localhost;dbname=testdb`**
- `mysql:host=localhost;dbname=testdb`
- `postgres:localhost/testdb`
- `pdo_pgsql://localhost/testdb`

🔧 Шаг 02:

Миссия 2

Марина заметила, что её код падает при попытке соединиться с базой. Она использует PDO. Какой блок кода поможет безопасно отловить ошибку подключения?

Подсказка: Вспомните конструкцию для обработки исключений в PHP.

Ответ:

- **`try { new PDO(...); } catch (PDOException $e) { echo $e->getMessage(); }`**
- `if (!PDO(...)) { die('Ошибка'); }`

- `PDO::connect_or_die();`
- `catch_exception(PDO) { echo 'Ошибка'; }`

🔗 Шаг 03:

Миссия 3

Иван подключился к PostgreSQL через PDO, но его запрос возвращает странные символы вместо кириллицы. Что он забыл указать?

Подсказка: Проверьте установку кодировки соединения.

Ответ:

- **`charset=utf8` в строке DSN**
- `password=utf8`
- `set_encoding=true`
- `include 'utf8.php'`

🔗 Шаг 04:

Миссия 4

Дмитрий хочет выполнить SQL-запрос с параметрами через PDO. Какой метод ему нужно использовать?

Подсказка: Этот метод подготавливает выражение и защищает от SQL-инъекций.

Ответ:

- **`prepare()`**
- `query()`
- `exec()`
- `bind()`

🔗 Шаг 05:

Миссия 5

Катя делает запрос через PDO и хочет получить все строки результата в виде ассоциативного массива. Какой метод подойдёт?

Подсказка: Вспомните про метод, который возвращает сразу все результаты.

Ответ:

- **`fetchAll(PDO::FETCH_ASSOC)`**
- `fetch()`
- `queryAll()`
- `getAssoc()`

🔗 Шаг 06:

Миссия 6

Разработчик подключился к базе через PDO, но забыл включить отображение ошибок. Какой параметр в `setAttribute` ему нужно указать?

Подсказка: Нужно включить режим выброса исключений.

Ответ:

- **`PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION`**
- `PDO::ATTR_DEBUG, true`
- `PDO::SHOW_ERRORS, 1`

- PDO::ATTR_LOG, 'on'

Шаг 07:

Миссия 7

Елена использует PDO::exec для INSERT-запроса. Что вернёт этот метод?

Подсказка: Этот метод не возвращает результат выборки.

Ответ:

- **Количество затронутых строк**
- Ассоциативный массив
- true/false
- Последний ID вставленной записи

Шаг 08:

Миссия 8

Виктор написал PDO-запрос INSERT и хочет получить id последней вставленной строки. Какой метод нужно вызвать?

Подсказка: Этот метод особенно полезен для таблиц с SERIAL или IDENTITY.

Ответ:

- **lastInsertId()**
- getLastId()
- lastRow()
- insertId()

Шаг 09:

Миссия 9

Анна использует pg_connect вместо PDO. Что вернёт pg_query при успешном SELECT?

Подсказка: Вспомните про ресурс результата в старом API PostgreSQL.

Ответ:

- **Ресурс результата (resource)**
- Ассоциативный массив
- Количество строк
- true/false

Шаг 10:

Миссия 10

 Сопоставьте функции pgSQL в PHP с их назначением.

Подсказка: Подумайте, какая функция для подключения, какая для запроса, а какая для получения строки.

Ответ:

- **pg_connect** — Создаёт соединение с PostgreSQL
- **pg_query** — Выполняет SQL-запрос
- **pg_fetch_assoc** — Возвращает строку в виде ассоциативного массива
- **pg_num_rows** — Возвращает количество строк в результате

🔮 Шаг 11:

Миссия 11

📖 Сопоставьте методы PDOStatement с их функциями.

Подсказка: Все они связаны с обработкой результата.

Ответ:

- **execute** — Выполняет подготовленный запрос
- **fetch** — Извлекает одну строку результата
- **fetchAll** — Извлекает все строки результата
- **rowCount** — Возвращает количество затронутых строк

🔮 Шаг 12:

Миссия 12

📖 Сопоставьте типы ошибок PDO с их описанием.

Подсказка: Обратите внимание, какой код отвечает за синтаксис, а какой — за соединение.

Ответ:

- **SQLSTATE[08006]** — Ошибка соединения с сервером
- **SQLSTATE[42601]** — Синтаксическая ошибка в SQL
- **SQLSTATE[23505]** — Нарушение уникального ограничения
- **SQLSTATE[22P02]** — Неверное преобразование типа данных

🔮 Шаг 13:

Миссия 13

📖 Сопоставьте методы PDO с их назначением.

Подсказка: Здесь есть метод для подготовки, выполнения запроса и управления атрибутами.

Ответ:

- **prepare** — Создаёт подготовленный запрос
- **query** — Выполняет запрос без параметров
- **exec** — Выполняет запрос без возврата результата
- **setAttribute** — Устанавливает параметры PDO

🔮 Шаг 14:

Миссия 14

📖 Сопоставьте константы PDO с их значением.

Подсказка: Подумайте, какие константы управляют режимом выборки и ошибок.

Ответ:

- **PDO::FETCH_ASSOC** — Возврат строки как ассоциативный массив
- **PDO::FETCH_NUM** — Возврат строки как нумерованный массив
- **PDO::ERRMODE_EXCEPTION** — Выбрасывать исключения при ошибках
- **PDO::ERRMODE_SILENT** — Ошибки без сообщений

🔮 Шаг 15:

Миссия 15

Расположите шаги подключения к PostgreSQL через PDO в правильном порядке.

Подсказка: Сначала создаётся объект PDO, потом включается обработка ошибок, далее выполняется запрос и извлекаются данные.

Ответ:

- Создание объекта PDO с DSN
- Установка режима ошибок через `setAttribute`
- Выполнение SQL-запроса
- Извлечение данных с помощью `fetch`/`fetchAll`

 **Шаг 16:**

Миссия 16

Расположите шаги работы с подготовленным запросом в PDO.

Подсказка: Вспомните: сначала готовим запрос, потом привязываем параметры, потом выполняем и извлекаем данные.

Ответ:

- `prepare()`
- `bindParam()` или `bindValue()`
- `execute()`
- `fetch()` или `fetchAll()`

Урок 4 - Подключение PHP к PostgreSQL и проверка соединения

 **Шаг 01:**

Миссия 1

Андрей подключает PHP к PostgreSQL. Какое расширение чаще всего используют для современных проектов?

Подсказка: Оно универсально, поддерживает разные СУБД и считается стандартом.

Ответ:

- **PDO**
- `mysql_connect`
- `mysqli`
- `pg_exec`

 **Шаг 02:**

Миссия 2

Елена запускает свой первый проект с PostgreSQL. Какой DSN правильный для подключения через PDO?

Подсказка: Формат включает драйвер, имя хоста, порт и базу данных.

Ответ:

- **`pgsql:host=localhost;port=5432;dbname=testdb`**
- `pgsql:dbname=testdb;user=postgres`
- `pgsql:port=5432;user=postgres`
- `host=localhost;port=5432;dbname=testdb;driver=pgsql`

 **Шаг 03:**

Миссия 3

Мария хочет проверить соединение с PostgreSQL. Какой код вернёт успешное сообщение при подключении?

Подсказка: Используйте try...catch и echo внутри catch при ошибке.

Ответ:

- **try { \$pdo = new PDO(...); echo 'OK'; } catch (PDOException \$e) { echo 'Error'; }**
- \$pdo = pg_connect(...); echo 'OK';
- if (\$pdo) echo 'OK'; else echo 'Error';
- echo 'Соединение установлено всегда';

Шаг 04:

Миссия 4

Игорь хочет настроить PDO так, чтобы при ошибке запросов выбрасывались исключения. Какой атрибут ему нужно установить?

Подсказка: Атрибут связан с обработкой ошибок (ERRMODE).

Ответ:

- **PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION**
- PDO::ATTR_TIMEOUT => 10
- PDO::ERRMODE_SILENT
- PDO::ATTR_CASE => PDO::CASE_UPPER

Шаг 05:

Миссия 5

Анна подключается к PostgreSQL. Какой порт используется по умолчанию?

Подсказка: Этот порт знают все админы PostgreSQL.

Ответ:

- **5432**
- 3306
- 1521
- 8080

Шаг 06:

Миссия 6

Алексей хочет проверить версию PostgreSQL после подключения. Какой SQL-запрос он должен выполнить?

Подсказка: Это специальная команда SELECT.

Ответ:

- **SELECT version();**
- SHOW version;
- GET VERSION();
- VERSION SELECT();

Шаг 07:

Миссия 7

 Сопоставьте атрибут PDO с его назначением.

Подсказка: Один атрибут отвечает за ошибки, другой за кодировку.

Ответ:

- **PDO::ATTR_ERRMODE** — Режим обработки ошибок
- **PDO::ATTR_TIMEOUT** — Максимальное время ожидания соединения
- **PDO::ATTR_DEFAULT_FETCH_MODE** — Режим получения строк (ассоциативный, числовой)
- **SET NAMES 'UTF8'** — Установка кодировки соединения

Шаг 08:

Миссия 8

 Сопоставьте функцию с её назначением в работе с PostgreSQL.

Подсказка: Обратите внимание на `pg_connect` и `pg_close`.

Ответ:

- **pg_connect** — Устанавливает соединение с базой данных
- **pg_close** — Закрывает соединение
- **pg_query** — Выполняет SQL-запрос
- **pg_fetch_assoc** — Извлекает строку результата в виде массива

Шаг 09:

Миссия 9

 Сопоставьте тип ошибки с её примером.

Подсказка: Подумайте о синтаксисе, доступе и сети.

Ответ:

- **Ошибка авторизации** — Неверный пароль пользователя
- **Сетевая ошибка** — Сервер PostgreSQL недоступен
- **Ошибка синтаксиса SQL** — Пропущено ключевое слово `SELECT`
- **Ошибка прав доступа** — Пользователь не имеет прав на таблицу

Шаг 10:

Миссия 10

 Сопоставьте драйвер с его особенностью.

Подсказка: `PDO` — универсальный, `pg_connect` — специфичный.

Ответ:

- **PDO** — Поддерживает разные СУБД
- **pg_connect** — Нативная функция PostgreSQL
- **mysqli** — Работает только с MySQL
- **ODBC** — Универсальный драйвер для разных систем

Шаг 11:

Миссия 11

 Сопоставьте компонент с его ролью в соединении.

Подсказка: Обратите внимание на `DSN` и порт.

Ответ:

- **DSN** — Строка с параметрами подключения
- **Порт** — Номер, через который идёт соединение
- **Пользователь** — Учётная запись для входа

- **Пароль** — Секрет для аутентификации

Шаг 12:

Миссия 12

Расположите шаги подключения к PostgreSQL через PDO в правильном порядке:

Подсказка: Сначала пишем DSN, затем создаём объект, потом настраиваем режим ошибок.

Ответ:

- Создание строки DSN
- Создание объекта PDO
- Установка атрибутов соединения
- Проверка соединения через try...catch

Шаг 13:

Миссия 13

Расположите шаги проверки соединения через pg_connect:

Подсказка: Сначала пишем строку, потом проверяем результат.

Ответ:

- Создание строки подключения
- Вызов pg_connect()
- Проверка возвращаемого значения
- Вывод сообщения об успехе или ошибке

Шаг 14:

Миссия 14

Расположите шаги обработки ошибки PDOException:

Подсказка: Сначала пробуем подключение, затем ловим ошибку.

Ответ:

- Открытие блока try
- Создание PDO объекта
- Ловля ошибки в catch
- Вывод сообщения пользователю

Шаг 15:

Миссия 15

Расположите шаги установки кодировки соединения UTF-8 в правильном порядке:

Подсказка: Нужно установить атрибут сразу после подключения.

Ответ:

- Создание PDO объекта
- Вызов метода exec()
- Выполнение SET NAMES 'UTF8'
- Проверка корректности кодировки

Шаг 16:

Миссия 16

Расположите шаги подключения нескольких пользователей к PostgreSQL:

Подсказка: У каждого пользователя свои логин и пароль, но общая база.

Ответ:

- Создание разных DSN
- Создание объектов PDO для каждого
- Установка атрибутов ошибок
- Проверка соединения каждого пользователя

Урок 5 - Основы работы с запросами в PostgreSQL

Шаг 01:

Миссия 1

Алексей пишет первое приложение на PHP и хочет извлечь данные из таблицы users . Какой SQL-запрос вернёт все записи?

Подсказка: Начинайте с ключевого слова SELECT .

Ответ:

- **SELECT * FROM users;**
- GET ALL FROM users;
- FETCH users;
- SHOW users;

Шаг 02:

Миссия 2

Марина анализирует базу и хочет выбрать только имена пользователей. Какой запрос ей подойдёт?

Подсказка: Указывайте конкретные колонки после SELECT.

Ответ:

- **SELECT name FROM users;**
- SELECT * FROM name;
- SELECT users.name();
- GET name IN users;

Шаг 03:

Миссия 3

Дмитрий хочет получить только тех пользователей, у которых возраст больше 18. Какой запрос верный?

Подсказка: Используйте WHERE для фильтрации.

Ответ:

- **SELECT * FROM users WHERE age > 18;**
- SELECT age > 18 FROM users;
- GET * FROM users age > 18;
- SELECT * users age>18;

Шаг 04:

Миссия 4

Ольга сортирует список книг по году издания. Какой запрос правильный?

Подсказка: Для сортировки используйте ORDER BY.

Ответ:

- **SELECT * FROM books ORDER BY year;**
- SELECT * FROM books SORT year;
- SELECT * FROM books BY year;
- SELECT * FROM books GROUP year;

Шаг 05:

Миссия 5

Иван ищет самого старшего сотрудника. Какой агрегатной функцией он воспользуется?

Подсказка: Вспомните функции COUNT, MAX, MIN, AVG.

Ответ:

- **SELECT MAX(age) FROM employees;**
- SELECT AVG(age) FROM employees;
- SELECT MIN(age) FROM employees;
- SELECT COUNT(age) FROM employees;

Шаг 06:

Миссия 6

Светлана хочет узнать количество студентов в таблице. Какой запрос правильный?

Подсказка: Для подсчёта строк используйте COUNT.

Ответ:

- **SELECT COUNT(*) FROM students;**
- SELECT SUM(*) FROM students;
- SELECT TOTAL(students);
- SELECT LENGTH(students);

Шаг 07:

Миссия 7

Никита ищет всех клиентов, у которых имя начинается на букву «А». Какой оператор используется?

Подсказка: Для шаблонов применяйте LIKE.

Ответ:

- **SELECT * FROM clients WHERE name LIKE 'A%';**
- SELECT * FROM clients WHERE name = 'A*';
- SELECT * FROM clients WHERE name MATCH 'A%';
- SELECT * FROM clients WHERE name START 'A';

Шаг 08:

Миссия 8

Василий делает отчёт и хочет объединить таблицы orders и customers . Какой оператор объединения он использует?

Подсказка: JOIN помогает связать таблицы по ключам.

Ответ:

- **SELECT * FROM orders JOIN customers ON orders.customer_id = customers.id;**
- **SELECT * FROM orders MERGE customers;**
- **SELECT * FROM orders WITH customers;**
- **SELECT * FROM orders UNION customers;**

Шаг 09:

Миссия 9

 Сопоставьте SQL-команду с её назначением.

Подсказка: Подумайте, какие команды читают данные, а какие изменяют.

Ответ:

- **SELECT** — Извлечение данных из таблицы.
- **INSERT** — Добавление новых строк.
- **UPDATE** — Изменение существующих записей.
- **DELETE** — Удаление строк из таблицы.

Шаг 10:

Миссия 10

 Сопоставьте тип фильтрации с правильным примером.

Подсказка: Сравните, где используются операторы = , > и LIKE .

Ответ:

- **Точное сравнение** — WHERE age = 30
- **Диапазонное условие** — WHERE age > 18
- **Шаблонное совпадение** — WHERE name LIKE 'A%'
- **Неравенство** — WHERE salary <> 50000

Шаг 11:

Миссия 11

 Сопоставьте функцию с её назначением в SQL.

Подсказка: Одна считает количество, другая вычисляет максимум и минимум.

Ответ:

- **COUNT** — Подсчёт количества строк.
- **MAX** — Поиск наибольшего значения.
- **MIN** — Поиск наименьшего значения.
- **AVG** — Вычисление среднего значения.

Шаг 12:

Миссия 12

 Сопоставьте тип объединения таблиц с его результатом.

Подсказка: Подумайте, чем отличается INNER от LEFT.

Ответ:

- **INNER JOIN** — Возвращает только совпадающие записи.
- **LEFT JOIN** — Возвращает все строки из левой таблицы и совпадения из правой.
- **RIGHT JOIN** — Возвращает все строки из правой таблицы и совпадения из левой.

- **FULL JOIN** — Возвращает все строки обеих таблиц.

Шаг 13:

Миссия 13

 Сопоставьте SQL-оператор с реальной задачей программиста.

Подсказка: Вспомните повседневные действия: добавить, удалить, обновить, выбрать.

Ответ:

- **INSERT** — Добавление нового заказа.
- **DELETE** — Удаление уволенного сотрудника.
- **UPDATE** — Изменение цены товара.
- **SELECT** — Получение списка клиентов.

Шаг 14:

Миссия 14

Расположите шаги выполнения SELECT-запроса в правильном порядке.

Подсказка: Сначала выбираем таблицу, потом фильтруем, затем сортируем.

Ответ:

- FROM
- WHERE
- ORDER BY
- SELECT

Шаг 15:

Миссия 15

Расположите этапы работы с данными: от вставки до удаления.

Подсказка: Подумайте, как обычно строится цикл работы: добавить → обновить → прочитать → удалить.

Ответ:

- INSERT
- UPDATE
- SELECT
- DELETE

Шаг 16:

Миссия 16

Расположите части SQL-запроса в правильном порядке.

Подсказка: Вначале указываем SELECT, потом FROM и условия.

Ответ:

- SELECT
- FROM
- WHERE
- ORDER BY

Шаг 17:

Миссия 17

Расположите действия администратора базы данных при проверке сложного запроса.

Подсказка: Сначала анализ структуры, потом выполнение, затем проверка результатов.

Ответ:

- Изучение структуры таблиц
- Составление запроса
- Выполнение запроса
- Анализ полученных данных

Модуль 2 - Основы работы с SQL в PostgreSQL

Урок 1 - Синтаксис SQL в PostgreSQL

Шаг 01:

Миссия 1

Молодой разработчик Лена подключается к PostgreSQL и хочет получить все строки из таблицы employees. Какой оператор она должна использовать?

Подсказка: Вспомните базовый оператор выборки данных.

Ответ:

- **SELECT * FROM employees;**
- SHOW * FROM employees;
- GET * FROM employees;
- READ * FROM employees;

Шаг 02:

Миссия 2

Андрей хочет удалить таблицу test_data, чтобы освободить место. Какой SQL-запрос ему нужен?

Подсказка: Здесь не про строки, а про удаление всей структуры таблицы.

Ответ:

- **DROP TABLE test_data;**
- DELETE TABLE test_data;
- REMOVE TABLE test_data;
- TRUNCATE TABLE test_data;

Шаг 03:

Миссия 3

Разработчик Даша пишет запрос для вставки новой записи в таблицу customers. Какой синтаксис верный?

Подсказка: Здесь важно помнить о ключевом слове INSERT.

Ответ:

- **INSERT INTO customers (name, email) VALUES ('Ivan', 'ivan@test.com');**
- ADD INTO customers (name, email) VALUES ('Ivan', 'ivan@test.com');
- UPDATE customers (name, email) VALUES ('Ivan', 'ivan@test.com');

- `PUT INTO customers (name, email) VALUES ('Ivan', 'ivan@test.com');`

Шаг 04:

Миссия 4

Сергей хочет изменить email пользователя в таблице users. Какой оператор применяется для изменения данных?

Подсказка: Подумайте, какой оператор меняет строки, а не создаёт или удаляет.

Ответ:

- **`UPDATE users SET email='new@mail.com' WHERE id=1;`**
- `CHANGE users SET email='new@mail.com' WHERE id=1;`
- `ALTER users SET email='new@mail.com' WHERE id=1;`
- `REPLACE users SET email='new@mail.com' WHERE id=1;`

Шаг 05:

Миссия 5

Марина пишет запрос, чтобы выбрать только уникальные города из таблицы addresses. Что нужно использовать?

Подсказка: Этот оператор убирает дубликаты.

Ответ:

- **`SELECT DISTINCT city FROM addresses;`**
- `SELECT UNIQUE city FROM addresses;`
- `SELECT DIFFERENT city FROM addresses;`
- `SELECT city FROM addresses GROUP;`

Шаг 06:

Миссия 6

Фронтендер Олег учится писать SQL-запросы. Какой оператор вернёт количество строк в таблице products?

Подсказка: Вспомните агрегатные функции.

Ответ:

- **`SELECT COUNT(*) FROM products;`**
- `SELECT SUM(*) FROM products;`
- `SELECT NUMBER(*) FROM products;`
- `SELECT TOTAL(*) FROM products;`

Шаг 07:

Миссия 7

 Задача: сопоставьте SQL-команды с их назначением.

Подсказка: Вспомните CRUD — Create, Read, Update, Delete.

Ответ:

- **`INSERT`** — Добавление новых записей
- **`SELECT`** — Выборка данных
- **`UPDATE`** — Изменение существующих записей
- **`DELETE`** — Удаление записей

🔮 Шаг 08:

Миссия 8

Разработчик Петя учит типы данных PostgreSQL. Сопоставьте типы с примерами.

Подсказка: Числа — числа, текст — строки.

Ответ:

- **INTEGER** — 42
- **TEXT** — 'Привет'
- **BOOLEAN** — TRUE
- **DATE** — '2025-09-29'

🔮 Шаг 09:

Миссия 9

Сопоставьте ключевые слова с их функциями в SQL.

Подсказка: Вспомните, что WHERE фильтрует, а ORDER BY сортирует.

Ответ:

- **WHERE** — Фильтрация строк
- **ORDER BY** — Сортировка результатов
- **GROUP BY** — Группировка строк
- **HAVING** — Фильтрация групп

🔮 Шаг 10:

Миссия 10

📄 Сопоставьте агрегатные функции с их назначением.

Подсказка: COUNT считает, AVG — среднее.

Ответ:

- **COUNT** — Подсчёт количества строк
- **SUM** — Суммирование значений
- **AVG** — Среднее значение
- **MAX** — Максимальное значение

🔮 Шаг 11:

Миссия 11

Сопоставьте команды изменения структуры с их назначением.

Подсказка: DROP удаляет, ALTER меняет.

Ответ:

- **CREATE TABLE** — Создание новой таблицы
- **ALTER TABLE** — Изменение структуры таблицы
- **DROP TABLE** — Удаление таблицы
- **TRUNCATE TABLE** — Очистка таблицы

🔮 Шаг 12:

Миссия 12

📄 Сопоставьте виды связей между таблицами с их описанием.

Подсказка: Один ко многим — классика отношений пользователей и заказов.

Ответ:

- **ONE TO ONE** — Один к одному
- **ONE TO MANY** — Один ко многим
- **MANY TO MANY** — Многие ко многим
- **SELF JOIN** — Связь таблицы самой с собой

Шаг 13:

Миссия 13

Поставьте шаги выполнения SELECT-запроса в правильном порядке.

Подсказка: Сначала выбираем источник, потом фильтруем, потом сортируем.

Ответ:

- FROM
- WHERE
- GROUP BY
- HAVING
- SELECT
- ORDER BY

Шаг 14:

Миссия 14

Алексей только начинает работать с PostgreSQL и хочет вывести все записи из таблицы users . Какой запрос будет правильным?

Подсказка: Для получения всех данных используйте ключевое слово SELECT и звёздочку.

Ответ:

- **SELECT * FROM users;**
- SHOW * FROM users;
- GET ALL users;
- EXTRACT users;

Шаг 15:

Миссия 15

Марина хочет узнать только имена студентов из таблицы students . Какой запрос ей подойдёт?

Подсказка: После SELECT можно указывать конкретные колонки.

Ответ:

- **SELECT name FROM students;**
- SELECT * FROM name;
- GET name FROM students;
- SELECT column(name) students;

Шаг 16:

Миссия 16

Дмитрий хочет получить всех пользователей старше 21 года из таблицы users . Какой запрос он должен использовать?

Подсказка: Для фильтрации применяют оператор WHERE.

Ответ:

- **SELECT * FROM users WHERE age > 21;**
- SELECT users WHERE age > 21;
- SELECT age > 21 FROM users;
- GET * FROM users age > 21;

 **Шаг 17:**

Миссия 17

Ольга анализирует таблицу books и хочет вывести список книг, отсортированный по названию. Какой запрос верный?

Подсказка: Для сортировки используйте ORDER BY.

Ответ:

- **SELECT * FROM books ORDER BY title;**
- SELECT * FROM books SORT title;
- SELECT * FROM books GROUP BY title;
- SELECT * FROM books SHOW title;

 **Шаг 18:**

Миссия 18

Иван ищет самого младшего сотрудника. Какую агрегатную функцию он применит к полю age ?

Подсказка: Для нахождения минимального значения есть специальная функция.

Ответ:

- **SELECT MIN(age) FROM employees;**
- SELECT MAX(age) FROM employees;
- SELECT COUNT(age) FROM employees;
- SELECT AVG(age) FROM employees;

 **Шаг 19:**

Миссия 19

Светлана подсчитывает количество заказов в таблице orders . Какой запрос верный?

Подсказка: Для подсчёта строк используйте COUNT.

Ответ:

- **SELECT COUNT(*) FROM orders;**
- SELECT TOTAL(*) FROM orders;
- SELECT LENGTH(*) FROM orders;
- SELECT SUM(*) FROM orders;

Урок 2 - Создание и оптимизация таблиц в PostgreSQL

 **Шаг 01:**

Миссия 1

Молодой программист Саша впервые запускает PostgreSQL и хочет создать таблицу для хранения пользователей. Что он должен сделать в первую очередь?

Подсказка: Прежде чем вставлять данные, нужно подготовить структуру.

Ответ:

- **Написать команду CREATE TABLE**
- Сразу вставить данные через INSERT
- Создать индекс
- Удалить базу данных

🔮 **Шаг 02:**

Миссия 2

Анна проектирует таблицу для заказов. Какой тип данных в PostgreSQL лучше выбрать для хранения уникального номера заказа?

Подсказка: Подумайте о числовом автоинкременте.

Ответ:

- **SERIAL**
- TEXT
- BOOLEAN
- DATE

🔮 **Шаг 03:**

Миссия 3

Игорь работает в стартапе и хочет хранить адреса электронной почты пользователей. Какой тип данных лучше использовать?

Подсказка: Этот тип хранит строки разной длины.

Ответ:

- **VARCHAR**
- INTEGER
- BOOLEAN
- REAL

🔮 **Шаг 04:**

Миссия 4

Компания Олега хранит даты регистрации пользователей. Какой тип данных стоит выбрать?

Подсказка: Этот тип специально создан для календарных значений.

Ответ:

- **DATE**
- VARCHAR
- SERIAL
- BOOLEAN

🔮 **Шаг 05:**

Миссия 5

Виктор проектирует таблицу для товаров и хочет, чтобы название не могло быть пустым. Какое ограничение следует добавить?

Подсказка: Это ограничение запрещает пустые значения.

Ответ:

- **NOT NULL**
- **DEFAULT**
- **CHECK**
- **UNIQUE**

🔗 **Шаг 06:**

Миссия 6

Марина хочет, чтобы в таблице email был уникальным у каждого пользователя. Какое ограничение ей стоит использовать?

Подсказка: Это ограничение проверяет уникальность значений в колонке.

Ответ:

- **UNIQUE**
- **NOT NULL**
- **PRIMARY KEY**
- **DEFAULT**

🔗 **Шаг 07:**

Миссия 7

Сергей оптимизирует базу данных и хочет понять разницу между типами данных. Сопоставьте тип с его описанием.

Подсказка: Вспомните, какие типы отвечают за числа, строки и даты.

Ответ:

- **INTEGER** — Хранит целые числа
- **VARCHAR** — Хранит строки ограниченной длины
- **DATE** — Хранит календарные даты
- **BOOLEAN** — Хранит логические значения истина/ложь

🔗 **Шаг 08:**

Миссия 8

Лена работает с таблицей сотрудников. Сопоставьте колонку с правильным ограничением.

Подсказка: Подумайте, где важна уникальность, а где нельзя оставлять пустые значения.

Ответ:

- **id** — **PRIMARY KEY**
- **email** — **UNIQUE**
- **name** — **NOT NULL**
- **salary** — **CHECK (salary > 0)**

🔗 **Шаг 09:**

Миссия 9

В команде обсуждают индексы. Сопоставьте тип индекса с его назначением.

Подсказка: Индексы ускоряют поиск, но бывают разные.

Ответ:

- **B-Tree** — Оптимален для сравнения чисел и строк

- **GIN** — Эффективен для полнотекстового поиска
- **BRIN** — Полезен для больших таблиц с упорядоченными данными
- **Hash** — Хорош для поиска по равенству

🔮 Шаг 10:

Миссия 10

Андрей анализирует запросы и хочет знать, как лучше организовать данные. Сопоставьте ситуацию с типом ключа.

Подсказка: Вспомните, что первичный ключ определяет уникальность, а внешний — связи.

Ответ:

- **id сотрудника** — PRIMARY KEY
- **id отдела** — FOREIGN KEY
- **email** — UNIQUE
- **имя** — NOT NULL

🔮 Шаг 11:

Миссия 11

Кирилл делает проект для интернет-магазина. Сопоставьте колонку с оптимальным типом данных.

Подсказка: Важна практичность: цены — числа с плавающей точкой, даты — специальные типы.

Ответ:

- **price** — NUMERIC
- **created_at** — TIMESTAMP
- **is_active** — BOOLEAN
- **title** — VARCHAR

🔮 Шаг 12:

Миссия 12

Юля проверяет знания коллег по оптимизации. Сопоставьте команду SQL с её действием.

Подсказка: CREATE создаёт, ALTER изменяет, DROP удаляет.

Ответ:

- **CREATE TABLE** — Создаёт таблицу
- **ALTER TABLE** — Изменяет структуру таблицы
- **DROP TABLE** — Удаляет таблицу
- **CREATE INDEX** — Создаёт индекс

🔮 Шаг 13:

Миссия 13

Расположите шаги создания таблицы в правильном порядке.

Подсказка: Сначала выбирается база, затем пишется структура, потом накладываются ограничения и, наконец, тестируется.

Ответ:

- Выбрать базу данных
- Написать команду CREATE TABLE
- Добавить ограничения (NOT NULL, UNIQUE и др.)

- Вставить тестовые данные

🔗 Шаг 14:

Миссия 14

Поставьте в правильном порядке шаги оптимизации таблицы.

Подсказка: Сначала анализируем, потом создаём индекс, затем проверяем и документируем.

Ответ:

- Анализировать медленные запросы
- Создать индекс
- Проверить ускорение запросов
- Задокументировать изменения

🔗 Шаг 15:

Миссия 15

Выстройте правильную последовательность работы с внешними ключами.

Подсказка: Сначала таблицы должны существовать, затем создаётся связь, а после — проверка и наполнение данными.

Ответ:

- Создать родительскую таблицу
- Создать дочернюю таблицу
- Добавить FOREIGN KEY
- Проверить корректность связей

🔗 Шаг 16:

Миссия 16

Выберите правильный порядок действий при изменении структуры таблицы.

Подсказка: Сначала анализируем изменения, затем создаём резервную копию, вносим правки и проверяем.

Ответ:

- Проанализировать необходимые изменения
- Создать резервную копию таблицы
- Выполнить ALTER TABLE
- Протестировать изменения

🔗 Шаг 17:

Миссия 17

Расположите шаги проектирования базы данных в правильном порядке.

Подсказка: Логика: сначала моделирование, потом нормализация, затем создание таблиц и индексов.

Ответ:

- Составить ER-диаграмму
- Применить нормализацию
- Создать таблицы в PostgreSQL
- Добавить индексы

Урок 3 - Индексы и их роль в производительности

Шаг 01:

Миссия 1

Молодой разработчик Саша заметил, что его запросы в PostgreSQL выполняются очень медленно. С чего стоит начать оптимизацию?

Подсказка: Иногда решение лежит в правильной структуре данных и индексах.

Ответ:

- **Проверить наличие и использование индексов**
- Удалить все таблицы и создать заново
- Добавить больше JOIN без анализа
- Игнорировать проблему

Шаг 02:

Миссия 2

Анна проектирует поиск по email. Какой индекс лучше всего использовать?

Подсказка: Подумайте о типе индекса, который подходит для равенства строк.

Ответ:

- **B-Tree**
- GIN
- BRIN
- GiST

Шаг 03:

Миссия 3

Игорь делает полнотекстовый поиск по описанию товаров. Какой индекс ускорит такие запросы?

Подсказка: Этот индекс часто применяется для текстовых полей и поиска слов.

Ответ:

- **GIN**
- Hash
- BRIN
- B-Tree

Шаг 04:

Миссия 4

Компания Олега хранит огромный лог в миллионы строк. Какой индекс экономичнее всего использовать для анализа диапазонов по времени?

Подсказка: Подходит для упорядоченных больших наборов данных.

Ответ:

- **BRIN**
- GIN
- Hash
- B-Tree

🔮 Шаг 05:

Миссия 5

Виктор ищет пользователей по точному совпадению их ID. Какой индекс будет эффективнее?

Подсказка: Идеален для равенства числовых значений.

Ответ:

- **Hash**
- BRIN
- GIN
- GiST

🔮 Шаг 06:

Миссия 6

Марина добавляет индекс к таблице заказов. Что произойдет с вставкой новых данных?

Подсказка: Индекс помогает SELECT, но INSERT становится немного тяжелее.

Ответ:

- **Вставка замедлится, так как индекс нужно обновлять**
- Вставка ускорится
- На вставку это никак не влияет
- INSERT будет невозможен

🔮 Шаг 07:

Миссия 7

Сергей изучает разные типы индексов в PostgreSQL. Сопоставьте тип с его назначением.

Подсказка: Каждый индекс решает свою задачу.

Ответ:

- **B-Tree** — Подходит для диапазонных и равенственных запросов
- **GIN** — Эффективен для полнотекстового поиска
- **BRIN** — Идеален для больших таблиц с упорядоченными данными
- **Hash** — Хорош для поиска по точному равенству

🔮 Шаг 08:

Миссия 8

Лена проверяет, где индексы реально нужны. Сопоставьте запрос с индексом, который его ускорит.

Подсказка: Подумайте, где поиск по диапазону, где по тексту.

Ответ:

- **SELECT * FROM users WHERE id = 100** — Hash
- **SELECT * FROM logs WHERE created_at > NOW() - interval '1 day'** — BRIN
- **SELECT * FROM products WHERE name LIKE 'A%'** — B-Tree
- **SELECT * FROM articles WHERE to_tsvector(body) @@ to_tsquery('postgres')** — GIN

🔮 Шаг 09:

Миссия 9

Иван экспериментирует с индексами. Сопоставьте команду с её действием.

Подсказка: CREATE создаёт, DROP удаляет, REINDEX обновляет.

Ответ:

- **CREATE INDEX idx_name ON users(name)** — Создать индекс
- **DROP INDEX idx_name** — Удалить индекс
- **REINDEX INDEX idx_name** — Перестроить индекс
- **ALTER INDEX idx_name RENAME TO idx_new** — Переименовать индекс

Шаг 10:

Миссия 10

Кирилл хочет знать, какие запросы используют индекс. Сопоставьте команду с её назначением.

Подсказка: EXPLAIN помогает увидеть план выполнения.

Ответ:

- **EXPLAIN SELECT * FROM users WHERE id=1** — Показать план запроса
- **EXPLAIN ANALYZE SELECT * FROM users WHERE id=1** — Показать план и время выполнения
- **VACUUM** — Очистить и оптимизировать таблицу
- **ANALYZE** — Обновить статистику для оптимизатора

Шаг 11:

Миссия 11

Юля проверяет знания стажёров. Сопоставьте индекс с примером его применения.

Подсказка: Подумайте о разнице между текстом, датами и точными значениями.

Ответ:

- **GIN** — Поиск по тексту статьи
- **B-Tree** — Фильтрация заказов по сумме > 1000
- **BRIN** — Запросы к логу за последние сутки
- **Hash** — Поиск пользователя по ID

Шаг 12:

Миссия 12

Расставьте шаги создания индекса в правильном порядке.

Подсказка: Сначала решаем задачу, потом пишем команду, проверяем и анализируем.

Ответ:

- Определить проблему в запросе
- Написать CREATE INDEX
- Запустить EXPLAIN для проверки
- Проверить улучшение скорости

Шаг 13:

Миссия 13

Выстройте порядок действий при удалении ненужного индекса.

Подсказка: Важно сначала убедиться, что индекс не используется.

Ответ:

- Проверить использование индекса

- Протестировать запросы без индекса
- Выполнить DROP INDEX
- Документировать изменения

Шаг 14:

Миссия 14

Определите правильный порядок оптимизации запросов.

Подсказка: Сначала пишется запрос, потом проверяется план, затем добавляется индекс.

Ответ:

- Написать запрос
- Посмотреть план через EXPLAIN
- Добавить индекс
- Сравнить производительность

Шаг 15:

Миссия 15

Расставьте шаги в процессе перестроения индексов.

Подсказка: Используется REINDEX, но важно заранее оценить нагрузку.

Ответ:

- Определить фрагментированный индекс
- Создать резервную копию
- Выполнить REINDEX
- Проверить улучшение

Шаг 16:

Миссия 16

Поставьте шаги проектирования индекса в правильный порядок.

Подсказка: Сначала формулируется задача, потом подбирается тип индекса и только после этого создаётся и тестируется.

Ответ:

- Определить цель индексации
- Выбрать тип индекса
- Создать индекс
- Проверить результат

Урок 4 - Использование функции и представлений

Шаг 01:

Миссия 1

Аня пишет функцию в PostgreSQL, чтобы автоматически вычислять скидку для клиентов. Какое ключевое слово используется для объявления функции?

Подсказка: Вспомни базовый синтаксис SQL: сначала идёт CREATE.

Ответ:

- **CREATE FUNCTION**
- CREATE VIEW

- ALTER FUNCTION
- DECLARE FUNCTION

🔮 Шаг 02:

Миссия 2

Сергей хочет создать функцию, которая возвращает значение. Какой оператор в конце функции указывает, что именно нужно вернуть?

Подсказка: Этот оператор есть почти во всех языках программирования.

Ответ:

- **RETURN**
- OUTPUT
- RESULT
- BACK

🔮 Шаг 03:

Миссия 3

Иван заметил, что функция работает, но её нужно удалить. Какой командой в PostgreSQL удаляется функция?

Подсказка: Она симметрична CREATE FUNCTION.

Ответ:

- **DROP FUNCTION**
- DELETE FUNCTION
- REMOVE FUNCTION
- TRUNCATE FUNCTION

🔮 Шаг 04:

Миссия 4

Маша решила упростить доступ к сложному запросу и создать представление. Как правильно объявить представление?

Подсказка: Это похоже на объявление виртуальной таблицы.

Ответ:

- **CREATE VIEW view_name AS SELECT ...**
- CREATE TABLE view_name AS SELECT ...
- DECLARE VIEW view_name
- INSERT VIEW view_name

🔮 Шаг 05:

Миссия 5

Петя хочет узнать, можно ли обновлять данные через представления. Когда это возможно?

Подсказка: Вспомни: простота структуры играет роль.

Ответ:

- **Если представление основано на одной таблице без агрегатов**
- Если представление содержит GROUP BY
- Если представление объединяет несколько таблиц JOIN
- Если представление использует подзапросы с UNION

💡 Шаг 06:

Миссия 6

Юля хочет скрыть сложную бизнес-логику от пользователей. Что ей лучше использовать?

Подсказка: Представление похоже на маску для данных.

Ответ:

- **Представление (VIEW)**
- Функция, возвращающая void
- Удаление таблиц
- Курсор

💡 Шаг 07:

Миссия 7

📖 Задача: Сопоставьте тип функции с её описанием.

Подсказка: Подумай о различии между функциями, которые что-то возвращают, и теми, которые действуют как процедуры.

Ответ:

- **Функция, возвращающая scalar** — Возвращает одно значение (например, число или строку).
- **Функция, возвращающая TABLE** — Возвращает результат в виде таблицы.
- **Функция без возвращаемого значения** — Выполняет действие, но ничего не возвращает.
- **Агрегатная функция** — Обрабатывает несколько строк и возвращает одно итоговое значение.

💡 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте команды с их назначением.

Подсказка: CREATE — для создания, DROP — для удаления.

Ответ:

- **CREATE FUNCTION** — Создать новую функцию.
- **DROP VIEW** — Удалить существующее представление.
- **ALTER FUNCTION** — Изменить параметры функции.
- **CREATE VIEW** — Создать новое представление.

💡 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте примеры функций с их результатами.

Подсказка: Представь, что эти функции запускают простые действия.

Ответ:

- **SELECT now();** — Возвращает текущую дату и время.
- **SELECT upper('php');** — Возвращает строку 'PHP'.
- **SELECT length('PostgreSQL');** — Возвращает число 10.
- **SELECT random();** — Возвращает случайное число от 0 до 1.

🔮 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте тип представления с его особенностью.

Подсказка: Вспомни, что материализованное представление хранит данные, а обычное — нет.

Ответ:

- **Обычное представление** — Не хранит данные, строится при каждом запросе.
- **Материализованное представление** — Хранит результат выборки в базе.
- **Обновляемое представление** — Позволяет изменять данные напрямую.
- **Невозможное к обновлению представление** — Основано на агрегатах или сложных объединениях.

🔮 Шаг 11:

Миссия 11

📖 Задача: Сопоставьте преимущества функций и представлений.

Подсказка: Подумай, что связано с переиспользованием кода, а что с удобством выборок.

Ответ:

- **Функции** — Позволяют переиспользовать бизнес-логику.
- **Представления** — Упрощают доступ к сложным запросам.
- **Материализованные представления** — Повышают производительность при повторных запросах.
- **Обычные представления** — Не занимают дополнительного места в БД.

🔮 Шаг 12:

Миссия 12

Расположите шаги создания функции в PostgreSQL в правильном порядке:

1. Написание тела функции (BEGIN...END)
2. Определение возвращаемого типа
3. Использование RETURN для возврата результата
4. Использование CREATE FUNCTION с именем и параметрами

Подсказка: Сначала объявляется функция, затем тип, потом тело и возврат.

Ответ:

- Использование CREATE FUNCTION с именем и параметрами
- Определение возвращаемого типа
- Написание тела функции (BEGIN...END)
- Использование RETURN для возврата результата

🔮 Шаг 13:

Миссия 13

Расположите шаги работы с представлением в правильной последовательности:

1. Написать SELECT-запрос
2. Создать представление с CREATE VIEW
3. Использовать SELECT из представления
4. Удалить представление при ненужности

Подсказка: Подумай о жизненном цикле объекта: от создания до удаления.

Ответ:

- Написать SELECT-запрос
- Создать представление с CREATE VIEW
- Использовать SELECT из представления
- Удалить представление при ненадобности

Шаг 14:

Миссия 14

Расположите шаги обновления материализованного представления:

1. Выполнить команду REFRESH MATERIALIZED VIEW
2. Создать материализованное представление
3. Использовать данные из него в запросах
4. При необходимости обновить снова

Подсказка: Обновление всегда делается командой REFRESH.

Ответ:

- Создать материализованное представление
- Использовать данные из него в запросах
- Выполнить команду REFRESH MATERIALIZED VIEW
- При необходимости обновить снова

Шаг 15:

Миссия 15

Расположите шаги использования представления в PHP-приложении:

1. Написать SQL-запрос SELECT к представлению
2. Отправить запрос через PDO/pg_connect
3. Получить результат выборки
4. Отобразить данные пользователю

Подсказка: Логика та же, что и при работе с таблицами.

Ответ:

- Написать SQL-запрос SELECT к представлению
- Отправить запрос через PDO/pg_connect
- Получить результат выборки
- Отобразить данные пользователю

Шаг 16:

Миссия 16

Никита заметил, что функция возвращает таблицу, и он хочет использовать её в SELECT. Как правильно это сделать?

Подсказка: Функции, возвращающие TABLE, можно использовать почти как виртуальные таблицы.

Ответ:

- **SELECT * FROM function_name();**
- EXEC function_name();

- CALL function_name();
- USE function_name();

Урок 5 - Транзакции и блокировки в PostgreSQL

Шаг 01:

Миссия 1

Андрей работает с базой и хочет начать транзакцию. Какой командой это сделать в PostgreSQL?
Подсказка: Вспомни базовый синтаксис: все начинается с ключевого слова BEGIN.

Ответ:

- **BEGIN**
- START TRANSACTION
- OPEN TRANSACTION
- INIT TRANSACTION

Шаг 02:

Миссия 2

Мария хочет зафиксировать изменения после успешного выполнения SQL-запросов в транзакции. Какая команда это делает?

Подсказка: Думай о слове, означающем «подтвердить» или «сохранить».

Ответ:

- **COMMIT**
- ROLLBACK
- SAVEPOINT
- END

Шаг 03:

Миссия 3

Иван заметил ошибку в транзакции и хочет отменить все изменения. Какой командой это сделать?

Подсказка: Слово означает «откатить» или «отменить».

Ответ:

- **ROLLBACK**
- UNDO
- COMMIT
- ABORT

Шаг 04:

Миссия 4

Светлана хочет установить точку отката внутри транзакции. Как это называется в PostgreSQL?

Подсказка: Это позволяет откатываться частично, а не всю транзакцию целиком.

Ответ:

- **SAVEPOINT**
- CHECKPOINT
- ROLLBACK POINT

- MARK

Шаг 05:

Миссия 5

Петр хочет убедиться, что его транзакция изолирована от других. Какой уровень изоляции гарантирует, что никто не видит промежуточные изменения?

Подсказка: Подумай о слове, означающем «полная изоляция».

Ответ:

- **SERIALIZABLE**
- READ COMMITTED
- READ UNCOMMITTED
- REPEATABLE READ

Шаг 06:

Миссия 6

Анна хочет временно заблокировать строку для обновления, чтобы другой пользователь не мог её изменить. Какой оператор использовать?

Подсказка: Он работает в SELECT и защищает строки от изменений другими транзакциями.

Ответ:

- **SELECT ... FOR UPDATE**
- LOCK TABLE
- SET LOCK
- BLOCK ROW

Шаг 07:

Миссия 7

 Задача: Сопоставьте термин с его объяснением.

Подсказка: Подумайте, что делают команды транзакций и блокировок.

Ответ:

- **BEGIN** — Начало транзакции.
- **COMMIT** — Подтверждение всех изменений транзакции.
- **ROLLBACK** — Отмена всех изменений транзакции.
- **SAVEPOINT** — Создание точки отката внутри транзакции.

Шаг 08:

Миссия 8

 Задача: Сопоставьте уровень изоляции транзакции с его особенностью.

Подсказка: Подумайте о том, какие аномалии данных предотвращаются.

Ответ:

- **READ UNCOMMITTED** — Можно видеть неподтверждённые изменения других транзакций.
- **READ COMMITTED** — Можно видеть только подтверждённые изменения.
- **REPEATABLE READ** — Гарантирует, что повторные чтения будут одинаковыми.
- **SERIALIZABLE** — Полная изоляция, как будто транзакции выполняются последовательно.

Шаг 09:

Миссия 9

 Сопоставьте термин транзакции с реальным примером из работы программиста.

Подсказка: Подумайте, когда вы фиксируете изменения и когда откатываете их.

Ответ:

- **Транзакция** — Перевод денег между счетами с возможностью отмены всех действий.
- **COMMIT** — Подтверждение успешного выполнения перевода.
- **ROLLBACK** — Отмена перевода при ошибке.
- **SAVEPOINT** — Создание контрольной точки, чтобы отменить часть перевода.

Шаг 10:

Миссия 10

 Сопоставьте тип блокировки с примером использования в реальной системе.

Подсказка: Блокировки могут применяться к строкам или к таблицам целиком.

Ответ:

- **ROW LOCK** — Один пользователь редактирует заказ, другой ждет.
- **TABLE LOCK** — Обновление всей таблицы пользователей во время миграции.
- **SHARE LOCK** — Несколько аналитиков читают таблицу одновременно.
- **EXCLUSIVE LOCK** — Миграция базы запрещает любые операции другим транзакциям.

Шаг 11:

Миссия 11

 Сопоставьте команды транзакции с действиями в PHP-приложении.

Подсказка: Подумайте, какие действия соответствуют отправке запросов и обработке ошибок.

Ответ:

- **BEGIN** — Начало блока SQL-запросов через PDO.
- **COMMIT** — Подтверждение успешного выполнения всех запросов.
- **ROLLBACK** — Откат изменений при возникновении ошибки.
- **SAVEPOINT** — Создание точки, чтобы можно было откатить часть запросов.

Шаг 12:

Миссия 12

 Сопоставьте уровень изоляции с ситуацией в реальном приложении.

Подсказка: Подумайте, кто видит изменения других пользователей.

Ответ:

- **READ UNCOMMITTED** — Можно видеть изменения других пользователей до их подтверждения.
- **READ COMMITTED** — Можно видеть только подтверждённые изменения.
- **REPEATABLE READ** — Повторное чтение одних и тех же строк возвращает одинаковый результат.
- **SERIALIZABLE** — Транзакции выполняются последовательно, полная изоляция.

Шаг 13:

Миссия 13

Расположите шаги обработки транзакции в PHP с PostgreSQL в правильном порядке:

1. Начало транзакции
2. Выполнение SQL-запросов
3. Обработка ошибок
4. COMMIT или ROLLBACK
5. Закрытие соединения

Подсказка: Сначала всё открывается, потом выполняется, потом подтверждается или откатывается.

Ответ:

- Начало транзакции
- Выполнение SQL-запросов
- Обработка ошибок
- COMMIT или ROLLBACK
- Закрытие соединения

Шаг 14:

Миссия 14

Расположите шаги частичного отката через SAVEPOINT:

1. Создание SAVEPOINT
2. Выполнение ряда SQL-запросов
3. Обнаружение ошибки
4. ROLLBACK TO SAVEPOINT
5. Продолжение транзакции

Подсказка: Точка отката используется для отмены части действий, не всей транзакции.

Ответ:

- Создание SAVEPOINT
- Выполнение ряда SQL-запросов
- Обнаружение ошибки
- ROLLBACK TO SAVEPOINT
- Продолжение транзакции

Шаг 15:

Миссия 15

Расположите действия при блокировке строк для обновления:

1. Начало транзакции
2. SELECT ... FOR UPDATE
3. Изменение выбранных строк
4. COMMIT
5. Доступ другим транзакциям после COMMIT

Подсказка: Сначала блокируется строка, потом изменения, потом подтверждение.

Ответ:

- Начало транзакции
- SELECT ... FOR UPDATE
- Изменение выбранных строк

- COMMIT
- Доступ другим транзакциям после COMMIT

Шаг 16:

Миссия 16

Расположите шаги применения блокировки таблицы:

1. BEGIN
2. LOCK TABLE ... IN EXCLUSIVE MODE
3. Выполнение операций с таблицей
4. COMMIT
5. Разблокировка таблицы

Подсказка: Таблица блокируется перед операциями и разблокируется после COMMIT.

Ответ:

- BEGIN
- LOCK TABLE ... IN EXCLUSIVE MODE
- Выполнение операций с таблицей
- COMMIT
- Разблокировка таблицы

Модуль 3 - Оптимизация работы с PostgreSQL

Урок 1 - Профилирование и анализ SQL-запросов

Шаг 01:

Миссия 1

Аня только начала работать с PostgreSQL и хочет понять, как посмотреть время выполнения запроса. Что ей следует использовать?

Подсказка: В PostgreSQL есть встроенный инструмент для анализа запросов по времени выполнения.

Ответ:

- **EXPLAIN ANALYZE**
- SELECT NOW()
- VACUUM FULL
- CREATE INDEX

Шаг 02:

Миссия 2

Пётр заметил, что один запрос к таблице с миллионом строк работает слишком долго. Что первым делом поможет ускорить запрос?

Подсказка: Подумайте о структуре данных и индексах.

Ответ:

- **Создать индекс на колонку, используемую в фильтре WHERE**
- Добавить лишние JOIN к другим таблицам
- Увеличить размер блока в PostgreSQL
- Удалить все ограничения NOT NULL

Шаг 03:

Миссия 3

Катя замечает, что её запрос с JOIN работает медленно, хотя таблицы имеют индексы. Что ещё можно проверить?

Подсказка: Иногда индекс есть, но оптимизатор его не использует.

Ответ:

- Проверить план выполнения через EXPLAIN
- Удалить все индексы
- Сделать VACUUM ANALYZE
- Использовать LIMIT 1 для теста

Шаг 04:

Миссия 4

 Задача: Сопоставьте понятие с его объяснением

Понятия:

Ответ:

- EXPLAIN
- EXPLAIN ANALYZE
- VACUUM
- ANALYZE

Определения:

1. Показывает план выполнения запроса без его выполнения
2. Показывает план и фактическое время выполнения запроса
3. Удаляет неиспользуемые строки и освобождает место
4. Собирает статистику о таблицах для оптимизатора

Подсказка: Подумайте о том, что реально выполняет запрос, а что только показывает план.

- **EXPLAIN** — Показывает план выполнения запроса без его выполнения
- **EXPLAIN ANALYZE** — Показывает план и фактическое время выполнения запроса
- **VACUUM** — Удаляет неиспользуемые строки и освобождает место
- **ANALYZE** — Собирает статистику о таблицах для оптимизатора

Шаг 05:

Миссия 5

 Задача: Сопоставьте тип индекса с его особенностью

Понятия:

Ответ:

- B-tree
- Hash
- GIN
- BRIN

Определения:

1. Подходит для быстрого поиска по точному значению

2. Оптимален для диапазонного поиска и сортировки
3. Идеален для полнотекстового поиска и массивов
4. Эффективен для очень больших таблиц с физически упорядоченными данными

Подсказка: Подумайте о том, для каких сценариев каждый индекс создавался.

- **B-tree** — Оптимален для диапазонного поиска и сортировки
- **Hash** — Подходит для быстрого поиска по точному значению
- **GIN** — Идеален для полнотекстового поиска и массивов
- **BRIN** — Эффективен для очень больших таблиц с физически упорядоченными данными

Шаг 06:

Миссия 6

 Сопоставьте тип запроса с его влиянием на производительность

Понятия:

Ответ:

- SELECT с подзапросом
- JOIN больших таблиц
- Использование функции в WHERE
- LIMIT 1

Определения:

1. Может сильно замедлять запрос при больших данных
2. Часто приводит к полному сканированию таблицы, если нет индекса
3. Используется для быстрого ограничения выборки
4. Функция препятствует использованию индекса

Подсказка: Подумайте, что заставляет PostgreSQL сканировать все строки.

- **SELECT с подзапросом** — Может сильно замедлять запрос при больших данных
- **JOIN больших таблиц** — Часто приводит к полному сканированию таблицы, если нет индекса
- **LIMIT 1** — Используется для быстрого ограничения выборки
- **Использование функции в WHERE** — Функция препятствует использованию индекса

Шаг 07:

Миссия 7

Расположите шаги оптимизации запроса в правильном порядке:

1. Создание индексов
2. Проверка плана выполнения через EXPLAIN
3. Рефакторинг запроса (убрать подзапросы, функции в WHERE)
4. Тестирование скорости после изменений

Подсказка: Сначала смотрим, что не так, потом исправляем и проверяем результат.

Ответ:

- Проверка плана выполнения через EXPLAIN
- Рефакторинг запроса (убрать подзапросы, функции в WHERE)
- Создание индексов
- Тестирование скорости после изменений

Шаг 08:

Миссия 8

Расположите действия по оптимизации больших таблиц в PostgreSQL:

1. ANALYZE для обновления статистики
2. VACUUM для очистки неиспользуемых строк
3. Создание BRIN индексов
4. Проверка плана выполнения запросов

Подсказка: Подумайте, что должно идти перед тем, как база сможет использовать новые индексы эффективно.

Ответ:

- VACUUM для очистки неиспользуемых строк
- ANALYZE для обновления статистики
- Создание BRIN индексов
- Проверка плана выполнения запросов

Шаг 09:

Миссия 9

Иван хочет понять, сколько времени занимает каждый шаг сложного запроса. Какой инструмент PostgreSQL использовать?

Подсказка: Есть вариант, который реально выполняет запрос и показывает точные тайминги.

Ответ:

- **EXPLAIN ANALYZE**
- EXPLAIN
- VACUUM FULL
- CHECKPOINT

Шаг 10:

Миссия 10

Лена видит, что JOIN работает медленно. Какие действия помогут ускорить его работу?

Подсказка: Подумайте о том, какие индексы нужны для соединяемых колонок.

Ответ:

- **Создать индексы на колонки для JOIN**
- Удалить все индексы
- Сделать VACUUM FULL без индексов
- Использовать LIMIT 100

Шаг 11:

Миссия 11

 Сопоставьте действия DBA с их эффектом

Понятия:

Ответ:

- VACUUM
- ANALYZE

- CLUSTER
- REINDEX

Определения:

1. Упорядочивание таблицы по индексу для ускорения диапазонных запросов
2. Обновление статистики для оптимизатора
3. Восстановление поврежденных или фрагментированных индексов
4. Удаление «мертвых» строк и освобождение места

Подсказка: Подумайте, что помогает ускорить конкретный тип запроса, а что чистит таблицу.

- **VACUUM** — Удаление «мертвых» строк и освобождение места
- **ANALYZE** — Обновление статистики для оптимизатора
- **CLUSTER** — Упорядочивание таблицы по индексу для ускорения диапазонных запросов
- **REINDEX** — Восстановление поврежденных или фрагментированных индексов

Шаг 12:

Миссия 12

 Сопоставьте шаги профилирования SQL с инструментами PostgreSQL

Понятия:

Ответ:

- Проверка плана запроса
- Сбор статистики по таблице
- Физическая оптимизация данных
- Проверка времени выполнения

Определения:

1. EXPLAIN
2. ANALYZE
3. VACUUM / CLUSTER
4. EXPLAIN ANALYZE

Подсказка: Обратите внимание, какой инструмент реально меняет данные, а какой только анализирует.

- **Проверка плана запроса** — EXPLAIN
- **Сбор статистики по таблице** — ANALYZE
- **Физическая оптимизация данных** — VACUUM / CLUSTER
- **Проверка времени выполнения** — EXPLAIN ANALYZE

Шаг 13:

Миссия 13

 Сопоставьте этапы анализа медленного запроса с их инструментами

Понятия:

Ответ:

- Сбор статистики
- Проверка плана выполнения
- Тестирование изменений
- Оптимизация запроса

Определения:

1. ANALYZE
2. EXPLAIN
3. Выполнение запроса и измерение времени
4. Рефакторинг SQL и индексов

Подсказка: Подумайте, какие действия помогают понять, где проблема, а какие её решают.

- **Сбор статистики** — ANALYZE
- **Проверка плана выполнения** — EXPLAIN
- **Тестирование изменений** — Выполнение запроса и измерение времени
- **Оптимизация запроса** — Рефакторинг SQL и индексов

Шаг 14:

Миссия 14

Расположите действия по оптимизации запросов с JOIN:

1. Проверка плана выполнения запроса
2. Создание индексов на ключевых колонках
3. Изменение запроса для уменьшения количества JOIN
4. Тестирование времени выполнения

Подсказка: Сначала анализируем, затем меняем, потом проверяем результат.

Ответ:

- Проверка плана выполнения запроса
- Создание индексов на ключевых колонках
- Изменение запроса для уменьшения количества JOIN
- Тестирование времени выполнения

Шаг 15:

Миссия 15

Расположите действия по оптимизации больших таблиц с частыми вставками:

1. VACUUM для очистки пространства
2. ANALYZE для обновления статистики
3. Создание или обновление индексов
4. Проверка времени выполнения запросов

Подсказка: Сначала освобождаем место, затем собираем статистику, потом улучшаем индексы и тестируем.

Ответ:

- VACUUM для очистки пространства
- ANALYZE для обновления статистики
- Создание или обновление индексов
- Проверка времени выполнения запросов

Шаг 16:

Миссия 16

Сергей хочет проверить, какой из нескольких индексов PostgreSQL использует при сложном запросе. Что лучше использовать?

Подсказка: Есть инструмент, который показывает план и реальное использование индексов.

Ответ:

- **EXPLAIN ANALYZE**
- EXPLAIN без ANALYZE
- VACUUM
- ANALYZE

Шаг 17:

Миссия 17

 Сопоставьте типы индексов с оптимальными сценариями использования

Понятия:

Ответ:

- B-tree
- GIN
- BRIN
- Hash

Определения:

1. Диапазонный поиск и сортировка
2. Полнотекстовый поиск и массивы
3. Очень большие таблицы с физическим порядком данных
4. Точные соответствия значений

Подсказка: Подумайте о размере таблиц и типе запросов.

- **B-tree** — Диапазонный поиск и сортировка
- **GIN** — Полнотекстовый поиск и массивы
- **BRIN** — Очень большие таблицы с физическим порядком данных
- **Hash** — Точные соответствия значений

Шаг 18:

Миссия 18

Расположите этапы анализа и оптимизации запроса, который использует сложные подзапросы:

1. Проверка плана запроса
2. Определение узких мест (slow operations)
3. Рефакторинг подзапросов и индексов
4. Тестирование скорости выполнения

Подсказка: Логическая последовательность: сначала анализ, потом исправления, потом проверка.

Ответ:

- Проверка плана запроса
- Определение узких мест (slow operations)
- Рефакторинг подзапросов и индексов
- Тестирование скорости выполнения

🔮 Шаг 19:

Миссия 19

Расположите шаги комплексной оптимизации большого проекта на PostgreSQL:

1. Проверка плана выполнения всех ключевых запросов
2. Сбор и обновление статистики таблиц
3. Оптимизация индексов и рефакторинг запросов
4. Тестирование производительности и мониторинг

Подсказка: Подумайте о цикле анализа, исправления и проверки.

Ответ:

- Проверка плана выполнения всех ключевых запросов
- Сбор и обновление статистики таблиц
- Оптимизация индексов и рефакторинг запросов
- Тестирование производительности и мониторинг

📖 Урок 2 - Индексация таблиц для ускорения запросов

🔮 Шаг 01:

Миссия 1

Алексей заметил, что SELECT по таблице с миллионом строк выполняется медленно. Что первым делом стоит сделать?

Подсказка: Подумайте, как ускоряют поиск по большим таблицам.

Ответ:

- **Создать индекс на колонку, используемую в WHERE**
- Увеличить размер таблицы
- Удалить все ограничения
- Переписать SELECT в UPDATE

🔮 Шаг 02:

Миссия 2

Мария добавила индекс, но JOIN двух таблиц всё ещё медленный. Что ей проверить первым?

Подсказка: Иногда индекс есть, но оптимизатор его не использует.

Ответ:

- **Проверить план выполнения через EXPLAIN**
- Удалить все индексы
- Создать триггер на таблицу
- Сделать VACUUM FULL без индексов

🔮 Шаг 03:

Миссия 3

Игорь хочет понять, какой индекс использовать для быстрого поиска по точному значению. Что ему подойдёт?

Подсказка: Разные типы индексов подходят для разных сценариев.

Ответ:

- **Hash индекс**

- BRIN индекс
- GIN индекс
- Поле VARCHAR без индекса

Шаг 04:

Миссия 4

 Сопоставьте тип индекса с его оптимальным использованием

Понятия:

Ответ:

- B-tree
- Hash
- GIN
- BRIN

Определения:

1. Диапазонный поиск и сортировка
2. Быстрый поиск по точному значению
3. Полнотекстовый поиск и массивы
4. Очень большие таблицы с физическим порядком данных

Подсказка: Подумайте о размере таблиц и типе запросов.

- **B-tree** — Диапазонный поиск и сортировка
- **Hash** — Быстрый поиск по точному значению
- **GIN** — Полнотекстовый поиск и массивы
- **BRIN** — Очень большие таблицы с физическим порядком данных

Шаг 05:

Миссия 5

 Сопоставьте термин и его объяснение

Понятия:

Ответ:

- EXPLAIN
- EXPLAIN ANALYZE
- VACUUM
- ANALYZE

Определения:

1. Показывает план выполнения запроса без его выполнения
2. Показывает план и фактическое время выполнения запроса
3. Удаляет «мертвые» строки и освобождает место
4. Обновляет статистику таблицы для оптимизатора

Подсказка: Подумайте, что реально выполняет запрос, а что только анализирует.

- **EXPLAIN** — Показывает план выполнения запроса без его выполнения
- **EXPLAIN ANALYZE** — Показывает план и фактическое время выполнения запроса
- **VACUUM** — Удаляет «мертвые» строки и освобождает место
- **ANALYZE** — Обновляет статистику таблицы для оптимизатора

Шаг 06:

Миссия 6

 Сопоставьте виды индексов с их особенностями

Понятия:

Ответ:

- Уникальный индекс
- Составной индекс
- Частичный индекс
- Функциональный индекс

Определения:

1. Обеспечивает уникальность значений в колонке
2. Создаётся на нескольких колонках для ускорения сложных запросов
3. Создаётся с условием WHERE для части таблицы
4. Использует функцию или выражение вместо обычной колонки

Подсказка: Обратите внимание, как индекс применяется: к колонке, набору колонок или результату функции.

- **Уникальный индекс** — Обеспечивает уникальность значений в колонке
- **Составной индекс** — Создаётся на нескольких колонках для ускорения сложных запросов
- **Частичный индекс** — Создаётся с условием WHERE для части таблицы
- **Функциональный индекс** — Использует функцию или выражение вместо обычной колонки

Шаг 07:

Миссия 7

 Сопоставьте операции с их влиянием на индексы

Понятия:

Ответ:

- INSERT
- UPDATE
- DELETE
- SELECT

Определения:

1. Обновление индекса при добавлении новой строки
2. Может вызвать перестройку части индекса при изменении индексируемой колонки
3. Удаляет записи и оставляет «мертвые» строки
4. Использует индекс для ускорения поиска

Подсказка: Подумайте, какие операции изменяют данные, а какие только читают их.

- **INSERT** — Обновление индекса при добавлении новой строки
- **UPDATE** — Может вызвать перестройку части индекса при изменении индексируемой колонки
- **DELETE** — Удаляет записи и оставляет «мертвые» строки
- **SELECT** — Использует индекс для ускорения поиска

Шаг 08:

Миссия 8

Расположите шаги создания частичного индекса в правильном порядке:

1. Определить колонку и условие WHERE
2. Выбрать тип индекса
3. Создать индекс с помощью CREATE INDEX
4. Проверить ускорение запросов

Подсказка: Сначала планируем индекс, потом создаём и проверяем эффективность.

Ответ:

- Определить колонку и условие WHERE
- Выбрать тип индекса
- Создать индекс с помощью CREATE INDEX
- Проверить ускорение запросов

Шаг 09:

Миссия 9

Расположите шаги создания функционального индекса:

1. Выбрать функцию или выражение, которое ускорит поиск
2. Определить колонку или колонки для индекса
3. Создать индекс с помощью CREATE INDEX
4. Проверить скорость выполнения запросов

Подсказка: Сначала планируем, потом создаём, затем проверяем эффективность.

Ответ:

- Выбрать функцию или выражение, которое ускорит поиск
- Определить колонку или колонки для индекса
- Создать индекс с помощью CREATE INDEX
- Проверить скорость выполнения запросов

Шаг 10:

Миссия 10

Олег заметил, что SELECT с диапазоном дат медленный. Какой тип индекса лучше использовать?

Подсказка: Подумайте, какой индекс ускоряет поиск по диапазонам значений.

Ответ:

- **B-tree индекс**
- Hash индекс
- BRIN индекс
- GIN индекс

Шаг 11:

Миссия 11

Анна хочет ускорить полнотекстовый поиск по колонке с описанием товаров. Какой индекс выбрать?

Подсказка: Для массивов и текстового поиска есть специальные индексы.

Ответ:

- **GIN индекс**
- B-tree индекс
- Hash индекс
- BRIN индекс

Шаг 12:

Миссия 12

 Сопоставьте тип индекса с его преимуществами для конкретных задач

Понятия:

Ответ:

- B-tree
- Hash
- GIN
- BRIN

Определения:

1. Быстро сортирует и ищет по диапазону значений
2. Эффективен для точного поиска по колонке
3. Подходит для полнотекстового поиска и массивов
4. Идеален для очень больших таблиц, где данные хранятся в порядке вставки

Подсказка: Смотрите на задачи поиска: диапазон, точное совпадение, текст, огромные таблицы.

- **B-tree** — Быстро сортирует и ищет по диапазону значений
- **Hash** — Эффективен для точного поиска по колонке
- **GIN** — Подходит для полнотекстового поиска и массивов
- **BRIN** — Идеален для очень больших таблиц, где данные хранятся в порядке вставки

Шаг 13:

Миссия 13

 Сопоставьте действия DBA с их результатами для индексов

Понятия:

Ответ:

- VACUUM
- ANALYZE
- REINDEX
- CREATE INDEX

Определения:

1. Удаляет мёртвые строки и освобождает место
2. Обновляет статистику для оптимизатора
3. Перестраивает повреждённый или фрагментированный индекс
4. Создаёт новый индекс на колонке или выражении

Подсказка: Некоторые действия касаются очистки, другие — создания или обновления.

- **VACUUM** — Удаляет мёртвые строки и освобождает место
- **ANALYZE** — Обновляет статистику для оптимизатора
- **REINDEX** — Перестраивает повреждённый или фрагментированный индекс
- **CREATE INDEX** — Создаёт новый индекс на колонке или выражении

Шаг 14:

Миссия 14

Расположите шаги для оптимизации SELECT с частичным индексом:

1. Определить условие WHERE, по которому фильтруем данные
2. Создать частичный индекс с этим условием
3. Выполнить EXPLAIN ANALYZE для проверки плана выполнения
4. Сравнить время выполнения до и после индекса

Подсказка: План действий: выбрать условие → создать индекс → проверить эффективность → измерить результат.

Ответ:

- Определить условие WHERE, по которому фильтруем данные
- Создать частичный индекс с этим условием
- Выполнить EXPLAIN ANALYZE для проверки плана выполнения
- Сравнить время выполнения до и после индекса

Шаг 15:

Миссия 15

Расположите этапы создания составного индекса для ускорения сложного запроса:

1. Определить колонки, которые участвуют в фильтрах и сортировках
2. Выбрать тип индекса (B-tree, GIN и т.д.)
3. Создать индекс с помощью CREATE INDEX
4. Проверить эффективность с EXPLAIN ANALYZE

Подсказка: Подумайте, какие колонки влияют на производительность и в каком порядке создавать индекс.

Ответ:

- Определить колонки, которые участвуют в фильтрах и сортировках
- Выбрать тип индекса (B-tree, GIN и т.д.)
- Создать индекс с помощью CREATE INDEX
- Проверить эффективность с EXPLAIN ANALYZE

Шаг 16:

Миссия 16

Расположите шаги для создания функционального индекса с проверкой эффективности:

1. Выбрать выражение или функцию для индекса
2. Создать индекс через CREATE INDEX
3. Выполнить EXPLAIN ANALYZE для проверки плана запроса
4. Сравнить производительность запроса с предыдущей версией

Подсказка: Сначала планируем индекс, затем создаём, проверяем план и измеряем ускорение.

Ответ:

- Выбрать выражение или функцию для индекса
- Создать индекс через CREATE INDEX
- Выполнить EXPLAIN ANALYZE для проверки плана запроса
- Сравнить производительность запроса с предыдущей версией

 Шаг 17:

Миссия 17

Расположите действия по комплексной оптимизации запросов с разными типами индексов:

1. Сбор статистики таблиц с помощью ANALYZE
2. Проверка плана выполнения запросов через EXPLAIN ANALYZE
3. Создание необходимых индексов (B-tree, GIN, функциональные, составные)
4. Тестирование производительности и мониторинг

Подсказка: Логика: сначала анализируем → создаём индексы → проверяем ускорение → мониторим результаты.

Ответ:

- Сбор статистики таблиц с помощью ANALYZE
- Проверка плана выполнения запросов через EXPLAIN ANALYZE
- Создание необходимых индексов (B-tree, GIN, функциональные, составные)
- Тестирование производительности и мониторинг

 Урок 3 - Использование EXPLAIN для анализа производительности

 Шаг 01:

Миссия 1

Алексей заметил, что запрос SELECT по большой таблице выполняется медленно. Что первым делом стоит сделать?

Подсказка: Подумайте, как быстро получить план выполнения запроса.

Ответ:

- **Выполнить EXPLAIN для запроса**
- Создать новый индекс сразу
- Переписать SELECT в UPDATE
- Удалить ограничения таблицы

 Шаг 02:

Миссия 2

Мария хочет понять, какие части запроса тормозят выполнение JOIN. Что ей поможет?

Подсказка: Нужно увидеть порядок выполнения операций и их стоимость.

Ответ:

- **EXPLAIN ANALYZE**
- CREATE INDEX
- VACUUM FULL
- Обновление статистики вручную

Шаг 03:

Миссия 3

 Сопоставьте понятие с его объяснением для анализа запросов с EXPLAIN

Ответ:

- EXPLAIN
 - EXPLAIN ANALYZE
 - Seq Scan
 - Index Scan
1. Показывает план выполнения без фактического запуска
 2. Показывает план и фактическое время выполнения
 3. Последовательное сканирование таблицы
 4. Использование индекса для поиска данных

Подсказка: Думайте, что реально выполняет запрос, а что только анализирует.

- **EXPLAIN** — Показывает план выполнения без фактического запуска
- **EXPLAIN ANALYZE** — Показывает план и фактическое время выполнения
- **Seq Scan** — Последовательное сканирование таблицы
- **Index Scan** — Использование индекса для поиска данных

Шаг 04:

Миссия 4

Игорь заметил, что EXPLAIN показывает «Seq Scan» по большой таблице. Что это значит?

Подсказка: Подумайте, как PostgreSQL ищет строки без индекса.

Ответ:

- **Все строки таблицы сканируются последовательно**
- Используется индекс
- Только часть таблицы проверяется
- Данные кешируются в памяти

Шаг 05:

Миссия 5

Анна хочет ускорить SELECT с условием WHERE по колонке с индексом. EXPLAIN показывает Index Scan. Что это значит?

Подсказка: Смотрите, используется ли индекс для поиска.

Ответ:

- **Индекс используется для быстрого поиска нужных строк**
- Таблица сканируется полностью
- EXPLAIN не отображает использование индекса
- Запрос выполняется медленно, потому что индекс не нужен

Шаг 06:

Миссия 6

Сергей хочет понять, почему JOIN двух больших таблиц выполняется медленно. Что поможет выявить причину?

Подсказка: Смотрите, какой тип сканирования используется и где создаются узкие места.

Ответ:

- **EXPLAIN ANALYZE**
- CREATE INDEX
- VACUUM
- TRUNCATE TABLE

Шаг 07:

Миссия 7

Ирина видит, что EXPLAIN показывает 'Nested Loop' между таблицами. Что это значит?

Подсказка: Подумайте, как PostgreSQL соединяет таблицы поочередно.

Ответ:

- **Каждая строка первой таблицы проверяется на соответствие во второй таблице**
- Таблицы объединяются с помощью индекса
- Производится последовательное сканирование только одной таблицы
- Запрос выполняется параллельно на всех ядрах CPU

Шаг 08:

Миссия 8

 Сопоставьте тип операции в EXPLAIN с его значением:

Ответ:

- Seq Scan
 - Index Scan
 - Bitmap Heap Scan
 - Nested Loop
1. Последовательное сканирование всей таблицы
 2. Использование индекса для поиска строк
 3. Поиск через битовую карту для выборки строк
 4. Соединение таблиц с вложенным перебором строк

Подсказка: Думайте, что выполняется напрямую, а что использует индекс или карту.

- **Seq Scan** — Последовательное сканирование всей таблицы
- **Index Scan** — Использование индекса для поиска строк
- **Bitmap Heap Scan** — Поиск через битовую карту для выборки строк
- **Nested Loop** — Соединение таблиц с вложенным перебором строк

Шаг 09:

Миссия 9

Петр хочет увидеть фактическое время выполнения каждого шага запроса. Какой ключ EXPLAIN использовать?

Подсказка: Нужно реально запустить запрос и измерить время.

Ответ:

- **ANALYZE**
- BUFFERS
- VERBOSE
- COSTS

Шаг 10:

Миссия 10

 Сопоставьте EXPLAIN ключи с их назначением

Ответ:

- COSTS
- BUFFERS
- VERBOSE
- ANALYZE

1. Показывает оценки затрат на выполнение операций
2. Показывает использование памяти и дисковых блоков
3. Выводит расширенную информацию о плане
4. Выполняет запрос и показывает реальные времена

Подсказка: Некоторые ключи показывают оценки, другие фактические показатели.

- **COSTS** — Показывает оценки затрат на выполнение операций
- **BUFFERS** — Показывает использование памяти и дисковых блоков
- **VERBOSE** — Выводит расширенную информацию о плане
- **ANALYZE** — Выполняет запрос и показывает реальные времена

Шаг 11:

Миссия 11

Наталья видит, что EXPLAIN показывает «Bitmap Index Scan». Что это значит?

Подсказка: Подумайте, как PostgreSQL выбирает строки через битовую карту.

Ответ:

- **Сначала строится битовая карта подходящих строк, потом выполняется выборка**
- Выполняется полное сканирование таблицы
- Используется индекс для точного поиска одной строки
- Производится сортировка результата запроса

Шаг 12:

Миссия 12

Расположите шаги анализа медленного запроса через EXPLAIN:

1. Выполнить EXPLAIN без ANALYZE для оценки плана
2. Добавить ANALYZE для получения реального времени
3. Посмотреть тип сканирования: Seq Scan, Index Scan, Bitmap Scan
4. Определить узкие места и возможности оптимизации

Подсказка: Сначала смотрим план, потом фактическое выполнение, потом анализируем проблемы.

Ответ:

- Выполнить EXPLAIN без ANALYZE для оценки плана
- Добавить ANALYZE для получения реального времени
- Посмотреть тип сканирования: Seq Scan, Index Scan, Bitmap Scan
- Определить узкие места и возможности оптимизации

Шаг 13:

Миссия 13

Андрей хочет сравнить время выполнения одного и того же запроса до и после индекса. Что ему использовать?

Подсказка: Нужно реальное измерение времени выполнения запроса.

Ответ:

- **EXPLAIN ANALYZE**
- EXPLAIN
- VACUUM FULL
- ANALYZE

Шаг 14:

Миссия 14

 Сопоставьте результат EXPLAIN ANALYZE с выводом:

Ответ:

- Actual Total Time
- Planning Time
- Buffers
- Rows Removed by Filter

1. Время, затраченное на фактическое выполнение запроса
2. Время, затраченное на планирование запроса
3. Количество использованных блоков памяти и диска
4. Сколько строк было отфильтровано

Подсказка: Разделяем планирование, выполнение и использование ресурсов.

- **Actual Total Time** — Время, затраченное на фактическое выполнение запроса
- **Planning Time** — Время, затраченное на планирование запроса
- **Buffers** — Количество использованных блоков памяти и диска
- **Rows Removed by Filter** — Сколько строк было отфильтровано

Шаг 15:

Миссия 15

Расположите шаги для оптимизации запроса после анализа EXPLAIN:

1. Выполнить EXPLAIN ANALYZE и посмотреть узкие места
2. Добавить индексы на колонки, участвующие в WHERE и JOIN
3. Переписать запрос для использования индекса и уменьшения Seq Scan
4. Повторно проверить план выполнения с EXPLAIN ANALYZE

Подсказка: Сначала анализируем, затем оптимизируем, потом проверяем результат.

Ответ:

- Выполнить EXPLAIN ANALYZE и посмотреть узкие места
- Добавить индексы на колонки, участвующие в WHERE и JOIN
- Переписать запрос для использования индекса и уменьшения Seq Scan
- Повторно проверить план выполнения с EXPLAIN ANALYZE

Шаг 16:

Миссия 16

Расположите этапы комплексного анализа производительности:

1. Сбор статистики таблиц через ANALYZE
2. Выполнение EXPLAIN без ANALYZE для оценки плана
3. Выполнение EXPLAIN ANALYZE для фактических времен
4. Определение узких мест и оптимизация индексов и запроса
5. Повторная проверка с EXPLAIN ANALYZE

Подсказка: Логика: сначала статистика, потом план, потом фактические времена, затем оптимизация и проверка.

Ответ:

- Сбор статистики таблиц через ANALYZE
- Выполнение EXPLAIN без ANALYZE для оценки плана
- Выполнение EXPLAIN ANALYZE для фактических времен
- Определение узких мест и оптимизация индексов и запроса
- Повторная проверка с EXPLAIN ANALYZE

Урок 4 - Работа с большими данными и оптимизация запросов

Шаг 01:

Миссия 1

Алексей обрабатывает таблицу с миллионами строк и замечает, что SELECT выполняется медленно. Что поможет ускорить выборку?

Подсказка: Сосредоточьтесь на способах оптимизации доступа к данным.

Ответ:

- **Создание индексов на часто используемых колонках**
- Увеличение количества строк в таблице
- Добавление сложных подзапросов
- Использование SELECT * без WHERE

Шаг 02:

Миссия 2

Марина заметила, что GROUP BY на большой таблице выполняется слишком долго. Что может помочь?

Подсказка: Подумайте о предварительной сортировке и индексах для агрегации.

Ответ:

- **Использование индексированных колонок для GROUP BY**
- Увеличение LIMIT без фильтров
- Добавление JOIN с другой большой таблицей
- Удаление WHERE условия

Шаг 03:

Миссия 3

 Сопоставьте методы оптимизации с их назначением:

Ответ:

- Индексация
 - Партиционирование
 - Материализованные представления
 - Предварительная агрегация
1. Ускоряет поиск строк по ключам
 2. Разделение таблицы на сегменты для быстрого доступа
 3. Сохраняет результат сложного запроса для повторного использования
 4. Суммирование данных заранее для ускорения запросов

Подсказка: Подумайте, что улучшает поиск, что уменьшает нагрузку на вычисления.

- **Индексация** — Ускоряет поиск строк по ключам
- **Партиционирование** — Разделение таблицы на сегменты для быстрого доступа
- **Материализованные представления** — Сохраняет результат сложного запроса для повторного использования
- **Предварительная агрегация** — Суммирование данных заранее для ускорения запросов

 Шаг 04:

Миссия 4

Сергей хочет уменьшить время выполнения JOIN двух больших таблиц. Что может помочь?

Подсказка: Подумайте о том, как ускорить соединение по ключу.

Ответ:

- **Создание индексов на колонках для JOIN**
- Удаление WHERE условий
- Использование SELECT DISTINCT без индексов
- Добавление ORDER BY на случайную колонку

 Шаг 05:

Миссия 5

 Сопоставьте типы партиционирования с их особенностями:

Ответ:

- Range Partitioning
 - List Partitioning
 - Hash Partitioning
1. Разделение по диапазону значений колонки
 2. Разделение по конкретным значениям колонки
 3. Равномерное распределение строк по сегментам с помощью хеша

Подсказка: Подумайте, что делит таблицу по диапазону, по списку и по хешу.

- **Range Partitioning** — Разделение по диапазону значений колонки
- **List Partitioning** — Разделение по конкретным значениям колонки
- **Hash Partitioning** — Равномерное распределение строк по сегментам с помощью хеша

 Шаг 06:

Миссия 6

Ирина хочет уменьшить нагрузку на базу при запросах с фильтрацией по дате. Что лучше использовать?

Подсказка: Думайте о том, как ускорить выборку с условием WHERE на колонке даты.

Ответ:

- **Создать индекс на колонку даты**
- Добавить больше JOIN без фильтрации
- Использовать SELECT * без условий
- Удалить все индексы

Шаг 07:

Миссия 7

Расположите шаги оптимизации медленного запроса:

1. Проанализировать запрос и определить узкие места
2. Добавить индексы на ключевые колонки
3. Рассмотреть партиционирование больших таблиц
4. Использовать материализованные представления или предварительную агрегацию
5. Проверить план выполнения с EXPLAIN и измерить время

Подсказка: Логика: сначала анализируем, затем оптимизируем доступ к данным, потом проверяем результат.

Ответ:

- Проанализировать запрос и определить узкие места
- Добавить индексы на ключевые колонки
- Рассмотреть партиционирование больших таблиц
- Использовать материализованные представления или предварительную агрегацию
- Проверить план выполнения с EXPLAIN и измерить время

Шаг 08:

Миссия 8

Олег хочет ускорить COUNT(*) на очень большой таблице. Что поможет?

Подсказка: Подумайте о предварительном подсчете или оптимизации доступа к строкам.

Ответ:

- **Использовать материализованное представление с предварительным подсчетом**
- Добавить ORDER BY на любую колонку
- Удалить индексы
- Применять SELECT *

Шаг 09:

Миссия 9

 Сопоставьте техники оптимизации с их эффектом:

Ответ:

- Индексация
- Материализованные представления
- Партиционирование

- Предварительная агрегация

1. Ускоряет фильтрацию и поиск по ключам
2. Сохраняет заранее вычисленный результат запроса
3. Разделяет таблицу для уменьшения нагрузки на каждый сегмент
4. Суммирует или считает данные заранее для ускорения выборки

Подсказка: Подумайте, что ускоряет поиск, а что уменьшает вычислительную нагрузку.

- **Индексация** — Ускоряет фильтрацию и поиск по ключам
- **Материализованные представления** — Сохраняет заранее вычисленный результат запроса
- **Партиционирование** — Разделяет таблицу для уменьшения нагрузки на каждый сегмент
- **Предварительная агрегация** — Суммирует или считает данные заранее для ускорения выборки

Шаг 10:

Миссия 10

Лена хочет уменьшить количество I/O при запросах к большой таблице. Что поможет?

Подсказка: Думайте о структуре хранения и использовании индексов.

Ответ:

- **Использовать покрывающий индекс**
- Добавить ORDER BY на случайную колонку
- Применять SELECT *
- Удалить индексы

Шаг 11:

Миссия 11

Лена хочет уменьшить количество I/O при запросах к большой таблице. Что поможет?

Подсказка: Думайте о структуре хранения и использовании индексов.

Ответ:

- **Использовать покрывающий индекс**
- Добавить ORDER BY на случайную колонку
- Применять SELECT *
- Удалить индексы

Шаг 12:

Миссия 12

Расположите шаги подготовки большой таблицы к быстрому анализу:

1. Сбор статистики таблицы с ANALYZE
2. Создание индексов на часто используемых колонках
3. Рассмотрение партиционирования таблицы
4. Материализованные представления или предварительная агрегация
5. Проверка скорости запросов с EXPLAIN

Подсказка: Логика: сначала статистика, потом индексы, партиционирование и предварительная агрегация, потом проверка.

Ответ:

- Сбор статистики таблицы с ANALYZE
- Создание индексов на часто используемых колонках
- Рассмотрение партиционирования таблицы
- Материализованные представления или предварительная агрегация
- Проверка скорости запросов с EXPLAIN

 Шаг 13:

Миссия 13

Дмитрий обрабатывает лог-файлы в PostgreSQL и хочет ускорить фильтрацию по пользователю. Что поможет?

Подсказка: Подумайте, как индекс может ускорить WHERE на колонке user_id.

Ответ:

- **Создать индекс на колонку user_id**
- Добавить ORDER BY случайной колонки
- Использовать SELECT * без WHERE
- Удалить все индексы

 Шаг 14:

Миссия 14

 Сопоставьте типы индексов с их преимуществами:

Ответ:

- B-Tree
 - Hash
 - GIN
 - BRIN
1. Подходит для точного поиска и сортировки по значениям
 2. Быстро ищет по точным соответствиям, но не поддерживает диапазоны
 3. Эффективен для полнотекстового поиска и массивов
 4. Хорош для больших таблиц с упорядоченными данными и диапазонами

Подсказка: Подумайте, какой индекс лучше для диапазона, какой для точного поиска.

- **B-Tree** — Подходит для точного поиска и сортировки по значениям
- **Hash** — Быстро ищет по точным соответствиям, но не поддерживает диапазоны
- **GIN** — Эффективен для полнотекстового поиска и массивов
- **BRIN** — Хорош для больших таблиц с упорядоченными данными и диапазонами

 Шаг 15:

Миссия 15

Расположите шаги подготовки логов к быстрому аналитическому запросу:

1. Создать партиционированную таблицу для логов
2. Создать индексы на колонках фильтрации
3. Использовать материализованные представления для агрегатов
4. Проверить время выполнения запросов с EXPLAIN ANALYZE
5. Настроить автоматическую агрегацию или предварительную обработку

Подсказка: Логика: сначала структура таблицы, потом индексы, потом агрегаты и проверка.

Ответ:

- Создать партиционированную таблицу для логов
- Создать индексы на колонках фильтрации
- Использовать материализованные представления для агрегатов
- Проверить время выполнения запросов с EXPLAIN ANALYZE
- Настроить автоматическую агрегацию или предварительную обработку

Урок 5 - Оптимизация работы с большими таблицами

Шаг 01:

Миссия 1

Алексей хочет ускорить SELECT-запросы по очень большой таблице заказов. Что поможет?

Подсказка: Думайте о том, как уменьшить количество строк, которые PostgreSQL должен просмотреть.

Ответ:

- **Создать индекс на колонке с фильтром**
- Использовать SELECT * без WHERE
- Добавить случайный ORDER BY
- Удалить все индексы

Шаг 02:

Миссия 2

Мария хочет ускорить агрегации по миллионам строк. Что выбрать?

Подсказка: Подумайте о предварительных вычислениях и структуре таблицы.

Ответ:

- **Использовать материализованное представление**
- Применять SELECT без индексов
- Создать триггер на каждую строку
- Удалить статистику таблицы

Шаг 03:

Миссия 3

Игорь работает с логами размером несколько терабайт. Что ускорит фильтрацию по дате?

Подсказка: Подумайте о партиционировании и индексах на колонках диапазона.

Ответ:

- **Партиционировать таблицу по дате**
- Создать индекс на случайной колонке
- Удалить все индексы
- Использовать SELECT без WHERE

Шаг 04:

Миссия 4

 Сопоставьте типы оптимизаций с их преимуществами:

Ответ:

- Индекс
- Материализованное представление
- Партиционирование
- Сбор статистики с ANALYZE

1. Ускоряет выборку по колонкам с условиями WHERE
2. Сохраняет результат запроса для быстрого доступа к агрегатам
3. Разбивает таблицу на логические части для уменьшения объема сканирования
4. Обновляет статистику для оптимизатора запросов

Подсказка: Подумайте, что помогает выборке, что — агрегатам, что — плану выполнения.

- **Индекс** — Ускоряет выборку по колонкам с условиями WHERE
- **Материализованное представление** — Сохраняет результат запроса для быстрого доступа к агрегатам
- **Партиционирование** — Разбивает таблицу на логические части для уменьшения объема сканирования
- **Сбор статистики с ANALYZE** — Обновляет статистику для оптимизатора запросов

 Шаг 05:

Миссия 5

 Сопоставьте техники оптимизации с примерами:

Ответ:

- Индексы B-Tree
- GIN
- BRIN
- EXPLAIN

1. Полнотекстовый поиск и массивы
2. Диапазонные запросы на огромных таблицах
3. Сортировка и поиск по значениям колонок
4. Анализ плана выполнения запроса

Подсказка: Вспомните, какие типы индексов подходят для разных задач.

- **Индексы B-Tree** — Сортировка и поиск по значениям колонок
- **GIN** — Полнотекстовый поиск и массивы
- **BRIN** — Диапазонные запросы на огромных таблицах
- **EXPLAIN** — Анализ плана выполнения запроса

 Шаг 06:

Миссия 6

Влад использует EXPLAIN для анализа медленных запросов. Что он ищет?

Подсказка: Смотрите на количество строк и тип сканирования таблицы.

Ответ:

- **Тип сканирования и количество строк**
- Цвет интерфейса PostgreSQL
- Размер шрифта SQL-запроса
- Название таблицы

Шаг 07:

Миссия 7

Расположите шаги оптимизации большого запроса:

1. Проанализировать план с EXPLAIN
2. Создать необходимые индексы
3. Разбить таблицу на партиции
4. Использовать материализованное представление при повторных агрегатах
5. Проверить время выполнения после изменений

Подсказка: Логика: сначала анализ, потом оптимизации, потом проверка.

Ответ:

- Проанализировать план с EXPLAIN
- Создать необходимые индексы
- Разбить таблицу на партиции
- Использовать материализованное представление при повторных агрегатах
- Проверить время выполнения после изменений

Шаг 08:

Миссия 8

Светлана работает с таблицей клиентов на 50 млн строк. Как ускорить поиск по email?

Подсказка: Подумайте, как сделать поиск по уникальному полю более эффективным.

Ответ:

- **Создать уникальный индекс на email**
- Использовать SELECT без WHERE
- Создать индекс на случайной колонке
- Сделать таблицу временной

Шаг 09:

Миссия 9

Николай замечает, что обновления в таблице заказов тормозят из-за большого числа индексов. Что оптимальнее?

Подсказка: Подумайте, какие индексы нужны только для SELECT-запросов.

Ответ:

- **Удалить неиспользуемые индексы**
- Добавить еще индексов
- Сделать все колонки уникальными
- Партиционировать таблицу случайно

Шаг 10:

Миссия 10

 Сопоставьте тип индекса с его оптимальной задачей:

Ответ:

- B-Tree
- GIN

- BRIN
- Hash

1. Поиск по конкретному значению колонки
2. Полнотекстовый поиск
3. Диапазонные запросы по большой таблице
4. Поиск по равенству с небольшим набором значений

Подсказка: Помните, какой индекс лучше для диапазонов, а какой для точного совпадения.

- **B-Tree** — Поиск по конкретному значению колонки
- **GIN** — Полнотекстовый поиск
- **BRIN** — Диапазонные запросы по большой таблице
- **Hash** — Поиск по равенству с небольшим набором значений

Шаг 11:

Миссия 11

 Сопоставьте методы работы с большими таблицами и их преимущества:

Ответ:

- Партиционирование
 - Материализованные представления
 - Индексы
 - EXPLAIN ANALYZE
1. Уменьшает объем сканируемых данных
 2. Сохраняет предварительно вычисленные результаты
 3. Ускоряет фильтрацию и сортировку
 4. Помогает понять, где узкие места запроса

Подсказка: Подумайте, какой метод уменьшает нагрузку на чтение, а какой помогает анализу.

- **Партиционирование** — Уменьшает объем сканируемых данных
- **Материализованные представления** — Сохраняет предварительно вычисленные результаты
- **Индексы** — Ускоряет фильтрацию и сортировку
- **EXPLAIN ANALYZE** — Помогает понять, где узкие места запроса

Шаг 12:

Миссия 12

Антон хочет уменьшить время вставки миллионов строк. Что выбрать?

Подсказка: Подумайте, какие операции на таблице могут замедлять вставку.

Ответ:

- **Временно отключить индексы на время вставки**
- Добавить больше триггеров
- Партиционировать случайным образом
- Сделать таблицу TEMP

Шаг 13:

Миссия 13

Расположите шаги оптимизации большого запроса с агрегацией:

1. Проанализировать план с EXPLAIN
2. Создать необходимые индексы
3. Использовать материализованное представление для агрегатов
4. Разбить таблицу на партиции, если нужно
5. Проверить время выполнения после изменений

Подсказка: Логика: сначала анализ, затем оптимизация, затем проверка.

Ответ:

- Проанализировать план с EXPLAIN
- Создать необходимые индексы
- Использовать материализованное представление для агрегатов
- Разбить таблицу на партиции, если нужно
- Проверить время выполнения после изменений

Шаг 14:

Миссия 14

Ирина хочет ускорить SELECT с фильтром по дате в таблице с миллионами записей. Что сделать?

Подсказка: Подумайте о структуре индекса и о диапазонах дат.

Ответ:

- **Создать B-Tree индекс по колонке даты**
- Создать Hash индекс по дате
- Удалить все индексы
- Использовать SELECT DISTINCT без WHERE

Шаг 15:

Миссия 15

 Сопоставьте приемы оптимизации больших таблиц с их описанием:

Ответ:

- VACUUM
 - CLUSTER
 - ANALYZE
 - REINDEX
1. Удаляет мертвые строки и освобождает место
 2. Перестраивает таблицу по индексу для ускорения сканирования
 3. Собирает статистику для планировщика запросов
 4. Восстанавливает эффективность индексов

Подсказка: Подумайте, что влияет на физическое хранение данных и что — на планирование запросов.

- **VACUUM** — Удаляет мертвые строки и освобождает место
- **CLUSTER** — Перестраивает таблицу по индексу для ускорения сканирования
- **ANALYZE** — Собирает статистику для планировщика запросов
- **REINDEX** — Восстанавливает эффективность индексов

Шаг 16:

Миссия 16

Петр хочет ускорить агрегацию по миллионам строк. Что эффективнее?

Подсказка: Подумайте о хранении предвычисленных агрегатов.

Ответ:

- **Использовать материализованное представление с агрегатами**
- Считать агрегаты на лету при каждом SELECT
- Создать триггер на каждую вставку
- Добавить случайный индекс

Шаг 17:

Миссия 17

Расположите шаги оптимизации таблицы с миллионами записей:

1. Проанализировать план запроса через EXPLAIN
2. Применить VACUUM и ANALYZE
3. Создать необходимые индексы
4. Использовать материализованные представления для тяжелых агрегатов
5. Проверить время выполнения и при необходимости пересмотреть стратегию

Подсказка: Логично сначала понять проблему, потом оптимизировать, и только потом проверять результаты.

Ответ:

- Проанализировать план запроса через EXPLAIN
- Применить VACUUM и ANALYZE
- Создать необходимые индексы
- Использовать материализованные представления для тяжелых агрегатов
- Проверить время выполнения и при необходимости пересмотреть стратегию

Модуль 4 - Работа с большими объемами данных

Урок 1 - Разделение данных с помощью PARTITION

Шаг 01:

Миссия 1

Алиса разрабатывает систему учёта заказов. Она заметила, что таблица заказов растёт слишком быстро. Что из перечисленного поможет разделить данные на части для улучшения производительности?

Подсказка: Подумайте о механизме, позволяющем хранить данные по категориям без изменения приложения.

Ответ:

- **Использование PARTITION для таблицы заказов**
- Создание отдельной базы для каждого заказа
- Хранение всех данных в одном огромном массиве PHP
- Удаление старых заказов каждую минуту

Шаг 02:

Миссия 2

Боб разрабатывает систему аналитики. Таблица с логами слишком большая. Что позволит ускорить выборку данных за конкретный год?

Подсказка: Подумайте о колонке, по которой удобно делить таблицу.

Ответ:

- **Создать партиции по году события**
- Создать индекс по имени пользователя
- Дублировать таблицу полностью
- Использовать временные таблицы для каждого запроса

Шаг 03:

Миссия 3

Клара хочет разделить таблицу пользователей по региону. Что нужно выбрать в качестве стратегии PARTITION?

Подсказка: Подумайте о дискретной колонке с фиксированным набором значений.

Ответ:

- **LIST (регион)**
- RANGE (по дате регистрации)
- HASH (по email)
- Создать отдельную таблицу для каждого пользователя

Шаг 04:

Миссия 4

 Сопоставьте тип PARTITION с его правильным описанием.

Понятия:

Ответ:

- RANGE
- LIST
- HASH
- DEFAULT

Определения:

1. Деление по диапазонам значений
2. Деление по фиксированным спискам значений
3. Деление с использованием хэш-функции
4. Партиция по умолчанию для всех остальных данных

Подсказка: Сопоставьте тип разделения с логикой его работы — диапазон, список, хэш или запасной вариант.

- **RANGE** — Деление по диапазонам значений
- **LIST** — Деление по фиксированным спискам значений
- **HASH** — Деление с использованием хэш-функции
- **DEFAULT** — Партиция по умолчанию для всех остальных данных

Шаг 05:

Миссия 5

 Сопоставьте действие с эффектом при работе с PARTITION.

Понятия:

Ответ:

- Добавление новой партиции
- Удаление старой партиции
- Перемещение данных между партициями
- Создание индекса на партиции

Определения:

1. Позволяет расширять таблицу для новых значений
2. Освобождает место и улучшает производительность
3. Обеспечивает корректное распределение данных
4. Ускоряет выборку для конкретной части данных

Подсказка: Думайте о том, как каждое действие влияет на структуру или скорость работы таблицы.

- **Добавление новой партиции** — Позволяет расширять таблицу для новых значений
- **Удаление старой партиции** — Освобождает место и улучшает производительность
- **Перемещение данных между партициями** — Обеспечивает корректное распределение данных
- **Создание индекса на партиции** — Ускоряет выборку для конкретной части данных

Шаг 06:

Миссия 6

 Сопоставьте термин с примером использования PARTITION.

Понятия:

Ответ:

- RANGE
- LIST
- HASH
- DEFAULT

Определения:

1. Разделение заказов по дате доставки
2. Разделение пользователей по стране
3. Распределение данных на серверы по id пользователя
4. Хранение аномальных или неожиданных значений

Подсказка: Подумайте о логике группировки данных и её назначении.

- **RANGE** — Разделение заказов по дате доставки
- **LIST** — Разделение пользователей по стране
- **HASH** — Распределение данных на серверы по id пользователя
- **DEFAULT** — Хранение аномальных или неожиданных значений

Шаг 07:

Миссия 7

 Сопоставьте термин PARTITION с плюсом его использования.

Понятия:

Ответ:

- RANGE
- LIST
- HASH
- DEFAULT

Определения:

1. Упрощает отчётность по временным диапазонам
2. Удобно для категориальных данных
3. Помогает равномерно распределить нагрузку на диски
4. Обеспечивает корректную обработку нетипичных записей

Подсказка: Рассмотрите, какие выгоды приносит каждая стратегия разделения.

- **RANGE** — Упрощает отчётность по временным диапазонам
- **LIST** — Удобно для категориальных данных
- **HASH** — Помогает равномерно распределить нагрузку на диски
- **DEFAULT** — Обеспечивает корректную обработку нетипичных записей

Шаг 08:

Миссия 8

Расположите шаги настройки партицирования таблицы заказов в правильном порядке:

1. Создание основной таблицы с указанием PARTITION BY
2. Создание конкретных партиций с условиями RANGE или LIST
3. Вставка данных в таблицу
4. Проверка корректности данных и работы SELECT

Подсказка: Сначала создаётся структура, потом партиции, потом данные, и только потом проверка.

Ответ:

- Создание основной таблицы с указанием PARTITION BY
- Создание конкретных партиций с условиями RANGE или LIST
- Вставка данных в таблицу
- Проверка корректности данных и работы SELECT

Шаг 09:

Миссия 9

Расположите шаги оптимизации запросов к партицированной таблице:

1. Проанализировать план выполнения запроса
2. Создать индексы на нужных партициях
3. Использовать фильтры по ключу PARTITION
4. Переписать медленные JOIN-операции

Подсказка: Сначала анализ, потом индексы, фильтры и оптимизация JOIN.

Ответ:

- Проанализировать план выполнения запроса
- Создать индексы на нужных партициях
- Использовать фильтры по ключу PARTITION
- Переписать медленные JOIN-операции

 Шаг 10:

Миссия 10

Расположите шаги добавления новой партиции при росте данных:

1. Определить диапазон или список для новой партиции
2. Создать партицию с указанными границами
3. Проверить, что запросы корректно выбирают данные
4. Обновить план резервного копирования

Подсказка: Сначала планирование, потом создание, проверка и только потом поддержка.

Ответ:

- Определить диапазон или список для новой партиции
- Создать партицию с указанными границами
- Проверить, что запросы корректно выбирают данные
- Обновить план резервного копирования

 Шаг 11:

Миссия 11

Расположите шаги мониторинга партицированных таблиц:

1. Собрать статистику по размеру каждой партиции
2. Проанализировать загрузку диска и запросы
3. Выявить партиции с наибольшей нагрузкой
4. Принять решение о создании новых партиций или реорганизации старых

Подсказка: Мониторинг начинается с измерения и анализа, затем выявление проблем и действия.

Ответ:

- Собрать статистику по размеру каждой партиции
- Проанализировать загрузку диска и запросы
- Выявить партиции с наибольшей нагрузкой
- Принять решение о создании новых партиций или реорганизации старых

 Шаг 12:

Миссия 12

Расположите шаги тестирования запросов на партицированной таблице:

1. Выполнить SELECT по конкретной партиции
2. Проверить выборку через все партиции
3. Сравнить результаты с тестовой таблицей без партиций
4. Проанализировать план выполнения и время запроса

Подсказка: Сначала локальная проверка, затем полная, потом сравнение и анализ.

Ответ:

- Выполнить SELECT по конкретной партиции
- Проверить выборку через все партиции
- Сравнить результаты с тестовой таблицей без партиций
- Проанализировать план выполнения и время запроса

Шаг 13:

Миссия 13

Расположите шаги резервного копирования партицированных таблиц:

1. Определить критичные партиции для бэкапа
2. Создать резервную копию каждой партиции
3. Проверить целостность данных в резервных копиях
4. Обновить расписание автоматического бэкапа

Подсказка: Сначала выбор, потом копия, проверка и автоматизация.

Ответ:

- Определить критичные партиции для бэкапа
- Создать резервную копию каждой партиции
- Проверить целостность данных в резервных копиях
- Обновить расписание автоматического бэкапа

Шаг 14:

Миссия 14

Расположите шаги миграции старой таблицы на партицированную:

1. Создать новую партицированную таблицу
2. Перенести данные из старой таблицы в партиции
3. Проверить корректность всех вставленных данных
4. Перенаправить приложения на новую таблицу

Подсказка: Сначала подготовка, затем перенос, проверка и подключение приложения.

Ответ:

- Создать новую партицированную таблицу
- Перенести данные из старой таблицы в партиции
- Проверить корректность всех вставленных данных
- Перенаправить приложения на новую таблицу

Шаг 15:

Миссия 15

Расположите шаги архивации старых партиций:

1. Определить партиции, которые можно архивировать
2. Экспортировать данные в отдельное хранилище
3. Удалить старые партиции из основной базы
4. Проверить доступность архивных данных при необходимости

Подсказка: Архив начинается с выбора, затем экспорт, удаление и проверка.

Ответ:

- Определить партиции, которые можно архивировать
- Экспортировать данные в отдельное хранилище
- Удалить старые партиции из основной базы
- Проверить доступность архивных данных при необходимости

 Шаг 16:

Миссия 16

Расположите шаги восстановления партицированной таблицы после сбоя:

1. Определить повреждённые или отсутствующие партиции
2. Восстановить данные из резервных копий
3. Проверить целостность и корректность восстановленных данных
4. Обновить индексы и планы выполнения запросов

Подсказка: Сначала диагностика, потом восстановление, проверка и оптимизация.

Ответ:

- Определить повреждённые или отсутствующие партиции
- Восстановить данные из резервных копий
- Проверить целостность и корректность восстановленных данных
- Обновить индексы и планы выполнения запросов

 Шаг 17:

Миссия 17

Расположите шаги анализа нагрузки на партицированную таблицу при масштабировании:

1. Собрать статистику запросов и размер партиций
2. Выявить узкие места и горячие партиции
3. Применить вертикальное или горизонтальное масштабирование
4. Проверить производительность после изменений

Подсказка: Анализ начинается с измерений, затем выявление проблем, масштабирование и проверка.

Ответ:

- Собрать статистику запросов и размер партиций
- Выявить узкие места и горячие партиции
- Применить вертикальное или горизонтальное масштабирование
- Проверить производительность после изменений

 Урок 2 - Использование JSONB и HSTORE

 Шаг 01:

Миссия 1

Алиса разрабатывает систему хранения пользовательских настроек. Она хочет хранить динамические пары ключ-значение в PostgreSQL. Что лучше использовать?

Подсказка: Подумайте о типах данных, которые позволяют хранить произвольные наборы пар ключ-значение.

Ответ:

- **HSTORE**
- TEXT
- INTEGER
- DATE

 **Шаг 02:**

Миссия 2

Боб хочет хранить сложные вложенные данные о заказах, включая массивы и объекты. Какой тип данных PostgreSQL подойдет лучше всего?

Подсказка: Подумайте о поддержке JSON-структур.

Ответ:

- **JSONB**
- HSTORE
- CHAR
- BOOLEAN

 **Шаг 03:**

Миссия 3

Клара хочет быстро искать пользователей по произвольным атрибутам, хранящимся в JSONB. Что поможет ускорить поиск?

Подсказка: Подумайте о механизме индексирования JSONB в PostgreSQL.

Ответ:

- **Создание GIN-индекса**
- Использование обычного B-Tree на весь JSON
- Создание отдельной таблицы для каждого ключа
- Добавление CHECK CONSTRAINT на JSONB

 **Шаг 04:**

Миссия 4

 Сопоставьте тип данных с примером использования.

Понятия:

Ответ:

- JSONB
- HSTORE
- TEXT
- INTEGER

Определения:

1. Хранение сложных объектов с вложенными массивами и объектами
2. Хранение пар ключ-значение для гибкой схемы
3. Хранение длинных текстов без структурирования
4. Хранение числовых значений для расчетов

Подсказка: Подумайте, какие данные поддерживают вложенные структуры, а какие нет.

- **JSONB** — Хранение сложных объектов с вложенными массивами и объектами
- **HSTORE** — Хранение пар ключ-значение для гибкой схемы
- **TEXT** — Хранение длинных текстов без структурирования
- **INTEGER** — Хранение числовых значений для расчетов

🔧 Шаг 05:

Миссия 5

📖 Сопоставьте метод PostgreSQL с его функцией для JSONB и HSTORE.

Понятия:

Ответ:

- ->
- ->>
- #>
- #>>

Определения:

1. Получение JSON-объекта по ключу
2. Получение текстового значения по ключу
3. Доступ к вложенному объекту по пути
4. Доступ к текстовому значению по пути

Подсказка: Обратите внимание на разницу между объектом и текстом, а также на вложенные структуры.

- -> — Получение JSON-объекта по ключу
- ->> — Получение текстового значения по ключу
- #> — Доступ к вложенному объекту по пути
- #>> — Доступ к текстовому значению по пути

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте функцию HSTORE с её назначением.

Понятия:

Ответ:

- akeys()
- avals()
- hstore -> key
- hstore || hstore

Определения:

1. Возвращает массив ключей
2. Возвращает массив значений
3. Получает значение по ключу
4. Объединяет два hstore

Подсказка: Вспомните, как извлекаются ключи и значения, а также как объединяются пары.

- **akeys()** — Возвращает массив ключей

- **avals()** — Возвращает массив значений
- **hstore -> key** — Получает значение по ключу
- **hstore || hstore** — Объединяет два hstore

🔧 Шаг 07:

Миссия 7

📄 Сопоставьте оператор JSONB с примером использования.

Понятия:

Ответ:

- @>
- <@
- ?
- ?|

Определения:

1. Проверка, содержит ли JSONB заданный объект
2. Проверка, содержится ли объект в JSONB
3. Проверка наличия конкретного ключа
4. Проверка наличия любого из списка ключей

Подсказка: Подумайте, какой оператор проверяет содержимое, а какой ключи.

- @> — Проверка, содержит ли JSONB заданный объект
- <@ — Проверка, содержится ли объект в JSONB
- ? — Проверка наличия конкретного ключа
- ?| — Проверка наличия любого из списка ключей

🔧 Шаг 08:

Миссия 8

Расположите шаги добавления нового JSONB поля и индекса для поиска:

1. Добавить колонку типа JSONB в таблицу
2. Заполнить поле начальными данными
3. Создать GIN-индекс на колонку
4. Проверить выборку через SELECT с условиями по ключам

Подсказка: Сначала структура, потом данные, индекс, и только потом проверка.

Ответ:

- Добавить колонку типа JSONB в таблицу
- Заполнить поле начальными данными
- Создать GIN-индекс на колонку
- Проверить выборку через SELECT с условиями по ключам

🔧 Шаг 09:

Миссия 9

Расположите шаги преобразования данных из HSTORE в JSONB:

1. Выбрать таблицу с HSTORE-данными
2. Использовать функцию hstore_to_jsonb() для конверсии

3. Обновить колонку JSONB в таблице
4. Проверить корректность преобразованных данных

Подсказка: Сначала выбор данных, затем конвертация, обновление и проверка.

Ответ:

- Выбрать таблицу с HSTORE-данными
- Использовать функцию `hstore_to_jsonb()` для конверсии
- Обновить колонку JSONB в таблице
- Проверить корректность преобразованных данных

Шаг 10:

Миссия 10

Расположите шаги оптимизации запросов к JSONB колонке:

1. Выявить часто используемые ключи
2. Создать GIN-индекс на эти ключи
3. Переписать SELECT-запросы с использованием индексированных ключей
4. Проверить время выполнения запросов

Подсказка: Оптимизация начинается с анализа, затем индекс, изменение запросов и проверка.

Ответ:

- Выявить часто используемые ключи
- Создать GIN-индекс на эти ключи
- Переписать SELECT-запросы с использованием индексированных ключей
- Проверить время выполнения запросов

Шаг 11:

Миссия 11

Расположите шаги объединения данных JSONB из нескольких таблиц:

1. Определить таблицы с JSONB колонками
2. Использовать оператор `||` для объединения JSONB
3. Создать новую колонку для объединенных данных
4. Проверить корректность объединенных данных

Подсказка: Сначала выбор, затем объединение, запись и проверка.

Ответ:

- Определить таблицы с JSONB колонками
- Использовать оператор `||` для объединения JSONB
- Создать новую колонку для объединенных данных
- Проверить корректность объединенных данных

Шаг 12:

Миссия 12

Расположите шаги создания исторических версий JSONB:

1. Добавить колонку типа JSONB для хранения истории
2. При обновлении основной колонки сохранять предыдущие данные в историю
3. Создать триггер для автоматической записи изменений

4. Проверить доступ к историческим версиям через SELECT

Подсказка: Сначала структура, затем логика обновлений, триггер и проверка.

Ответ:

- Добавить колонку типа JSONB для хранения истории
- При обновлении основной колонки сохранять предыдущие данные в историю
- Создать триггер для автоматической записи изменений
- Проверить доступ к историческим версиям через SELECT

Шаг 13:

Миссия 13

Расположите шаги фильтрации пользователей по JSONB атрибутам:

1. Определить ключи JSONB для фильтрации
2. Создать индекс на ключи, если требуется
3. Составить SELECT-запрос с условием по JSONB
4. Проверить корректность возвращаемых данных

Подсказка: Анализ → индекс → запрос → проверка.

Ответ:

- Определить ключи JSONB для фильтрации
- Создать индекс на ключи, если требуется
- Составить SELECT-запрос с условием по JSONB
- Проверить корректность возвращаемых данных

Шаг 14:

Миссия 14

Расположите шаги обновления JSONB через PHP:

1. Получить JSONB данные из PostgreSQL через PDO
2. Преобразовать данные в PHP массив
3. Изменить значения и структуру массива
4. Сохранить обновленный JSONB обратно в PostgreSQL

Подсказка: Сначала извлечение, потом преобразование, редактирование и запись обратно.

Ответ:

- Получить JSONB данные из PostgreSQL через PDO
- Преобразовать данные в PHP массив
- Изменить значения и структуру массива
- Сохранить обновленный JSONB обратно в PostgreSQL

Шаг 15:

Миссия 15

Расположите шаги удаления ключей из HSTORE:

1. Определить ключи, которые нужно удалить
2. Использовать оператор - для удаления ключа
3. Обновить колонку в таблице
4. Проверить, что ключи удалены

Подсказка: Анализ → оператор → обновление → проверка.

Ответ:

- Определить ключи, которые нужно удалить
- Использовать оператор - для удаления ключа
- Обновить колонку в таблице
- Проверить, что ключи удалены

Шаг 16:

Миссия 16

Расположите шаги объединения HSTORE и JSONB данных:

1. Преобразовать HSTORE в JSONB
2. Использовать оператор || для объединения JSONB
3. Сохранить объединенные данные в новую колонку
4. Проверить корректность объединения

Подсказка: Сначала преобразование, затем объединение, сохранение и проверка.

Ответ:

- Преобразовать HSTORE в JSONB
- Использовать оператор || для объединения JSONB
- Сохранить объединенные данные в новую колонку
- Проверить корректность объединения

Шаг 17:

Миссия 17

Расположите шаги миграции текстовых настроек в JSONB для улучшенной аналитики:

1. Выбрать колонку TEXT с настройками
2. Преобразовать данные в JSONB с помощью функции to_jsonb()
3. Создать GIN-индекс для ускорения поиска
4. Проверить корректность и производительность SELECT-запросов

Подсказка: Сначала выбор, затем преобразование, индекс и проверка запросов.

Ответ:

- Выбрать колонку TEXT с настройками
- Преобразовать данные в JSONB с помощью функции to_jsonb()
- Создать GIN-индекс для ускорения поиска
- Проверить корректность и производительность SELECT-запросов

Урок 3 - Загрузка и выгрузка данных с помощью COPY и pg_bulkload

Шаг 01:

Миссия 1

Аня загружает большой CSV-файл в PostgreSQL. Какой инструмент позволит сделать это быстро и эффективно?

Подсказка: Подумайте о встроенных методах PostgreSQL для массовой загрузки данных.

Ответ:

- COPY

- INSERT
- UPDATE
- SELECT INTO

🔗 Шаг 02:

Миссия 2

Иван хочет загружать миллионы строк данных с минимальной нагрузкой на сервер. Что поможет ускорить процесс?

Подсказка: Подумайте о сторонних инструментах оптимизации массовой загрузки.

Ответ:

- **pg_bulkload**
- psql SELECT
- pg_dump
- pg_restore

🔗 Шаг 03:

Миссия 3

Ольга хочет выгрузить данные таблицы в CSV с разделителем `;`. Какой инструмент PostgreSQL использовать?

Подсказка: Подумайте о команде для экспорта данных напрямую из таблицы.

Ответ:

- **COPY TO CSV**
- INSERT INTO CSV
- SELECT * INTO CSV
- pg_bulkload EXPORT

🔗 Шаг 04:

Миссия 4

📄 Сопоставьте инструмент с его особенностью.

Понятия:

Ответ:

- COPY
- pg_bulkload
- INSERT
- psql -c

Определения:

1. Быстрая загрузка больших объемов данных через встроенную команду PostgreSQL
2. Оптимизированная сторонняя утилита для массовой загрузки
3. Пошаговая вставка записей, медленная при больших объемах
4. Команда для запуска SQL из командной строки

Подсказка: Обратите внимание на разницу между встроенной командой и сторонними инструментами.

- **COPY** — Быстрая загрузка больших объемов данных через встроенную команду PostgreSQL

- **pg_bulkload** — Оптимизированная сторонняя утилита для массовой загрузки
- **INSERT** — Пошаговая вставка записей, медленная при больших объемах
- **psql -c** — Команда для запуска SQL из командной строки

🔧 Шаг 05:

Миссия 5

📄 Сопоставьте параметр COPY с его назначением.

Понятия:

Ответ:

- DELIMITER
- CSV
- HEADER
- NULL

Определения:

1. Указывает символ разделителя колонок
2. Формат данных для удобного чтения и записи
3. Указывает наличие строки заголовков
4. Задаёт представление NULL значений в файле

Подсказка: Подумайте, что чаще всего указывают при выгрузке в CSV.

- **DELIMITER** — Указывает символ разделителя колонок
- **CSV** — Формат данных для удобного чтения и записи
- **HEADER** — Указывает наличие строки заголовков
- **NULL** — Задаёт представление NULL значений в файле

🔧 Шаг 06:

Миссия 6

📄 Сопоставьте сценарий с подходящим методом загрузки:

Понятия:

Ответ:

- Миллионы записей из CSV
- Несколько сотен строк
- Экспорт для резервного копирования
- Быстрое обновление данных из внешнего источника

Определения:

1. pg_bulkload
2. INSERT
3. COPY TO
4. COPY FROM

Подсказка: Большие объемы требуют оптимизированных методов, маленькие — обычных вставок.

- **Миллионы записей из CSV** — pg_bulkload
- **Несколько сотен строк** — INSERT
- **Экспорт для резервного копирования** — COPY TO

- **Быстрое обновление данных из внешнего источника — COPY FROM**

Шаг 07:

Миссия 7

 Сопоставьте команду PostgreSQL с её задачей при массовой загрузке:

Понятия:

Ответ:

- `psql -f file.sql`
- `COPY table FROM 'file.csv'`
- `pg_bulkload -i file.csv`
- `INSERT INTO table VALUES (...)`

Определения:

1. Выполняет SQL-скрипт из файла
2. Загружает данные напрямую в таблицу из CSV
3. Использует оптимизированную утилиту для огромных файлов
4. Вставляет данные по одной записи за раз

Подсказка: Обратите внимание на отличие встроенной команды COPY и внешнего инструмента pg_bulkload.

- **`psql -f file.sql`** — Выполняет SQL-скрипт из файла
- **`COPY table FROM 'file.csv'`** — Загружает данные напрямую в таблицу из CSV
- **`pg_bulkload -i file.csv`** — Использует оптимизированную утилиту для огромных файлов
- **`INSERT INTO table VALUES (...)`** — Вставляет данные по одной записи за раз

Шаг 08:

Миссия 8

Расположите шаги массовой загрузки CSV через COPY:

1. Создать таблицу с нужной структурой
2. Подготовить CSV файл с правильным разделителем и форматом
3. Выполнить команду COPY FROM
4. Проверить корректность загруженных данных

Подсказка: Сначала структура, потом данные, команда и проверка.

Ответ:

- Создать таблицу с нужной структурой
- Подготовить CSV файл с правильным разделителем и форматом
- Выполнить команду COPY FROM
- Проверить корректность загруженных данных

Шаг 09:

Миссия 9

Расположите шаги выгрузки данных из PostgreSQL в CSV с помощью COPY:

1. Определить таблицу и нужные колонки
2. Выбрать формат CSV и разделитель
3. Выполнить COPY TO с указанным файлом

4. Проверить созданный файл и данные

Подсказка: Анализ → формат → команда → проверка.

Ответ:

- Определить таблицу и нужные колонки
- Выбрать формат CSV и разделитель
- Выполнить COPY TO с указанным файлом
- Проверить созданный файл и данные

Шаг 10:

Миссия 10

Расположите шаги использования pg_bulkload для загрузки больших CSV:

1. Подготовить CSV с правильными разделителями и кодировкой
2. Создать конфигурационный файл для pg_bulkload
3. Запустить pg_bulkload с указанием таблицы и файла
4. Проверить, что данные корректно загружены

Подсказка: Подготовка → конфиг → запуск → проверка.

Ответ:

- Подготовить CSV с правильными разделителями и кодировкой
- Создать конфигурационный файл для pg_bulkload
- Запустить pg_bulkload с указанием таблицы и файла
- Проверить, что данные корректно загружены

Шаг 11:

Миссия 11

Расположите шаги загрузки данных с пропуском заголовка в CSV:

1. Создать таблицу с нужными колонками
2. Подготовить CSV с заголовком
3. Выполнить COPY FROM с опцией HEADER
4. Проверить корректность вставки данных без первой строки

Подсказка: Сначала структура, потом CSV, затем команда с HEADER и проверка.

Ответ:

- Создать таблицу с нужными колонками
- Подготовить CSV с заголовком
- Выполнить COPY FROM с опцией HEADER
- Проверить корректность вставки данных без первой строки

Шаг 12:

Миссия 12

Расположите шаги выгрузки данных с фильтром в COPY TO:

1. Определить таблицу и необходимые колонки
2. Составить SELECT с фильтром
3. Выполнить COPY TO с результатом SELECT
4. Проверить созданный CSV файл

Подсказка: Фильтры применяются до команды COPY TO.

Ответ:

- Определить таблицу и необходимые колонки
- Составить SELECT с фильтром
- Выполнить COPY TO с результатом SELECT
- Проверить созданный CSV файл

 Шаг 13:

Миссия 13

Расположите шаги создания конфигурационного файла для pg_bulkload:

1. Определить таблицу назначения и колонки
2. Указать источник данных (CSV файл)
3. Настроить разделители, кодировку и NULL представления
4. Сохранить и проверить конфигурацию

Подсказка: Конфигурация должна полностью описывать источник и формат данных.

Ответ:

- Определить таблицу назначения и колонки
- Указать источник данных (CSV файл)
- Настроить разделители, кодировку и NULL представления
- Сохранить и проверить конфигурацию

 Шаг 14:

Миссия 14

Расположите шаги проверки корректности загруженных данных после COPY:

1. Сравнить количество строк в таблице и файле CSV
2. Проверить соответствие типов данных колонок
3. Просмотреть случайные записи на корректность значений
4. Исправить ошибки и при необходимости повторить загрузку

Подсказка: Сначала количественная проверка, потом качественная.

Ответ:

- Сравнить количество строк в таблице и файле CSV
- Проверить соответствие типов данных колонок
- Просмотреть случайные записи на корректность значений
- Исправить ошибки и при необходимости повторить загрузку

 Шаг 15:

Миссия 15

Расположите шаги безопасного обновления данных через COPY:

1. Создать временную таблицу с той же структурой
2. Загрузить новые данные во временную таблицу
3. Проверить корректность данных во временной таблице
4. Обновить основную таблицу с помощью транзакции

Подсказка: Используйте временную таблицу, чтобы избежать потери данных.

Ответ:

- Создать временную таблицу с той же структурой
- Загрузить новые данные во временную таблицу
- Проверить корректность данных во временной таблице
- Обновить основную таблицу с помощью транзакции

 Шаг 16:

Миссия 16

Расположите шаги оптимизации массовой загрузки с `pg_bulkload`:

1. Отключить индексы и триггеры временно
2. Настроить правильные разделители и кодировку в конфиге
3. Запустить загрузку через `pg_bulkload`
4. Включить обратно индексы и триггеры после успешной загрузки

Подсказка: Отключение индексов ускоряет массовую загрузку.

Ответ:

- Отключить индексы и триггеры временно
- Настроить правильные разделители и кодировку в конфиге
- Запустить загрузку через `pg_bulkload`
- Включить обратно индексы и триггеры после успешной загрузки

 Шаг 17:

Миссия 17

Расположите шаги для корректного экспорта и резервного копирования таблицы:

1. Выбрать таблицу для резервного копирования
2. Выполнить COPY TO с указанием файла и формата
3. Проверить созданный файл и его целостность
4. Сохранить файл в безопасном месте

Подсказка: Проверка целостности и безопасное хранение обязательны.

Ответ:

- Выбрать таблицу для резервного копирования
- Выполнить COPY TO с указанием файла и формата
- Проверить созданный файл и его целостность
- Сохранить файл в безопасном месте

 Урок 4 - Параллельное выполнение запросов в PostgreSQL

 Шаг 01:

Миссия 1

Что позволяет выполнять параллельные запросы в PostgreSQL?

Подсказка: Думайте о распределении работы на несколько процессов.

Ответ:

- **Использование параллельных планов выполнения**
- Запуск нескольких серверов PostgreSQL одновременно
- Создание дубликата базы для каждого запроса

- Использование HSTORE для хранения данных

🔮 Шаг 02:

Миссия 2

Какая операция чаще всего выигрывает от параллельного выполнения?

Подсказка: Думайте о больших таблицах и агрегатных вычислениях.

Ответ:

- **Сканирование больших таблиц с агрегатными функциями**
- Малые SELECT запросы по индексу
- Мелкие UPDATE одной строки
- Простое создание таблиц

🔮 Шаг 03:

Миссия 3

Какая настройка отвечает за максимальное число параллельных воркеров?

Подсказка: Ищите параметр, связанный с parallel workers.

Ответ:

- **max_parallel_workers_per_gather**
- max_connections
- shared_buffers
- work_mem

🔮 Шаг 04:

Миссия 4

Сопоставьте понятия с их объяснением в контексте параллельного выполнения.

Понятия:

Ответ:

- Gather
- Parallel Seq Scan
- Parallel Hash Join
- Worker process

Определения:

1. Процесс, который выполняет часть параллельного запроса.
2. Операция объединения таблиц с использованием нескольких потоков.
3. Сбор результатов от нескольких воркеров в один поток.
4. Последовательное сканирование таблицы с использованием нескольких процессов.

Подсказка: Gather собирает результаты, а Worker — выполняет их.

- **Gather** — Сбор результатов от нескольких воркеров в один поток.
- **Parallel Seq Scan** — Последовательное сканирование таблицы с использованием нескольких процессов.
- **Parallel Hash Join** — Операция объединения таблиц с использованием нескольких потоков.
- **Worker process** — Процесс, который выполняет часть параллельного запроса.

Шаг 05:

Миссия 5

Сопоставьте термины и их описание для мониторинга параллельных запросов.

Ответ:

- `pg_stat_activity`
 - `EXPLAIN (ANALYZE)`
 - `pg_stat_progress_parallel_vacuum`
 - `parallel setup cost`
1. Выводит текущие активности всех сессий
 2. Позволяет увидеть план параллельного запроса и затраты
 3. Отслеживает прогресс параллельной очистки таблиц
 4. Порог стоимости для включения параллельного выполнения

Подсказка: Мониторинг — это статистика и планы выполнения.

- **`pg_stat_activity`** — Выводит текущие активности всех сессий
- **`EXPLAIN (ANALYZE)`** — Позволяет увидеть план параллельного запроса и затраты
- **`pg_stat_progress_parallel_vacuum`** — Отслеживает прогресс параллельной очистки таблиц
- **`parallel setup cost`** — Порог стоимости для включения параллельного выполнения

Шаг 06:

Миссия 6

Расположите шаги выполнения параллельного SELECT:

1. Planner решает использовать параллель
2. Запускаются воркеры
3. Воркеры обрабатывают свои сегменты таблицы
4. Gather собирает результаты
5. Результат возвращается клиенту

Подсказка: Сначала планирование, затем выполнение, в конце сбор результатов.

Ответ:

- Planner решает использовать параллель
- Запускаются воркеры
- Воркеры обрабатывают свои сегменты таблицы
- Gather собирает результаты
- Результат возвращается клиенту

Шаг 07:

Миссия 7

Как PostgreSQL определяет, стоит ли использовать параллельный план?

Подсказка: Смотрите на стоимость выполнения запроса и количество строк.

Ответ:

- **На основе оценочной стоимости запроса**
- По случайному выбору
- Всегда используется параллель

- Зависит только от количества соединений

🔮 Шаг 08:

Миссия 8

Что может ограничить эффективность параллельного выполнения?

Подсказка: Подумайте о количестве доступных CPU и структуре запроса.

Ответ:

- **Малое количество строк или ресурсов CPU**
- Использование индексов
- Наличие только одной таблицы
- Наличие триггеров

🔮 Шаг 09:

Миссия 9

Какой оператор часто улучшает производительность при больших таблицах?

Подсказка: Он используется для параллельного сканирования.

Ответ:

- **Parallel Seq Scan**
- Index Scan
- Bitmap Heap Scan
- Nested Loop Join

🔮 Шаг 10:

Миссия 10

Сопоставьте термины параллельного выполнения и их характеристики.

Ответ:

- Gather Merge
 - Parallel Index Scan
 - Set Parallel Degree
 - Parallel Aggregates
1. Собирает отсортированные результаты от воркеров
 2. Сканирование индекса с использованием нескольких процессов
 3. Определяет количество параллельных воркеров
 4. Выполнение агрегатных функций параллельно

Подсказка: Обратите внимание, что разные типы операторов имеют разные способы распределения работы.

- **Gather Merge** — Собирает отсортированные результаты от воркеров
- **Parallel Index Scan** — Сканирование индекса с использованием нескольких процессов
- **Set Parallel Degree** — Определяет количество параллельных воркеров
- **Parallel Aggregates** — Выполнение агрегатных функций параллельно

🔮 Шаг 11:

Миссия 11

Сопоставьте понятия для планирования параллельных операций.

Ответ:

- Parallel Setup Cost
- Parallel Tuple Cost
- Effective Parallelism
- Gather Threshold

1. Стоимость подготовки параллельных воркеров
2. Оценочная стоимость обработки одной строки
3. Достижимое ускорение запроса
4. Минимальная стоимость запроса для параллельного выполнения

Подсказка: Думайте о том, что оценивает планировщик при выборе параллельного плана.

- **Parallel Setup Cost** — Стоимость подготовки параллельных воркеров
- **Parallel Tuple Cost** — Оценочная стоимость обработки одной строки
- **Effective Parallelism** — Достижимое ускорение запроса
- **Gather Threshold** — Минимальная стоимость запроса для параллельного выполнения

 Шаг 12:

Миссия 12

Сопоставьте термины и ситуации использования параллельного выполнения.

Ответ:

- Parallel Bitmap Heap Scan
- Parallel Materialize
- Parallel Merge Join
- Parallel Vacuum

1. Используется для фильтрации большого числа строк с bitmap
2. Сохраняет промежуточные результаты для параллельной обработки
3. Объединение отсортированных потоков параллельно
4. Параллельная очистка больших таблиц

Подсказка: Разные типы операторов подходят для разных целей: фильтрация, объединение, очистка.

- **Parallel Bitmap Heap Scan** — Используется для фильтрации большого числа строк с bitmap
- **Parallel Materialize** — Сохраняет промежуточные результаты для параллельной обработки
- **Parallel Merge Join** — Объединение отсортированных потоков параллельно
- **Parallel Vacuum** — Параллельная очистка больших таблиц

 Шаг 13:

Миссия 13

Расположите этапы выполнения параллельного Hash Join:

1. Планировщик выбирает параллельный Hash Join
2. Создаются воркеры
3. Построение хеш-таблицы в воркерах
4. Сканирование другой таблицы параллельно
5. Сбор результатов через Gather

Подсказка: Сначала планирование, затем создание воркеров и построение структуры, потом сбор.

Ответ:

- Планировщик выбирает параллельный Hash Join
- Создаются воркеры
- Построение хеш-таблицы в воркерах
- Сканирование другой таблицы параллельно
- Сбор результатов через Gather

Шаг 14:

Миссия 14

Расположите шаги параллельного агрегирования:

1. Сканирование таблицы параллельно
2. Вычисление агрегатов в воркерах
3. Сбор промежуточных результатов
4. Финальная агрегация в Gather
5. Возврат результата клиенту

Подсказка: Подумайте о распределении вычислений и их сборе в конце.

Ответ:

- Сканирование таблицы параллельно
- Вычисление агрегатов в воркерах
- Сбор промежуточных результатов
- Финальная агрегация в Gather
- Возврат результата клиенту

Шаг 15:

Миссия 15

Расположите этапы параллельного сканирования с фильтром:

1. Определение необходимости параллелизма
2. Создание воркеров
3. Разделение таблицы на сегменты
4. Применение фильтра к сегментам параллельно
5. Сбор результатов в Gather

Подсказка: Сначала планирование, затем сегментация и параллельная обработка.

Ответ:

- Определение необходимости параллелизма
- Создание воркеров
- Разделение таблицы на сегменты
- Применение фильтра к сегментам параллельно
- Сбор результатов в Gather

Шаг 16:

Миссия 16

Расположите этапы оптимизации параллельного запроса:

1. Анализ узких мест
2. Настройка `max_parallel_workers_per_gather`
3. Использование подходящих операторов (Parallel Seq Scan, Parallel Join)
4. Мониторинг с `pg_stat_activity` и EXPLAIN
5. Тестирование и корректировка

Подсказка: Оптимизация требует анализа, настройки и проверки результатов.

Ответ:

- Анализ узких мест
- Настройка `max_parallel_workers_per_gather`
- Использование подходящих операторов (Parallel Seq Scan, Parallel Join)
- Мониторинг с `pg_stat_activity` и EXPLAIN
- Тестирование и корректировка

Урок 5 - Управление многоконтурными запросами с подзапросами

Шаг 01:

Миссия 1

Что такое подзапрос в PostgreSQL?

Подсказка: Это запрос внутри другого запроса.

Ответ:

- **Запрос, вложенный внутри другого запроса**
- Запрос к внешней базе данных
- Запрос к таблице без условий
- Любой SQL-запрос с JOIN

Шаг 02:

Миссия 2

Когда подзапрос возвращает одно значение, какой тип подзапроса используется?

Подсказка: Смотрите на агрегаты и одиночные результаты.

Ответ:

- **Скалярный подзапрос**
- Коррелированный подзапрос
- IN-подзапрос
- EXISTS-подзапрос

Шаг 03:

Миссия 3

Для чего используют коррелированные подзапросы?

Подсказка: Они зависят от строк внешнего запроса.

Ответ:

- **Чтобы использовать значения текущей строки внешнего запроса**
- Чтобы ускорить обычный SELECT
- Для работы с внешними соединениями

- Чтобы создать временную таблицу

Шаг 04:

Миссия 4

Сопоставьте тип подзапроса с его особенностью.

Ответ:

- Скалярный подзапрос
 - IN-подзапрос
 - EXISTS-подзапрос
 - Коррелированный подзапрос
1. Возвращает одно значение
 2. Проверяет наличие значения в списке
 3. Проверяет существование строк
 4. Зависит от текущей строки внешнего запроса

Подсказка: Обратите внимание на количество возвращаемых строк и зависимость от внешнего запроса.

- **Скалярный подзапрос** — Возвращает одно значение
- **IN-подзапрос** — Проверяет наличие значения в списке
- **EXISTS-подзапрос** — Проверяет существование строк
- **Коррелированный подзапрос** — Зависит от текущей строки внешнего запроса

Шаг 05:

Миссия 5

Сопоставьте ключевые слова с их функционалом в подзапросах.

Ответ:

- ANY
 - ALL
 - EXISTS
 - NOT EXISTS
1. Сравнивает с любым значением из подзапроса
 2. Сравнивает со всеми значениями из подзапроса
 3. Проверяет наличие хотя бы одной строки
 4. Проверяет отсутствие строк

Подсказка: Подумайте, как эти операторы изменяют условие в WHERE.

- **ANY** — Сравнивает с любым значением из подзапроса
- **ALL** — Сравнивает со всеми значениями из подзапроса
- **EXISTS** — Проверяет наличие хотя бы одной строки
- **NOT EXISTS** — Проверяет отсутствие строк

Шаг 06:

Миссия 6

Сопоставьте виды вложенных запросов с примером использования.

Ответ:

- Скалярный подзапрос
 - IN-подзапрос
 - EXISTS-подзапрос
 - Коррелированный подзапрос
1. SELECT AVG(salary) FROM employees
 2. SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers)
 3. SELECT * FROM products WHERE EXISTS (SELECT 1 FROM inventory WHERE product_id = products.id)
 4. SELECT * FROM sales s WHERE s.amount > (SELECT AVG(amount) FROM sales WHERE region = s.region)

Подсказка: Смотрите на контекст использования и зависимость от внешнего запроса.

- **Скалярный подзапрос** — SELECT AVG(salary) FROM employees
- **IN-подзапрос** — SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers)
- **EXISTS-подзапрос** — SELECT * FROM products WHERE EXISTS (SELECT 1 FROM inventory WHERE product_id = products.id)
- **Коррелированный подзапрос** — SELECT * FROM sales s WHERE s.amount > (SELECT AVG(amount) FROM sales WHERE region = s.region)

 Шаг 07:

Миссия 7

Что происходит, если подзапрос возвращает несколько значений, а внешняя часть ожидает одно?

Подсказка: Подумайте о скалярных подзапросах и их ограничениях.

Ответ:

- **Произойдет ошибка выполнения**
- В PostgreSQL вернется первое значение
- Подзапрос автоматически преобразуется в список
- Внешний запрос игнорирует подзапрос

 Шаг 08:

Миссия 8

Какой тип подзапроса лучше использовать для фильтрации по множеству значений?

Подсказка: Подумайте о конструкции IN.

Ответ:

- **IN-подзапрос**
- Скалярный подзапрос
- EXISTS-подзапрос
- Коррелированный подзапрос

 Шаг 09:

Миссия 9

Сопоставьте типы подзапросов с типичными ошибками.

Ответ:

- Скалярный подзапрос

- IN-подзапрос
- EXISTS-подзапрос
- Коррелированный подзапрос

1. Ошибка, если возвращает больше одного значения
2. Может вернуть неожиданные дубли при отсутствии DISTINCT
3. Ошибки редки, проверяет только наличие строк
4. Может быть медленным при больших таблицах

Подсказка: Подумайте, какие ошибки характерны для каждого типа подзапроса.

- **Скалярный подзапрос** — Ошибка, если возвращает больше одного значения
- **IN-подзапрос** — Может вернуть неожиданные дубли при отсутствии DISTINCT
- **EXISTS-подзапрос** — Ошибки редки, проверяет только наличие строк
- **Коррелированный подзапрос** — Может быть медленным при больших таблицах

Шаг 10:

Миссия 10

Сопоставьте ключевые слова с их поведением в запросе.

Ответ:

- EXISTS
 - NOT EXISTS
 - ANY
 - ALL
1. Возвращает TRUE, если хотя бы одна строка подзапроса существует
 2. Возвращает TRUE, если ни одной строки нет
 3. Сравнивает с любым значением из подзапроса
 4. Сравнивает со всеми значениями подзапроса

Подсказка: Подумайте, как логика подзапроса влияет на результат WHERE.

- **EXISTS** — Возвращает TRUE, если хотя бы одна строка подзапроса существует
- **NOT EXISTS** — Возвращает TRUE, если ни одной строки нет
- **ANY** — Сравнивает с любым значением из подзапроса
- **ALL** — Сравнивает со всеми значениями подзапроса

Шаг 11:

Миссия 11

Почему коррелированные подзапросы могут работать медленно?

Подсказка: Каждый внешний ряд вызывает выполнение подзапроса.

Ответ:

- **Подзапрос выполняется для каждой строки внешнего запроса**
- Подзапрос выполняется один раз и кэшируется
- PostgreSQL всегда распараллеливает подзапросы автоматически
- Подзапрос не влияет на скорость выполнения

Шаг 12:

Миссия 12

Как улучшить производительность многоконтурных запросов?

Подсказка: Смотрите на индексы, JOIN и агрегации.

Ответ:

- **Использовать индексы для столбцов в подзапросах**
- **Переписать подзапрос как JOIN, если возможно**
- Добавлять больше подзапросов для каждой таблицы
- Игнорировать условия WHERE

Шаг 13:

Миссия 13

Расположите шаги оптимизации подзапросов в правильном порядке:

1. Проанализировать план выполнения запроса
2. Добавить индексы для столбцов, участвующих в фильтрах
3. Переписать подзапрос как JOIN, если возможно
4. Тестировать производительность после изменений

Подсказка: Сначала нужно понять проблему, потом вносить изменения, и только в конце проверять результат.

Ответ:

- Проанализировать план выполнения запроса
- Добавить индексы для столбцов, участвующих в фильтрах
- Переписать подзапрос как JOIN, если возможно
- Тестировать производительность после изменений

Шаг 14:

Миссия 14

Расположите последовательность выполнения коррелированного подзапроса:

1. Внешний SELECT выбирает строку
2. Выполняется подзапрос для этой строки
3. Возвращается результат подзапроса
4. Внешний SELECT формирует итоговый набор

Подсказка: Подумайте, как каждая строка внешнего запроса взаимодействует с подзапросом.

Ответ:

- Внешний SELECT выбирает строку
- Выполняется подзапрос для этой строки
- Возвращается результат подзапроса
- Внешний SELECT формирует итоговый набор

Шаг 15:

Миссия 15

Что использовать, если нужно проверить наличие строк, но не нужны сами данные?

Подсказка: Смотрите на EXISTS и NOT EXISTS.

Ответ:

- **EXISTS-подзапрос**
- Скалярный подзапрос

- IN-подзапрос
- ALL-подзапрос

Шаг 16:

Миссия 16

Расположите шаги написания многоконтурного запроса с подзапросами:

1. Определить, какие данные нужны в результате
2. Выбрать внешнюю таблицу для основного SELECT
3. Спроектировать подзапросы для фильтров и агрегатов
4. Тестировать запрос на корректность и производительность

Подсказка: Сначала проектируем, потом строим подзапросы, затем проверяем результат.

Ответ:

- Определить, какие данные нужны в результате
- Выбрать внешнюю таблицу для основного SELECT
- Спроектировать подзапросы для фильтров и агрегатов
- Тестировать запрос на корректность и производительность

Шаг 17:

Миссия 17

Какой подход предпочтительнее при оптимизации подзапросов для больших таблиц?

Подсказка: Подумайте о JOIN и индексах вместо многократных сканирований.

Ответ:

- **Использовать JOIN вместо коррелированного подзапроса, если возможно**
- **Создать индексы на столбцы, участвующие в фильтрах подзапросов**
- Добавлять подзапросы к каждой таблице без анализа
- Игнорировать выполнение плана запроса

Модуль 5 - Интеграция и взаимодействие с внешними системами

Урок 1 - Интеграция PostgreSQL с внешними API через PHP

Шаг 01:

Миссия 1

Аня создает простой скрипт на PHP, чтобы получить данные с внешнего API и сохранить их в PostgreSQL. Какой подход безопаснее всего для передачи параметров в запрос?

Подсказка: Думайте о защите от SQL-инъекций.

Ответ:

- **Использовать подготовленные выражения (prepared statements)**
- Вставлять переменные напрямую в SQL
- Формировать SQL через конкатенацию строк
- Игнорировать параметры и использовать значения по умолчанию

Шаг 02:

Миссия 2

Борис пишет PHP-код для получения JSON от внешнего сервиса. Какой метод лучше использовать для безопасного преобразования JSON в массив PHP?

Подсказка: PHP умеет декодировать строки JSON в массивы.

Ответ:

- **json_decode(\$json, true)**
- serialize(\$json)
- var_export(\$json)
- htmlspecialchars(\$json)

Шаг 03:

Миссия 3

Ирина хочет кэшировать данные API перед сохранением в PostgreSQL. Какой способ кэширования подходит для быстрого прототипа?

Подсказка: Иногда достаточно временно хранить данные в памяти или в файле.

Ответ:

- **Использовать файл или массив в памяти для временного хранения**
- Записывать все данные напрямую в PostgreSQL без кэша
- Отправлять данные в лог только для отладки
- Использовать FTP для хранения данных

Шаг 04:

Миссия 4

Павел интегрирует внешний API для получения курсов валют. Какой формат передачи данных предпочтителен для последующей обработки в PostgreSQL?

Подсказка: Думайте о структурированных данных, которые легко парсить.

Ответ:

- **JSON**
- Plain text
- HTML
- PDF

Шаг 05:

Миссия 5

 Задача: Сопоставьте термины интеграции API с их описанием.

Ответ:

- GET-запрос
- POST-запрос
- JSON
- REST API

1. Запрос для получения данных с сервера.
2. Отправка данных на сервер.
3. Структурированный формат данных, легко читаемый PHP.

4. Архитектура для взаимодействия клиент-сервер через HTTP.

Подсказка: Думайте о том, как браузер общается с сервером и как сервер отправляет данные обратно.

- **GET-запрос** — Запрос для получения данных с сервера.
- **POST-запрос** — Отправка данных на сервер.
- **JSON** — Структурированный формат данных, легко читаемый PHP.
- **REST API** — Архитектура для взаимодействия клиент-сервер через HTTP.

Шаг 06:

Миссия 6

 Задача: Сопоставьте методы работы с PostgreSQL через PHP с их назначением.

Ответ:

- PDO
 - pg_connect
 - prepare
 - execute
1. Объектно-ориентированное подключение к базе данных.
 2. Функция для установления соединения с PostgreSQL.
 3. Подготовка SQL-запроса для безопасного выполнения.
 4. Выполнение подготовленного запроса.

Подсказка: Разделяйте создание запроса и его выполнение.

- **PDO** — Объектно-ориентированное подключение к базе данных.
- **pg_connect** — Функция для установления соединения с PostgreSQL.
- **prepare** — Подготовка SQL-запроса для безопасного выполнения.
- **execute** — Выполнение подготовленного запроса.

Шаг 07:

Миссия 7

 Сопоставьте ошибки интеграции с их вероятной причиной.

Ответ:

- Ошибка 500
 - Ошибка 404
 - Ошибка SQLSTATE
 - Ошибка JSON
1. Проблема на стороне сервера.
 2. Запрашиваемый ресурс не найден.
 3. Некорректный SQL-запрос или ошибка БД.
 4. Некорректный формат данных.

Подсказка: Смотрите на код ошибки и контекст запроса.

- **Ошибка 500** — Проблема на стороне сервера.
- **Ошибка 404** — Запрашиваемый ресурс не найден.
- **Ошибка SQLSTATE** — Некорректный SQL-запрос или ошибка БД.
- **Ошибка JSON** — Некорректный формат данных.

Шаг 08:

Миссия 8

 Сопоставьте методы оптимизации интеграции с их эффектом.

Ответ:

- Использование транзакций
 - Пакетная вставка данных
 - Кэширование ответов API
 - Логирование ошибок
1. Повышает надежность операций с БД.
 2. Снижает количество запросов к базе.
 3. Уменьшает нагрузку на внешние сервисы.
 4. Помогает отслеживать проблемы интеграции.

Подсказка: Думайте о скорости, надежности и отладке.

- **Использование транзакций** — Повышает надежность операций с БД.
- **Пакетная вставка данных** — Снижает количество запросов к базе.
- **Кэширование ответов API** — Уменьшает нагрузку на внешние сервисы.
- **Логирование ошибок** — Помогает отслеживать проблемы интеграции.

Шаг 09:

Миссия 9

 Сопоставьте термины для работы с API и PostgreSQL в PHP с их особенностью.

Ответ:

- fetchAll()
 - cURL
 - bindParam
 - json_encode
1. Получение всех строк результата запроса.
 2. Отправка HTTP-запросов к внешнему API.
 3. Привязка значения к параметру подготовленного запроса.
 4. Преобразование PHP-массива в JSON.

Подсказка: Обратите внимание на направление данных: из PHP в API или наоборот.

- **fetchAll()** — Получение всех строк результата запроса.
- **cURL** — Отправка HTTP-запросов к внешнему API.
- **bindParam** — Привязка значения к параметру подготовленного запроса.
- **json_encode** — Преобразование PHP-массива в JSON.

Шаг 10:

Миссия 10

Расположите шаги интеграции внешнего API в PostgreSQL через PHP в правильном порядке:

1. Обработка ответа API
2. Отправка HTTP-запроса к API
3. Подключение к PostgreSQL
4. Сохранение данных в базу

Подсказка: Сначала нужно подключиться к базе и запросить данные, затем обработать их и сохранить.

Ответ:

- Подключение к PostgreSQL
- Отправка HTTP-запроса к API
- Обработка ответа API
- Сохранение данных в базу

Шаг 11:

Миссия 11

Расположите этапы отладки интеграции API в правильном порядке:

1. Проверка структуры данных
2. Отслеживание ошибок HTTP
3. Логирование SQL-запросов
4. Тестирование сохранения в PostgreSQL

Подсказка: Сначала убедитесь, что данные приходят, потом проверьте сохранение.

Ответ:

- Отслеживание ошибок HTTP
- Проверка структуры данных
- Тестирование сохранения в PostgreSQL
- Логирование SQL-запросов

Шаг 12:

Миссия 12

Расположите действия при обработке большого объема данных из API:

1. Разбивка данных на пакеты
2. Отправка запросов к API
3. Сохранение пакета в PostgreSQL
4. Повторение для всех пакетов

Подсказка: Подумайте о том, как избежать перегрузки сервера и БД.

Ответ:

- Разбивка данных на пакеты
- Отправка запросов к API
- Сохранение пакета в PostgreSQL
- Повторение для всех пакетов

Шаг 13:

Миссия 13

Расположите шаги по обеспечению безопасности при интеграции с внешним API:

1. Использование HTTPS
2. Валидация данных
3. Использование токенов доступа
4. Логирование попыток доступа

Подсказка: Сначала защищаем канал, затем проверяем данные, а потом фиксируем события.

Ответ:

- Использование HTTPS
- Валидация данных
- Использование токенов доступа
- Логирование попыток доступа

Шаг 14:

Миссия 14

Расположите шаги оптимизации запросов к PostgreSQL при интеграции с API:

1. Использование индексов
2. Подготовленные выражения
3. Пакетная вставка данных
4. Проверка плана выполнения запросов

Подсказка: Сначала создаем структуру, потом используем эффективные методы вставки и проверяем результат.

Ответ:

- Использование индексов
- Подготовленные выражения
- Пакетная вставка данных
- Проверка плана выполнения запросов

Шаг 15:

Миссия 15

Расположите шаги масштабирования интеграции API для крупной системы:

1. Вертикальное масштабирование базы данных
2. Использование очередей для обработки запросов
3. Горизонтальное масштабирование приложений
4. Мониторинг нагрузки и оптимизация

Подсказка: Сначала расширяем БД, потом распределяем нагрузку, и в конце анализируем производительность.

Ответ:

- Вертикальное масштабирование базы данных
- Использование очередей для обработки запросов
- Горизонтальное масштабирование приложений
- Мониторинг нагрузки и оптимизация

Шаг 16:

Миссия 16

Расположите этапы отладки и улучшения интеграции API с PostgreSQL в крупной системе:

1. Сбор статистики ошибок и метрик
2. Анализ узких мест и проблем производительности
3. Внесение изменений в код и конфигурацию
4. Повторное тестирование и проверка результатов

Подсказка: Сначала фиксируем и анализируем, потом исправляем и проверяем.

Ответ:

- Сбор статистики ошибок и метрик
- Анализ узких мест и проблем производительности
- Внесение изменений в код и конфигурацию
- Повторное тестирование и проверка результатов

Урок 2 - Использование репликации PostgreSQL для масштабирования

Шаг 01:

Миссия 1

Алексей впервые настраивает репликацию PostgreSQL для тестового сервера. Что необходимо сделать в первую очередь?

Подсказка: Начните с подготовки базового соединения между главной и репликой.

Ответ:

- **Настроить соединение master-slave и права доступа**
- Сразу включить логирование всех транзакций
- Развернуть реплику без проверки сети
- Просто скопировать базу на второй сервер

Шаг 02:

Миссия 2

Марина хочет настроить потоковую репликацию PostgreSQL. Какой параметр важен для синхронизации данных в реальном времени?

Подсказка: Думайте о настройках, влияющих на задержку передачи изменений.

Ответ:

- **wal_level = replica**
- max_connections = 10
- shared_buffers = 128MB
- autovacuum = off

Шаг 03:

Миссия 3

Иван проверяет состояние реплики с помощью pg_stat_replication. Что можно узнать из этой команды?

Подсказка: Подумайте, какие показатели важны для репликации: соединение, задержка, поток.

Ответ:

- **Статус реплик и задержку репликации**
- Список всех таблиц в базе
- Количество активных пользователей
- Время последнего бэкапа

Шаг 04:

Миссия 4

Николай хочет предотвратить потерю данных при сбое главного сервера. Какой параметр репликации поможет?

Подсказка: Подумайте о строгой записи WAL на диск.

Ответ:

- **synchronous_commit = on**
- autovacuum = off
- max_wal_size = 1GB
- log_min_duration_statement = 0

Шаг 05:

Миссия 5

 Сопоставьте термины репликации PostgreSQL с их описанием.

Ответ:

- Master
 - Replica
 - WAL
 - Hot Standby
1. Основной сервер, на котором происходят изменения данных.
 2. Сервер, получающий изменения от главного.
 3. Журнал транзакций для синхронизации реплик.
 4. Режим реплики, позволяющий читать данные во время репликации.

Подсказка: Подумайте, кто пишет изменения, а кто их читает и применяет.

- **Master** — Основной сервер, на котором происходят изменения данных.
- **Replica** — Сервер, получающий изменения от главного.
- **WAL** — Журнал транзакций для синхронизации реплик.
- **Hot Standby** — Режим реплики, позволяющий читать данные во время репликации.

Шаг 06:

Миссия 6

 Сопоставьте типы репликации с их особенностями.

Ответ:

- Streaming Replication
 - Logical Replication
 - Physical Replication
 - Asynchronous Replication
1. Передача WAL напрямую в реальном времени.
 2. Репликация отдельных таблиц или данных по логическим идентификаторам.
 3. Копирование всего физического состояния базы данных.
 4. Репликация без гарантии задержки; может быть небольшая задержка.

Подсказка: Думайте о том, что именно копируется: физический диск или отдельные таблицы.

- **Streaming Replication** — Передача WAL напрямую в реальном времени.
- **Logical Replication** — Репликация отдельных таблиц или данных по логическим идентификаторам.
- **Physical Replication** — Копирование всего физического состояния базы данных.

- **Asynchronous Replication** — Репликация без гарантии задержки; может быть небольшая задержка.

Шаг 07:

Миссия 7

 Сопоставьте команды PostgreSQL с их функцией для репликации.

Ответ:

- `pg_basebackup`
 - `pg_ctl promote`
 - `pg_stat_replication`
 - `pg_receivewal`
1. Создание физической копии базы для реплики.
 2. Перевод реплики в главный сервер.
 3. Проверка состояния подключений реплик.
 4. Получение WAL-файлов для восстановления или репликации.

Подсказка: Обратите внимание на назначение каждой команды: резервное копирование, мониторинг, продвижение.

- **`pg_basebackup`** — Создание физической копии базы для реплики.
- **`pg_ctl promote`** — Перевод реплики в главный сервер.
- **`pg_stat_replication`** — Проверка состояния подключений реплик.
- **`pg_receivewal`** — Получение WAL-файлов для восстановления или репликации.

Шаг 08:

Миссия 8

 Сопоставьте параметры PostgreSQL с их влиянием на репликацию.

Ответ:

- `max_wal_senders`
 - `wal_keep_segments`
 - `synchronous_standby_names`
 - `hot_standby`
1. Количество процессов для отправки WAL на реплики.
 2. Сколько WAL-сегментов хранить для реплик.
 3. Указывает реплики для синхронной репликации.
 4. Разрешает чтение с реплики во время репликации.

Подсказка: Думайте, какие параметры управляют отправкой, хранением и доступом.

- **`max_wal_senders`** — Количество процессов для отправки WAL на реплики.
- **`wal_keep_segments`** — Сколько WAL-сегментов хранить для реплик.
- **`synchronous_standby_names`** — Указывает реплики для синхронной репликации.
- **`hot_standby`** — Разрешает чтение с реплики во время репликации.

Шаг 09:

Миссия 9

 Сопоставьте термины отказоустойчивости с их назначением.

Ответ:

- Failover
- Switchover
- Standby
- Replication Slot

1. Автоматическое переключение на резервный сервер при сбое.
2. Ручное переключение главного и реплики без потери данных.
3. Резервная реплика для чтения или готовности к продвижению.
4. Механизм удержания WAL для реплики.

Подсказка: Думайте о сценариях аварийного восстановления и поддержки реплик.

- **Failover** — Автоматическое переключение на резервный сервер при сбое.
- **Switchover** — Ручное переключение главного и реплики без потери данных.
- **Standby** — Резервная реплика для чтения или готовности к продвижению.
- **Replication Slot** — Механизм удержания WAL для реплики.

 Шаг 10:

Миссия 10

Расположите шаги настройки физической реплики PostgreSQL:

1. Настройка конфигурации реплики
2. Создание резервной копии с помощью pg_basebackup
3. Запуск реплики и подключение к главной БД
4. Проверка состояния с помощью pg_stat_replication

Подсказка: Сначала создаем копию, потом конфигурируем и проверяем работу.

Ответ:

- Создание резервной копии с помощью pg_basebackup
- Настройка конфигурации реплики
- Запуск реплики и подключение к главной БД
- Проверка состояния с помощью pg_stat_replication

 Шаг 11:

Миссия 11

Расположите шаги настройки логической репликации:

1. Создание публикации на главном сервере
2. Создание подписки на реплике
3. Проверка синхронизации таблиц
4. Мониторинг задержки и ошибок

Подсказка: Логическая репликация начинается с публикации изменений, затем подписка и контроль.

Ответ:

- Создание публикации на главном сервере
- Создание подписки на реплике
- Проверка синхронизации таблиц
- Мониторинг задержки и ошибок

Шаг 12:

Миссия 12

Расположите действия для безопасного failover:

1. Остановить главную БД
2. Продвинуть реплику в главный сервер
3. Перенастроить другие реплики на новый мастер
4. Проверить корректность работы приложений

Подсказка: Сначала останавливаем, потом переводим и проверяем систему.

Ответ:

- Остановить главную БД
- Продвинуть реплику в главный сервер
- Перенастроить другие реплики на новый мастер
- Проверить корректность работы приложений

Шаг 13:

Миссия 13

Расположите этапы мониторинга репликации:

1. Сбор статистики из `pg_stat_replication`
2. Анализ задержки реплик
3. Логирование ошибок и предупреждений
4. Принятие мер при проблемах

Подсказка: Сначала собираем данные, затем анализируем и реагируем.

Ответ:

- Сбор статистики из `pg_stat_replication`
- Анализ задержки реплик
- Логирование ошибок и предупреждений
- Принятие мер при проблемах

Шаг 14:

Миссия 14

Расположите шаги оптимизации репликации при большом объеме данных:

1. Настройка `wal_keep_segments`
2. Использование потоковой репликации
3. Мониторинг пропускной способности сети
4. Регулярное архивация WAL

Подсказка: Сначала настраиваем хранение, потом передачу, контроль и архивирование.

Ответ:

- Настройка `wal_keep_segments`
- Использование потоковой репликации
- Мониторинг пропускной способности сети
- Регулярное архивация WAL

Шаг 15:

Миссия 15

Расположите шаги восстановления реплики после сбоя:

1. Удаление поврежденных данных или файлов WAL
2. Создание новой резервной копии с главного сервера
3. Настройка конфигурации реплики и подключение к мастеру
4. Проверка синхронизации и корректности данных

Подсказка: Сначала устраняем проблемы, потом создаем новую копию, подключаем и проверяем работу.

Ответ:

- Удаление поврежденных данных или файлов WAL
- Создание новой резервной копии с главного сервера
- Настройка конфигурации реплики и подключение к мастеру
- Проверка синхронизации и корректности данных

Шаг 16:

Миссия 16

Расположите шаги масштабирования системы с использованием нескольких реплик:

1. Добавление новых реплик для распределения нагрузки
2. Настройка балансировщика чтения запросов
3. Мониторинг состояния всех реплик
4. Оптимизация параметров репликации для снижения задержки

Подсказка: Сначала увеличиваем количество реплик, затем распределяем запросы, мониторим и оптимизируем.

Ответ:

- Добавление новых реплик для распределения нагрузки
- Настройка балансировщика чтения запросов
- Мониторинг состояния всех реплик
- Оптимизация параметров репликации для снижения задержки

Шаг 17:

Миссия 17

Расположите шаги регулярного обслуживания репликации для крупной системы:

1. Архивирование и очистка старых WAL-файлов
2. Обновление конфигураций при изменении нагрузки
3. Тестирование failover и восстановления реплик
4. Проверка синхронизации и корректности данных

Подсказка: Подумайте, что нужно делать регулярно: архивирование, обновление, тестирование и контроль.

Ответ:

- Архивирование и очистка старых WAL-файлов
- Обновление конфигураций при изменении нагрузки
- Тестирование failover и восстановления реплик

- Проверка синхронизации и корректности данных

Урок 3 - Использование Foreign Data Wrappers для интеграции

Шаг 01:

Миссия 1

Анна впервые использует FDW для подключения к внешней MySQL-базе. Что нужно сделать в первую очередь?

Подсказка: Подумайте, с чего начинается любое внешнее подключение.

Ответ:

- **Установить соответствующий FDW и создать сервер**
- Сразу создавать представления и запросы
- Копировать все данные локально
- Изменить локальную базу на MySQL-формат

Шаг 02:

Миссия 2

Сергей хочет выполнять запросы к внешней базе через FDW. Какой объект PostgreSQL ему нужно создать?

Подсказка: Подумайте о том, что представляет собой соединение с внешним источником.

Ответ:

- **FOREIGN TABLE**
- VIEW
- MATERIALIZED VIEW
- SEQUENCE

Шаг 03:

Миссия 3

Иван хочет использовать FDW для чтения и записи в удалённую базу. Какой параметр необходимо правильно настроить?

Подсказка: Думайте о доступе: кто может читать и писать данные.

Ответ:

- **OPTIONS (user, password) при создании сервера**
- owner базы данных PostgreSQL
- локальные права на таблицы
- autovacuum для FDW

Шаг 04:

Миссия 4

Ольга хочет подключить PostgreSQL к внешней CSV-базе через FDW. Какой FDW подойдет лучше всего?

Подсказка: Подумайте, какие существуют расширения для файловых источников.

Ответ:

- **file_fdw**
- mysql_fdw

- postgres_fdw
- oracle_fdw

Шаг 05:

Миссия 5

 Сопоставьте объекты FDW с их назначением:

Ответ:

- SERVER
- USER MAPPING
- FOREIGN TABLE
- FDW

1. Определяет соединение с внешним источником
2. Соответствие пользователей PostgreSQL и внешней базы
3. Таблица в PostgreSQL, которая отображает внешние данные
4. Расширение для работы с внешними источниками

Подсказка: Думайте, что создается для соединения, что для пользователя, что для таблицы.

- **SERVER** — Определяет соединение с внешним источником
- **USER MAPPING** — Соответствие пользователей PostgreSQL и внешней базы
- **FOREIGN TABLE** — Таблица в PostgreSQL, которая отображает внешние данные
- **FDW** — Расширение для работы с внешними источниками

Шаг 06:

Миссия 6

 Сопоставьте тип FDW с внешней системой:

Ответ:

- postgres_fdw
- mysql_fdw
- file_fdw
- oracle_fdw

1. Подключение к PostgreSQL
2. Подключение к MySQL
3. Чтение данных из файлов
4. Подключение к Oracle

Подсказка: Подумайте, какое расширение поддерживает какую СУБД.

- **postgres_fdw** — Подключение к PostgreSQL
- **mysql_fdw** — Подключение к MySQL
- **file_fdw** — Чтение данных из файлов
- **oracle_fdw** — Подключение к Oracle

Шаг 07:

Миссия 7

 Сопоставьте команды FDW с их функцией:

Ответ:

- CREATE SERVER
- CREATE USER MAPPING
- IMPORT FOREIGN SCHEMA
- SELECT * FROM foreign_table

1. Создание подключения к внешнему серверу
2. Настройка учетной записи для внешней базы
3. Импорт структуры таблиц из внешней базы
4. Чтение данных из внешней таблицы

Подсказка: Думайте, что создается, что импортируется, а что используется для запросов.

- **CREATE SERVER** — Создание подключения к внешнему серверу
- **CREATE USER MAPPING** — Настройка учетной записи для внешней базы
- **IMPORT FOREIGN SCHEMA** — Импорт структуры таблиц из внешней базы
- **SELECT * FROM foreign_table** — Чтение данных из внешней таблицы

 Шаг 08:

Миссия 8

 Сопоставьте типы операций FDW с их особенностями:

Ответ:

- READ
- INSERT
- UPDATE
- DELETE

1. Чтение данных из внешней таблицы
2. Добавление записей во внешнюю таблицу
3. Изменение существующих данных
4. Удаление записей из внешнего источника

Подсказка: Подумайте, какие операции поддерживаются для внешних таблиц.

- **READ** — Чтение данных из внешней таблицы
- **INSERT** — Добавление записей во внешнюю таблицу
- **UPDATE** — Изменение существующих данных
- **DELETE** — Удаление записей из внешнего источника

 Шаг 09:

Миссия 9

 Сопоставьте параметры CREATE SERVER с их назначением:

Ответ:

- OPTIONS
- FOREIGN DATA WRAPPER
- OWNER
- VERSION

1. Дополнительные настройки соединения
2. Выбор типа FDW для сервера

3. Владелец сервера
4. Версия внешней СУБД

Подсказка: Думайте, что задает тип FDW, что — параметры соединения и права.

- **OPTIONS** — Дополнительные настройки соединения
- **FOREIGN DATA WRAPPER** — Выбор типа FDW для сервера
- **OWNER** — Владелец сервера
- **VERSION** — Версия внешней СУБД

Шаг 10:

Миссия 10

Расположите шаги подключения к внешней базе через FDW:

1. Установка расширения FDW
2. Создание SERVER
3. Создание USER MAPPING
4. Создание FOREIGN TABLE
5. Запрос к внешней таблице

Подсказка: Сначала подключение, потом пользователь, таблица, и только потом запросы.

Ответ:

- Установка расширения FDW
- Создание SERVER
- Создание USER MAPPING
- Создание FOREIGN TABLE
- Запрос к внешней таблице

Шаг 11:

Миссия 11

Расположите шаги для импорта внешней схемы:

1. Создать сервер и пользовательскую карту
2. Использовать команду IMPORT FOREIGN SCHEMA
3. Проверить созданные FOREIGN TABLE
4. Настроить разрешения на чтение

Подсказка: Сначала создаем соединение, потом импортируем схему и настраиваем права.

Ответ:

- Создать сервер и пользовательскую карту
- Использовать команду IMPORT FOREIGN SCHEMA
- Проверить созданные FOREIGN TABLE
- Настроить разрешения на чтение

Шаг 12:

Миссия 12

Аня — разработчица, она хочет подключить внешнюю таблицу PostgreSQL через FDW. Какой шаг она должна сделать первой?

Подсказка: Чтобы подключить внешние данные, сначала нужно установить расширение.

Ответ:

- **Создать расширение postgres_fdw или другое соответствующее**
- Сразу писать SQL-запрос к удалённой таблице
- Создать материализованный вид
- Запросить репликацию данных

🔮 **Шаг 13:**

Миссия 13

Борис хочет подключить таблицу из удалённой базы. Какой параметр DSN для FDW нужен обязательно?

Подсказка: DSN — строка соединения к удалённому серверу PostgreSQL.

Ответ:

- **host=... port=... dbname=... user=... password=...**
- schema=... table=... index=...
- FOREIGN TABLE имя TABLE ...
- GRANT ALL ON FDW

🔮 **Шаг 14:**

Миссия 14

Валерия хочет разрешить чтение внешней таблицы, но не запись. Какой параметр в IMPORT FOREIGN SCHEMA или CREATE FOREIGN TABLE она должна использовать?

Подсказка: В FDW есть атрибуты с access (access options).

Ответ:

- **options (updatable 'false')**
- with (readonly = 'true')
- GRANT SELECT, INSERT
- WITH (writable = false)

🔮 **Шаг 15:**

Миссия 15

Семён уже подключил внешнюю таблицу, но получил ошибку «permission denied». Что он мог забыть?

Подсказка: Не только расширение, но и права доступа на сервере и локально важны.

Ответ:

- **Не выдал права на удалённую таблицу (GRANT) для пользователя FDW**
- Не создал локальный индекс
- Не очистил кэш PostgreSQL
- Не применил VACUUM

🔮 **Шаг 16:**

Миссия 16

Ирина хочет импортировать все таблицы из схемы удалённой БД через FDW. Какой SQL-команда ей подойдёт?

Подсказка: Есть команда IMPORT FOREIGN SCHEMA.

Ответ:

- **IMPORT FOREIGN SCHEMA remote_schema INTO local_schema OPTIONS (...)**
- COPY remote::schema TO local
- SELECT * FROM FOREIGN_SCHEMA()
- CREATE MATERIALIZED VIEW FROM remote_schema

🔗 **Шаг 17:**

Миссия 17

Сопоставьте понятие FDW с его описанием.

Понятия:

Ответ:

- FOREIGN TABLE
- IMPORT FOREIGN SCHEMA
- SERVER
- USER MAPPING

Определения:

1. Описывает удалённый сервер и способ подключения.
2. Позволяет импортировать таблицы целой схемы.
3. Отдельная таблица в локальной БД, ссылающаяся на внешний ресурс.
4. Определяет, с каким пользователем и правами подключаться к удалённому серверу.

Подсказка: Подумайте, какое звено отвечает за схему, создание таблицы, права и сам сервер.

- **FOREIGN TABLE** — Отдельная таблица в локальной БД, ссылающаяся на внешний ресурс.
- **IMPORT FOREIGN SCHEMA** — Позволяет импортировать таблицы целой схемы.
- **SERVER** — Описывает удалённый сервер и способ подключения.
- **USER MAPPING** — Определяет, с каким пользователем и правами подключаться к удалённому серверу.

📖 Урок 4 - Интеграция с REST и SOAP API для взаимодействия

🔗 **Шаг 01:**

Миссия 1

Анна хочет получить данные с внешнего REST API через PHP. Что нужно сделать первым шагом?

Подсказка: Любое взаимодействие с внешним сервисом начинается с установления соединения и проверки эндпоинта.

Ответ:

- **Отправить HTTP-запрос на API-эндпоинт**
- Сначала создавать локальную таблицу
- Вставить данные напрямую в PostgreSQL
- Игнорировать ключ API и сразу делать запрос

🔗 **Шаг 02:**

Миссия 2

Сергей интегрирует SOAP-сервис с PHP. Какой инструмент PHP лучше использовать для взаимодействия с SOAP?

Подсказка: Подумайте о стандартных расширениях PHP для работы с веб-сервисами.

Ответ:

- **SOAPClient**
- cURL только для REST
- mysqli
- file_get_contents без SOAP

Шаг 03:

Миссия 3

Иван хочет интегрировать REST API и сохранять данные в PostgreSQL. Какой подход эффективнее всего?

Подсказка: Сначала получаем данные, затем обрабатываем их перед вставкой в базу.

Ответ:

- **Получить JSON, обработать и вставить в таблицу**
- Создать пустую таблицу и ждать данных
- Вставлять сырые данные без обработки
- Игнорировать структуру данных API

Шаг 04:

Миссия 4

Анна хочет обрабатывать ошибки при вызове REST API. Какой метод наиболее правильный?

Подсказка: Любой внешний сервис может вернуть ошибку — это нужно учитывать.

Ответ:

- **Проверять HTTP-код ответа и обрабатывать исключения**
- Игнорировать ошибки и продолжать
- Просто логировать ошибки без обработки
- Повторять запрос без проверки кода ответа

Шаг 05:

Миссия 5

 Сопоставьте понятия интеграции API с их описанием:

Ответ:

- REST API
 - SOAP API
 - JSON
 - XML
1. Протокол обмена сообщениями через HTTP с лёгкой структурой
 2. Протокол для обмена сообщениями с жёсткой структурой и WSDL
 3. Формат данных для REST с ключ-значение
 4. Формат данных для SOAP с тегами и схемой

Подсказка: REST обычно с JSON, SOAP — с XML.

- **REST API** — Протокол обмена сообщениями через HTTP с лёгкой структурой
- **SOAP API** — Протокол для обмена сообщениями с жёсткой структурой и WSDL
- **JSON** — Формат данных для REST с ключ-значение

- **XML** — Формат данных для SOAP с тегами и схемой

Шаг 06:

Миссия 6

 Сопоставьте методы HTTP с их назначением при работе с REST API:

Ответ:

- GET
 - POST
 - PUT
 - DELETE
1. Получение данных
 2. Создание новой записи
 3. Обновление существующей записи
 4. Удаление записи

Подсказка: GET — чтение, POST — создание, PUT — обновление, DELETE — удаление.

- **GET** — Получение данных
- **POST** — Создание новой записи
- **PUT** — Обновление существующей записи
- **DELETE** — Удаление записи

Шаг 07:

Миссия 7

 Сопоставьте термины обработки API с их назначением:

Ответ:

- Кэширование
 - Аутентификация
 - Логирование
 - Ретрай
1. Хранение временных данных для ускорения повторных запросов
 2. Проверка прав доступа пользователя или приложения
 3. Фиксация всех запросов и ответов для анализа
 4. Повторный запрос при ошибке или тайм-ауте

Подсказка: Подумайте, что важно для безопасности, диагностики и надежности API.

- **Кэширование** — Хранение временных данных для ускорения повторных запросов
- **Аутентификация** — Проверка прав доступа пользователя или приложения
- **Логирование** — Фиксация всех запросов и ответов для анализа
- **Ретрай** — Повторный запрос при ошибке или тайм-ауте

Шаг 08:

Миссия 8

 Сопоставьте виды HTTP-ответов с их значением:

Ответ:

- 200 OK

- 201 Created
- 400 Bad Request
- 401 Unauthorized

1. Запрос успешно обработан
2. Ресурс успешно создан
3. Неправильный запрос или данные
4. Нет прав доступа

Подсказка: HTTP-коды помогают понять статус выполнения запроса.

- **200 OK** — Запрос успешно обработан
- **201 Created** — Ресурс успешно создан
- **400 Bad Request** — Неправильный запрос или данные
- **401 Unauthorized** — Нет прав доступа

Шаг 09:

Миссия 9

 Сопоставьте форматы данных с их применением:

Ответ:

- JSON
 - XML
 - YAML
 - CSV
1. REST API
 2. SOAP API
 3. Конфигурационные файлы
 4. Табличные данные

Подсказка: Формат подбирается под способ передачи и структуру данных.

- **JSON** — REST API
- **XML** — SOAP API
- **YAML** — Конфигурационные файлы
- **CSV** — Табличные данные

Шаг 10:

Миссия 10

 Сопоставьте методы авторизации с их описанием:

Ответ:

- API Key
 - OAuth 2.0
 - Basic Auth
 - JWT
1. Используется статический ключ для доступа
 2. Протокол с токенами для безопасного доступа
 3. Передача логина и пароля в заголовке
 4. Токен, который можно проверять без запроса к серверу

Подсказка: Разные методы обеспечивают разный уровень безопасности и удобства.

- **API Key** — Используется статический ключ для доступа
- **OAuth 2.0** — Протокол с токенами для безопасного доступа
- **Basic Auth** — Передача логина и пароля в заголовке
- **JWT** — Токен, который можно проверять без запроса к серверу

Шаг 11:

Миссия 11

Расположите шаги интеграции REST API с PostgreSQL в правильном порядке:

1. Создание таблиц в PostgreSQL
2. Отправка запроса к REST API
3. Обработка полученных данных
4. Вставка данных в базу
5. Обработка ошибок и логирование

Подсказка: Сначала создаем структуру базы, затем получаем и обрабатываем данные, и только потом вставляем и логируем.

Ответ:

- Создание таблиц в PostgreSQL
- Отправка запроса к REST API
- Обработка полученных данных
- Вставка данных в базу
- Обработка ошибок и логирование

Шаг 12:

Миссия 12

Расположите этапы работы SOAP API с PHP:

1. Создание SOAPClient
2. Вызов метода веб-сервиса
3. Обработка ответа
4. Обработка ошибок и исключений

Подсказка: Сначала клиент, потом вызов метода, затем обработка результата и ошибок.

Ответ:

- Создание SOAPClient
- Вызов метода веб-сервиса
- Обработка ответа
- Обработка ошибок и исключений

Шаг 13:

Миссия 13

Расположите шаги обработки JSON-ответа от REST API:

1. Получение JSON-строки
2. Декодирование в массив или объект
3. Проверка наличия ошибок
4. Использование данных в приложении

Подсказка: Сначала получаем строку, потом декодируем, проверяем ошибки и используем данные.

Ответ:

- Получение JSON-строки
- Декодирование в массив или объект
- Проверка наличия ошибок
- Использование данных в приложении

 Шаг 14:

Миссия 14

Расположите шаги интеграции SOAP API с базой PostgreSQL:

1. Создание таблиц
2. Создание SOAPClient
3. Вызов метода SOAP
4. Обработка ответа и вставка в базу
5. Логирование и обработка ошибок

Подсказка: Сначала таблицы, потом клиент, вызов метода, обработка ответа и логирование.

Ответ:

- Создание таблиц
- Создание SOAPClient
- Вызов метода SOAP
- Обработка ответа и вставка в базу
- Логирование и обработка ошибок

 Шаг 15:

Миссия 15

Расположите шаги настройки аутентификации для API:

1. Получение ключа или токена
2. Сохранение ключа безопасно
3. Добавление ключа в заголовок запроса
4. Проверка ответа на ошибки авторизации

Подсказка: Сначала ключ, потом безопасное хранение, затем добавление в запрос и проверка ответа.

Ответ:

- Получение ключа или токена
- Сохранение ключа безопасно
- Добавление ключа в заголовок запроса
- Проверка ответа на ошибки авторизации

 Шаг 16:

Миссия 16

Расположите шаги оптимизации работы с REST API:

1. Кэширование повторяющихся запросов
2. Пакетная обработка данных

3. Логирование ошибок и медленных запросов
4. Повторные попытки при тайм-аутах

Подсказка: Сначала ускоряем запросы кэшем, затем пакетная обработка, логирование и повторные попытки.

Ответ:

- Кэширование повторяющихся запросов
- Пакетная обработка данных
- Логирование ошибок и медленных запросов
- Повторные попытки при тайм-аутах

Шаг 17:

Миссия 17

Расположите шаги полной интеграции API с приложением:

1. Настройка аутентификации и ключей
2. Создание структуры базы данных
3. Получение и обработка данных с API
4. Сохранение данных в PostgreSQL
5. Логирование и мониторинг ошибок
6. Оптимизация производительности

Подсказка: Сначала безопасность, затем структура, получение и обработка данных, сохранение, логирование и оптимизация.

Ответ:

- Настройка аутентификации и ключей
- Создание структуры базы данных
- Получение и обработка данных с API
- Сохранение данных в PostgreSQL
- Логирование и мониторинг ошибок
- Оптимизация производительности

Урок 5 - Репликация и резервное копирование данных в PostgreSQL

Шаг 01:

Миссия 1

Что из перечисленного является преимуществом репликации PostgreSQL?

Подсказка: Репликация помогает распределять нагрузку и обеспечивать отказоустойчивость.

Ответ:

- **Повышение доступности и масштабируемости системы**
- Снижение размера базы данных
- Автоматическое исправление багов в приложении
- Ускорение компиляции PHP-кода

Шаг 02:

Миссия 2

Какой тип репликации PostgreSQL передает только изменения на уровне WAL?

Подсказка: Обратите внимание, что WAL — это журнал транзакций.

Ответ:

- **Логическая репликация**
- Физическая репликация
- Синхронная репликация
- Асинхронная репликация

Шаг 03:

Миссия 3

 Сопоставьте типы резервного копирования с их характеристиками:

Ответ:

- Полное резервное копирование
 - Дифференциальное резервное копирование
 - Инкрементное резервное копирование
 - Резервное копирование PITR
1. Сохраняет всю базу целиком
 2. Сохраняет изменения с последнего полного бэкапа
 3. Сохраняет изменения с момента последнего любого бэкапа
 4. Позволяет восстановить базу к определённой точке во времени

Подсказка: Подумайте о том, как часто меняются данные и как их можно восстановить.

- **Полное резервное копирование** — Сохраняет всю базу целиком
- **Дифференциальное резервное копирование** — Сохраняет изменения с последнего полного бэкапа
- **Инкрементное резервное копирование** — Сохраняет изменения с момента последнего любого бэкапа
- **Резервное копирование PITR** — Позволяет восстановить базу к определённой точке во времени

Шаг 04:

Миссия 4

 Сопоставьте параметры репликации с их назначением:

Ответ:

- primary_conninfo
 - wal_level
 - max_wal_senders
 - archive_mode
1. Информация о подключении слейва к мастеру
 2. Уровень логирования WAL для репликации
 3. Количество потоков для отправки WAL на слейвы
 4. Включение архивирования WAL

Подсказка: Параметры конфигурации определяют поведение репликации и сохранение данных.

- **primary_conninfo** — Информация о подключении слейва к мастеру
- **wal_level** — Уровень логирования WAL для репликации
- **max_wal_senders** — Количество потоков для отправки WAL на слейвы
- **archive_mode** — Включение архивирования WAL

Шаг 05:

Миссия 5

Расположите шаги настройки резервного копирования и репликации в правильном порядке:

1. Настройка архивации WAL
2. Конфигурация слейв-сервера
3. Запуск резервного копирования базы данных
4. Настройка параметров репликации (wal_level, max_wal_senders)
5. Проверка подключения слейва и тестовая репликация

Подсказка: Сначала настраиваем архивирование и репликацию, затем делаем бэкап и проверяем работу слейва.

Ответ:

- Настройка архивации WAL
- Конфигурация слейв-сервера
- Запуск резервного копирования базы данных
- Настройка параметров репликации (wal_level, max_wal_senders)
- Проверка подключения слейва и тестовая репликация

Шаг 06:

Миссия 6

Какая команда используется для создания резервной копии базы данных PostgreSQL с помощью pg_dump?

Подсказка: Подумайте о стандартном инструменте командной строки для бэкапа.

Ответ:

- **pg_dump mydb > backup.sql**
- pg_restore mydb > backup.sql
- psql mydb > backup.sql
- createdb mydb > backup.sql

Шаг 07:

Миссия 7

Что необходимо для настройки асинхронной репликации PostgreSQL?

Подсказка: Смотрите на параметры подключения и количество потоков WAL.

Ответ:

- **primary_conninfo на слейве**
- **max_wal_senders на мастере**
- wal_compression на слейве
- archive_timeout на слейве

Шаг 08:

Миссия 8

 Сопоставьте режимы репликации с их особенностями:

Ответ:

- Синхронная репликация
- Асинхронная репликация

- Логическая репликация
- Физическая репликация

1. Транзакция считается завершенной только после подтверждения слейва
2. Мастер не ждет подтверждения от слейва
3. Реплицирует таблицы и данные на уровне SQL
4. Копирует данные на уровне файлов и WAL

Подсказка: Подумайте, чем отличаются уровни репликации и задержки.

- **Синхронная репликация** — Транзакция считается завершенной только после подтверждения слейва
- **Асинхронная репликация** — Мастер не ждет подтверждения от слейва
- **Логическая репликация** — Реплицирует таблицы и данные на уровне SQL
- **Физическая репликация** — Копирует данные на уровне файлов и WAL

🔔 Шаг 09:

Миссия 9

📖 Сопоставьте типы бэкапов PostgreSQL с примерами использования:

Ответ:

- Полное резервное копирование
 - Дифференциальное резервное копирование
 - Инкрементное резервное копирование
 - PITR
1. При первом запуске бэкапа или для архивации всей базы
 2. Для сохранения изменений с последнего полного бэкапа
 3. Для сохранения всех изменений с момента последнего бэкапа
 4. Чтобы восстановить базу к конкретной дате и времени после сбоя

Подсказка: Подумайте, какой бэкап экономит место, а какой обеспечивает точное восстановление.

- **Полное резервное копирование** — При первом запуске бэкапа или для архивации всей базы
- **Дифференциальное резервное копирование** — Для сохранения изменений с последнего полного бэкапа
- **Инкрементное резервное копирование** — Для сохранения всех изменений с момента последнего бэкапа
- **PITR** — Чтобы восстановить базу к конкретной дате и времени после сбоя

🔔 Шаг 10:

Миссия 10

Что из перечисленного позволяет выполнить восстановление базы до определённого момента времени?

Подсказка: Это связано с журналами WAL и архивированием.

Ответ:

- **PITR (Point-in-Time Recovery)**
- Полное резервное копирование
- Дифференциальный бэкап
- pg_dumpall

Шаг 11:

Миссия 11

Какая команда проверяет статус репликации на слейве PostgreSQL?

Подсказка: Думайте о системных представлениях, где отображается состояние подключений к мастеру.

Ответ:

- **SELECT * FROM pg_stat_wal_receiver;**
- SELECT * FROM pg_stat_activity;
- SHOW replication_status;
- SELECT * FROM pg_stat_database;

Шаг 12:

Миссия 12

 Сопоставьте команды PostgreSQL с их назначением в контексте репликации:

Ответ:

- pg_basebackup
- pg_ctl start/stop
- pg_receivewal
- pg_restore

1. Создание физической копии базы для слейва
2. Запуск или остановка сервера
3. Получение WAL-файлов из архива
4. Восстановление данных из дампа

Подсказка: Смотрите, какие команды работают с физическими файлами, а какие — с дампами SQL.

- **pg_basebackup** — Создание физической копии базы для слейва
- **pg_ctl start/stop** — Запуск или остановка сервера
- **pg_receivewal** — Получение WAL-файлов из архива
- **pg_restore** — Восстановление данных из дампа

Шаг 13:

Миссия 13

 Сопоставьте сценарии с типами восстановления в PostgreSQL:

Ответ:

- Восстановление после сбоя оборудования
- Восстановление удалённой таблицы
- Восстановление к конкретной точке времени
- Восстановление из физического бэкапа

1. Используем полный или инкрементный бэкап + WAL
2. Используем дамп таблицы через pg_restore
3. Используем PITR с WAL
4. Восстанавливаем весь сервер с помощью pg_basebackup

Подсказка: Подумайте, когда нужен полный бэкап, а когда достаточно отдельных таблиц.

- **Восстановление после сбоя оборудования** — Используем полный или инкрементный бэкап + WAL
- **Восстановление удалённой таблицы** — Используем дампы таблицы через pg_restore
- **Восстановление к конкретной точке времени** — Используем PITR с WAL
- **Восстановление из физического бэкапа** — Восстанавливаем весь сервер с помощью pg_basebackup

🔮 Шаг 14:

Миссия 14

Какая стратегия репликации минимизирует потерю данных при сбое мастера?

Подсказка: Подумайте о транзакциях и подтверждениях от слейвов.

Ответ:

- **Синхронная репликация**
- Асинхронная репликация
- Логическая репликация
- Физическая репликация без WAL

🔮 Шаг 15:

Миссия 15

Расположите шаги настройки асинхронной репликации PostgreSQL в правильном порядке:

1. Настроить max_wal_senders на мастере
2. Создать слейв с помощью pg_basebackup
3. Настроить primary_conninfo на слейве
4. Запустить сервер слейва и проверить соединение

Подсказка: Сначала нужно подготовить мастер, потом слейв, и только после запуска проверять работу.

Ответ:

- Настроить max_wal_senders на мастере
- Создать слейв с помощью pg_basebackup
- Настроить primary_conninfo на слейве
- Запустить сервер слейва и проверить соединение

🔮 Шаг 16:

Миссия 16

📄 Сопоставьте команды PostgreSQL с их назначением при резервном копировании:

Ответ:

- pg_dump
 - pg_dumpall
 - pg_basebackup
 - pg_restore
1. Создание дампа конкретной базы
 2. Создание дампа всех баз сервера
 3. Создание физической копии для репликации
 4. Восстановление данных из дампа

Подсказка: Подумайте, что работает на уровне SQL, а что на уровне файлов сервера.

- **pg_dump** — Создание дампа конкретной базы
- **pg_dumpall** — Создание дампа всех баз сервера
- **pg_basebackup** — Создание физической копии для репликации
- **pg_restore** — Восстановление данных из дампа

🔮 Шаг 17:

Миссия 17

Что поможет уменьшить нагрузку на мастер при больших объёмах репликации?

Подсказка: Смотрите на настройку слейвов и WAL.

Ответ:

- **Использовать слейвы для чтения**
- **Включить асинхронную репликацию**
- Уменьшить WAL-сегменты до минимума
- Выключить archive_mode

🔮 Шаг 18:

Миссия 18

📄 Сопоставьте сценарии с типами восстановления в PostgreSQL:

Ответ:

- Восстановление после сбоя оборудования
 - Восстановление удалённой таблицы
 - Восстановление к конкретной точке времени
 - Восстановление из физического бэкапа
1. Используем полный или инкрементный бэкап + WAL
 2. Используем дамп таблицы через pg_restore
 3. Используем PITR с WAL
 4. Восстанавливаем весь сервер с помощью pg_basebackup

Подсказка: Подумайте, когда нужен полный бэкап, а когда достаточно отдельных таблиц.

- **Восстановление после сбоя оборудования** — Используем полный или инкрементный бэкап + WAL
- **Восстановление удалённой таблицы** — Используем дамп таблицы через pg_restore
- **Восстановление к конкретной точке времени** — Используем PITR с WAL
- **Восстановление из физического бэкапа** — Восстанавливаем весь сервер с помощью pg_basebackup

🔮 Шаг 19:

Миссия 19

Какая стратегия репликации минимизирует потерю данных при сбое мастера?

Подсказка: Подумайте о транзакциях и подтверждениях от слейвов.

Ответ:

- **Синхронная репликация**
- Асинхронная репликация
- Логическая репликация

- Физическая репликация без WAL

Модуль 6 - Безопасность и управление данными

Урок 1 - Роли и права доступа в PostgreSQL

Шаг 01:

Миссия 1

Алекс работает над проектом, где каждому пользователю нужно разное право доступа. Какой тип роли в PostgreSQL лучше всего использовать для простого управления доступом?

Подсказка: Подумайте, как объединить пользователей с похожими правами.

Ответ:

- **Групповая роль (Role)**
- Функция (Function)
- Триггер (Trigger)
- Схема (Schema)

Шаг 02:

Миссия 2

Мария хочет, чтобы новые пользователи автоматически получали минимальный набор прав на чтение таблиц. Что лучше использовать?

Подсказка: Думайте о настройке шаблонов при создании пользователей.

Ответ:

- **Роль по умолчанию (Default Role)**
- Секретный ключ
- Индексация таблицы
- Встроенная функция доступа

Шаг 03:

Миссия 3

Иван хочет запретить конкретному пользователю удалять записи из таблицы. Какой инструмент PostgreSQL следует использовать?

Подсказка: Подумайте о правах на уровне объектов.

Ответ:

- **GRANT и REVOKE на таблицу**
- Создание триггера
- ALTER DATABASE
- CREATE INDEX

Шаг 04:

Миссия 4

 Задача: Сопоставьте тип роли PostgreSQL с её назначением

Роли:

Ответ:

- Superuser

- Login role
- Group role
- Readonly role

Назначение:

1. Может выполнять любые действия в базе данных
2. Используется для подключения пользователя к базе
3. Объединяет несколько пользователей для управления правами
4. Разрешает только чтение данных

Подсказка: Подумайте о том, какие права обычно нужны для суперпользователя и для обычного пользователя.

- **Superuser** — Может выполнять любые действия в базе данных
- **Login role** — Используется для подключения пользователя к базе
- **Group role** — Объединяет несколько пользователей для управления правами
- **Readonly role** — Разрешает только чтение данных

Шаг 05:

Миссия 5

 Задача: Сопоставьте команду PostgreSQL с её действием

Команды:

Ответ:

- GRANT
- REVOKE
- ALTER ROLE
- CREATE ROLE

Действие:

1. Выдаёт права пользователю или роли
2. Удаляет права пользователя
3. Изменяет свойства существующей роли
4. Создаёт новую роль

Подсказка: Сравните команды с их логическим действием — кто что делает с ролями?

- **GRANT** — Выдаёт права пользователю или роли
- **REVOKE** — Удаляет права пользователя
- **ALTER ROLE** — Изменяет свойства существующей роли
- **CREATE ROLE** — Создаёт новую роль

Шаг 06:

Миссия 6

 Задача: Сопоставьте объект PostgreSQL с действием управления доступом

Объекты:

Ответ:

- Таблица
- Схема
- Функция

- База данных

Действие:

1. Можно назначить права на чтение/запись конкретных данных
2. Можно ограничить создание объектов внутри схемы
3. Можно ограничить выполнение определённой процедуры
4. Можно ограничить подключение и создание всех объектов в базе

Подсказка: Думайте о масштабах: от конкретного объекта до всей базы.

- **Таблица** — Можно назначить права на чтение/запись конкретных данных
- **Схема** — Можно ограничить создание объектов внутри схемы
- **Функция** — Можно ограничить выполнение определённой процедуры
- **База данных** — Можно ограничить подключение и создание всех объектов в базе

Шаг 07:

Миссия 7

 Задача: Сопоставьте тип пользователя с его безопасными действиями

Пользователи:

Ответ:

- Администратор
- Разработчик
- Аналитик
- Гость

Действия:

1. Может изменять структуры таблиц и роли
2. Может писать и изменять код, но не изменяет роли
3. Может только читать данные для отчетов
4. Имеет минимальные права для просмотра

Подсказка: Подумайте о принципе «минимальных привилегий» для каждого типа.

- **Администратор** — Может изменять структуры таблиц и роли
- **Разработчик** — Может писать и изменять код, но не изменяет роли
- **Аналитик** — Может только читать данные для отчетов
- **Гость** — Имеет минимальные права для просмотра

Шаг 08:

Миссия 8

Расположите действия по созданию роли и выдаче прав в правильном порядке:

1. Создание новой роли
2. Подключение пользователя к роли
3. Выдача прав на объекты
4. Проверка доступа

Подсказка: Думайте логически: сначала создаём роль, потом выдаём права и проверяем их.

Ответ:

- Создание новой роли

- Подключение пользователя к роли
- Выдача прав на объекты
- Проверка доступа

Шаг 09:

Миссия 9

Расположите шаги по ограничению прав пользователя на таблицу:

1. Определение необходимых ограничений
2. Создание роли с ограниченными правами
3. Назначение роли пользователю
4. Проверка доступа через SELECT/INSERT

Подсказка: Сначала нужно понять, что ограничивать, потом создать роль и проверить.

Ответ:

- Определение необходимых ограничений
- Создание роли с ограниченными правами
- Назначение роли пользователю
- Проверка доступа через SELECT/INSERT

Шаг 10:

Миссия 10

Расположите действия администратора для безопасного управления базой:

1. Проверка текущих ролей и привилегий
2. Ревизия прав пользователей
3. Удаление ненужных ролей
4. Назначение обновленных прав

Подсказка: Сначала анализируем, потом удаляем лишние права и обновляем их.

Ответ:

- Проверка текущих ролей и привилегий
- Ревизия прав пользователей
- Удаление ненужных ролей
- Назначение обновленных прав

Шаг 11:

Миссия 11

Светлана разрабатывает систему, где роль аналитика должна иметь доступ только к определённым таблицам. Какой способ наиболее эффективен?

Подсказка: Рассмотрите права на уровне отдельных таблиц, а не всей базы.

Ответ:

- **GRANT SELECT на конкретные таблицы**
- GRANT ALL PRIVILEGES на базу
- CREATE FUNCTION для всех операций
- ALTER DATABASE для ограничения доступа

Шаг 12:

Миссия 12

Дмитрий хочет создать роль разработчика, которая может создавать функции и таблицы, но не управлять пользователями. Какой подход верный?

Подсказка: Обратите внимание на гранулярные права, а не на SUPERUSER.

Ответ:

- Создать роль с **CREATEDB** и **CREATEROLE**, но без **SUPERUSER**
- Назначить **SUPERUSER**
- Создать триггер для всех операций
- Дать права только на **SELECT**

Шаг 13:

Миссия 13

Елена хочет, чтобы новая роль автоматически наследовала права другой роли. Как это сделать?

Подсказка: Подумайте про механизм наследования ролей.

Ответ:

- Использовать **INHERIT** при создании роли
- Создать триггер на таблицу
- **ALTER DATABASE OWNER**
- Использовать **REVOKE ALL**

Шаг 14:

Миссия 14

 Задача: Сопоставьте команды PostgreSQL с их результатом

Команды:

Ответ:

- **GRANT ALL PRIVILEGES ON TABLE**
- **REVOKE INSERT ON TABLE**
- **ALTER ROLE INHERIT TRUE**
- **CREATE ROLE**

Результат:

1. Даёт все права на таблицу
2. Удаляет право вставки данных
3. Позволяет новой роли наследовать права другой роли
4. Создаёт новую роль

Подсказка: Смотрите, что делает каждая команда: выдаёт права, убирает права, наследует или создаёт роль.

- **GRANT ALL PRIVILEGES ON TABLE** — Даёт все права на таблицу
- **REVOKE INSERT ON TABLE** — Удаляет право вставки данных
- **ALTER ROLE INHERIT TRUE** — Позволяет новой роли наследовать права другой роли
- **CREATE ROLE** — Создаёт новую роль

Шаг 15:

Миссия 15

Расположите действия администратора для безопасного предоставления прав разработчику:

1. Создание роли разработчика
2. Выдача прав на создание таблиц и функций
3. Назначение роли конкретному пользователю
4. Проверка, что права работают корректно

Подсказка: Сначала создаём роль, потом выдаём права, назначаем пользователю и тестируем.

Ответ:

- Создание роли разработчика
- Выдача прав на создание таблиц и функций
- Назначение роли конкретному пользователю
- Проверка, что права работают корректно

Шаг 16:

Миссия 16

Расположите шаги по созданию безопасного окружения для аналитиков:

1. Создание роли аналитика
2. Выдача права только на SELECT определённых таблиц
3. Назначение роли конкретным аналитикам
4. Проверка доступа и тестирование запросов

Подсказка: Думайте о принципе минимальных привилегий: сначала создаём роль, затем права, потом назначаем и проверяем.

Ответ:

- Создание роли аналитика
- Выдача права только на SELECT определённых таблиц
- Назначение роли конкретным аналитикам
- Проверка доступа и тестирование запросов

Урок 2 - Шифрование данных и защита паролей

Шаг 01:

Миссия 1

Алекс хочет хранить пароли пользователей безопасно. Какой метод наиболее безопасен?

Подсказка: Подумайте о том, что нельзя хранить пароли в открытом виде.

Ответ:

- Хеширование с солью (например, bcrypt)
- Просто зашифровать с симметричным ключом
- Хранить в базе в открытом виде
- Закодировать в Base64

Шаг 02:

Миссия 2

Мария разрабатывает форму регистрации. Как лучше всего защитить передачу паролей от клиента к серверу?

Подсказка: Обратите внимание на шифрование соединения.

Ответ:

- **Использовать HTTPS с TLS**
- Отправлять через обычный HTTP
- Отправлять пароль в Base64
- Использовать GET-запрос

Шаг 03:

Миссия 3

Иван хочет хранить секретные токены в базе. Что следует использовать для их безопасного хранения?

Подсказка: Думайте о двустороннем шифровании, а не просто о хешировании.

Ответ:

- **Симметричное шифрование (AES) с безопасным ключом**
- Хранить в открытом виде
- Просто закодировать в hex
- Использовать ROT13

Шаг 04:

Миссия 4

 Задача: Сопоставьте метод защиты данных с его применением

Методы:

Ответ:

- Хеширование с солью
- Симметричное шифрование
- Асимметричное шифрование
- TLS/HTTPS

Применение:

1. Безопасное хранение паролей
2. Хранение токенов и секретных данных
3. Обмен зашифрованными сообщениями с открытым ключом
4. Безопасная передача данных по сети

Подсказка: Сравните, где нужен однонаправленный хеш, а где нужен ключ для расшифровки.

- **Хеширование с солью** — Безопасное хранение паролей
- **Симметричное шифрование** — Хранение токенов и секретных данных
- **Асимметричное шифрование** — Обмен зашифрованными сообщениями с открытым ключом
- **TLS/HTTPS** — Безопасная передача данных по сети

Шаг 05:

Миссия 5

 Сопоставьте алгоритм хеширования с его характеристикой:

Алгоритмы:

Ответ:

- MD5
- SHA-256
- Bcrypt
- Argon2

Характеристика:

1. Устаревший, небезопасный для паролей
2. Хеш средней криптографической стойкости
3. Безопасно для паролей с солью и замедлением
4. Современный, устойчивый к атакам на GPU

Подсказка: Обратите внимание на скорость и устойчивость к подбору паролей.

- **MD5** — Устаревший, небезопасный для паролей
- **SHA-256** — Хеш средней криптографической стойкости
- **Bcrypt** — Безопасно для паролей с солью и замедлением
- **Argon2** — Современный, устойчивый к атакам на GPU

Шаг 06:

Миссия 6

 Сопоставьте термин с описанием:

Термины:

Ответ:

- Соль
- Pepper
- Ключ шифрования
- IV (инициализационный вектор)

Описание:

1. Случайная строка, добавляемая к паролю для уникальности хеша
2. Секретная константа, добавляемая к паролю для усиления безопасности
3. Секрет для симметричного шифрования/расшифровки
4. Случайное значение для инициализации шифрования блоков

Подсказка: Сравните роли каждой добавочной переменной при защите данных.

- **Соль** — Случайная строка, добавляемая к паролю для уникальности хеша
- **Pepper** — Секретная константа, добавляемая к паролю для усиления безопасности
- **Ключ шифрования** — Секрет для симметричного шифрования/расшифровки
- **IV (инициализационный вектор)** — Случайное значение для инициализации шифрования блоков

Шаг 07:

Миссия 7

 Сопоставьте атаку с уязвимостью:

Атаки:

Ответ:

- Brute-force
- Rainbow table
- Man-in-the-middle
- SQL injection

Уязвимость:

1. Подбор пароля перебором всех вариантов
2. Использование предварительно вычисленных хешей для быстрого подбора
3. Перехват данных при передаче
4. Вставка вредоносного кода в запрос

Подсказка: Смотрите на то, где атакующий получает данные и как он их использует.

- **Brute-force** — Подбор пароля перебором всех вариантов
- **Rainbow table** — Использование предварительно вычисленных хешей для быстрого подбора
- **Man-in-the-middle** — Перехват данных при передаче
- **SQL injection** — Вставка вредоносного кода в запрос

 Шаг 08:

Миссия 8

Расположите шаги безопасного хранения пароля:

1. Сгенерировать соль
2. Объединить пароль и соль
3. Хешировать комбинацию с безопасным алгоритмом
4. Сохранить хеш и соль в базе

Подсказка: Сначала нужно подготовить уникальность, потом применить хеш и сохранить.

Ответ:

- Сгенерировать соль
- Объединить пароль и соль
- Хешировать комбинацию с безопасным алгоритмом
- Сохранить хеш и соль в базе

 Шаг 09:

Миссия 9

Расположите шаги по безопасной передаче токена между клиентом и сервером:

1. Сервер генерирует токен
2. Сервер шифрует токен с ключом
3. Отправка токена клиенту через HTTPS
4. Клиент расшифровывает токен и сохраняет

Подсказка: Думайте о порядке генерации, шифрования, передачи и расшифровки.

Ответ:

- Сервер генерирует токен
- Сервер шифрует токен с ключом
- Отправка токена клиенту через HTTPS

- Клиент расшифровывает токен и сохраняет

Шаг 10:

Миссия 10

Расположите шаги безопасного восстановления пароля пользователя:

1. Пользователь запрашивает восстановление
2. Генерация одноразового токена на сервере
3. Отправка токена на email пользователя
4. Проверка токена и установка нового пароля

Подсказка: Думайте о том, что токен должен быть одноразовым и проверяться сервером.

Ответ:

- Пользователь запрашивает восстановление
- Генерация одноразового токена на сервере
- Отправка токена на email пользователя
- Проверка токена и установка нового пароля

Шаг 11:

Миссия 11

Расположите шаги безопасного хранения секретных ключей приложения:

1. Создать хранилище ключей с ограниченным доступом
2. Сгенерировать уникальный ключ для каждого сервиса
3. Шифровать ключи перед сохранением
4. Периодически обновлять и проверять доступ

Подсказка: Думайте о минимизации риска утечки и регулярном обновлении.

Ответ:

- Создать хранилище ключей с ограниченным доступом
- Сгенерировать уникальный ключ для каждого сервиса
- Шифровать ключи перед сохранением
- Периодически обновлять и проверять доступ

Шаг 12:

Миссия 12

 Сопоставьте тип шифрования с примером применения:

Ответ:

- Симметричное шифрование
 - Асимметричное шифрование
 - Хеширование
 - TLS
1. Шифрование сообщений между сервером и клиентом с одним ключом
 2. Обмен секретами с открытым и закрытым ключом
 3. Хранение паролей в базе
 4. Обеспечение безопасного соединения при передаче данных

Подсказка: Обратите внимание на направление и возможность расшифровки данных.

- **Симметричное шифрование** — Шифрование сообщений между сервером и клиентом с одним ключом
- **Асимметричное шифрование** — Обмен секретами с открытым и закрытым ключом
- **Хеширование** — Хранение паролей в базе
- **TLS** — Обеспечение безопасного соединения при передаче данных

🔮 Шаг 13:

Миссия 13

📄 Сопоставьте функцию PostgreSQL с её ролью в защите данных:

Ответ:

- `pgcrypto::crypt()`
 - `pgcrypto::gen_salt()`
 - `pgcrypto::digest()`
 - SSL-соединение PostgreSQL
1. Хеширование пароля с солью
 2. Генерация уникальной соли
 3. Создание криптографического хеша данных
 4. Безопасная передача данных между клиентом и сервером

Подсказка: Подумайте, какие функции PostgreSQL используются для хеширования, а какие — для безопасного канала передачи.

- **`pgcrypto::crypt()`** — Хеширование пароля с солью
- **`pgcrypto::gen_salt()`** — Генерация уникальной соли
- **`pgcrypto::digest()`** — Создание криптографического хеша данных
- **SSL-соединение PostgreSQL** — Безопасная передача данных между клиентом и сервером

🔮 Шаг 14:

Миссия 14

Расположите шаги безопасной авторизации с использованием токенов:

1. Пользователь вводит логин и пароль
2. Сервер проверяет хеш пароля
3. Сервер генерирует JWT-токен
4. Сервер отправляет токен клиенту через HTTPS
5. Клиент сохраняет токен и использует его для запросов

Подсказка: Сначала проверка подлинности, потом генерация токена и безопасная передача.

Ответ:

- Пользователь вводит логин и пароль
- Сервер проверяет хеш пароля
- Сервер генерирует JWT-токен
- Сервер отправляет токен клиенту через HTTPS
- Клиент сохраняет токен и использует его для запросов

🔮 Шаг 15:

Миссия 15

Расположите шаги шифрования конфиденциального файла в базе:

1. Генерация симметричного ключа
2. Шифрование файла с этим ключом
3. Сохранение зашифрованного файла в базе
4. Сохранение ключа в защищённом хранилище

Подсказка: Думайте о порядке: сначала ключ, потом шифрование, потом хранение.

Ответ:

- Генерация симметричного ключа
- Шифрование файла с этим ключом
- Сохранение зашифрованного файла в базе
- Сохранение ключа в защищённом хранилище

Шаг 16:

Миссия 16

Расположите шаги обновления пароля с защитой от повторного использования:

1. Пользователь вводит новый пароль
2. Сервер проверяет, что пароль не использовался ранее
3. Генерация соли и хеширование нового пароля
4. Сохранение нового хеша и соли в базе

Подсказка: Сначала проверка старых паролей, потом генерация и сохранение нового хеша.

Ответ:

- Пользователь вводит новый пароль
- Сервер проверяет, что пароль не использовался ранее
- Генерация соли и хеширование нового пароля
- Сохранение нового хеша и соли в базе

Шаг 17:

Миссия 17

Расположите этапы безопасного обмена данными между микросервисами:

1. Каждый сервис генерирует свой ключ шифрования
2. Шифрование сообщений перед отправкой
3. Отправка сообщений через защищённый канал (TLS)
4. Расшифровка сообщений на стороне получателя
5. Проверка целостности сообщений

Подсказка: Думайте о порядке: генерация ключа, шифрование, передача, расшифровка и проверка.

Ответ:

- Каждый сервис генерирует свой ключ шифрования
- Шифрование сообщений перед отправкой
- Отправка сообщений через защищённый канал (TLS)
- Расшифровка сообщений на стороне получателя
- Проверка целостности сообщений

Урок 3 - Как работать с SSL-соединениями для защиты данных

Шаг 01:

Миссия 1

Анна настраивает соединение с базой PostgreSQL. Что необходимо включить для защиты данных при передаче?

Подсказка: Думайте о шифровании канала передачи.

Ответ:

- **Использовать SSL-соединение**
- Просто использовать TCP без шифрования
- Включить логирование паролей в открытом виде
- Отправлять данные через HTTP

Шаг 02:

Миссия 2

Пётр хочет удостовериться, что серверная база принимает только безопасные соединения. Какую опцию PostgreSQL следует включить?

Подсказка: Думайте о строгом требовании SSL для всех клиентов.

Ответ:

- **ssl = on и sslmode=require**
- ssl = off
- ssl = optional
- sslmode=disable

Шаг 03:

Миссия 3

Маша хочет удостовериться, что данные между клиентом и сервером шифруются. Какой сертификат следует использовать?

Подсказка: Сертификат обеспечивает доверие и шифрование.

Ответ:

- **Сертификат X.509 от доверенного СА**
- Самоподписанный сертификат без проверки
- Пароль в открытом виде
- Файл hosts.txt

Шаг 04:

Миссия 4

 Сопоставьте термин SSL с его назначением:

Ответ:

- SSL-сертификат
- Ключ сервера
- SSL-шифрование
- SSL-верификация клиента

1. Документ, удостоверяющий личность сервера

2. Секретная часть для шифрования соединения
3. Обеспечение конфиденциальности данных при передаче
4. Проверка подлинности клиента

Подсказка: Смотрите на роли документа, ключа и проверок при SSL.

- **SSL-сертификат** — Документ, удостоверяющий личность сервера
- **Ключ сервера** — Секретная часть для шифрования соединения
- **SSL-шифрование** — Обеспечение конфиденциальности данных при передаче
- **SSL-верификация клиента** — Проверка подлинности клиента

Шаг 05:

Миссия 5

 Сопоставьте параметры подключения PostgreSQL с их эффектом для SSL:

Ответ:

- `sslmode=require`
 - `sslmode=verify-ca`
 - `sslmode=verify-full`
 - `sslmode=disable`
1. Соединение обязательно через SSL, но без проверки сертификата
 2. Соединение через SSL с проверкой сертификата CA
 3. Соединение через SSL с проверкой полного соответствия имени хоста и сертификата
 4. SSL соединение отключено

Подсказка: Обратите внимание, где проверяется доверие к серверу.

- **`sslmode=require`** — Соединение обязательно через SSL, но без проверки сертификата
- **`sslmode=verify-ca`** — Соединение через SSL с проверкой сертификата CA
- **`sslmode=verify-full`** — Соединение через SSL с проверкой полного соответствия имени хоста и сертификата
- **`sslmode=disable`** — SSL соединение отключено

Шаг 06:

Миссия 6

 Сопоставьте этапы настройки SSL-соединения в PostgreSQL:

Ответ:

- Создание сертификата сервера
 - Генерация приватного ключа
 - Настройка `postgresql.conf`
 - Перезапуск сервера
1. Создать ключ для шифрования
 2. Создать документ удостоверения сервера
 3. Включить `ssl` и указать пути к сертификатам
 4. Применить изменения

Подсказка: Сначала генерируем ключ и сертификат, потом настраиваем сервер и перезапускаем.

- **Генерация приватного ключа** — Создать ключ для шифрования

- **Создание сертификата сервера** — Создать документ удостоверения сервера
- **Настройка postgresql.conf** — Включить ssl и указать пути к сертификатам
- **Перезапуск сервера** — Применить изменения

💡 Шаг 07:

Миссия 7

📖 Сопоставьте тип атаки с риском при отсутствии SSL:

Ответ:

- Man-in-the-middle
 - Прослушка сети
 - Подмена данных
 - SQL-инъекция
1. Атакующий перехватывает соединение и данные
 2. Чтение передаваемой информации в открытом виде
 3. Изменение сообщений в пути
 4. Внедрение кода в запросы к базе

Подсказка: SSL защищает от перехвата и подмены, но не исправляет SQL-инъекции.

- **Man-in-the-middle** — Атакующий перехватывает соединение и данные
- **Прослушка сети** — Чтение передаваемой информации в открытом виде
- **Подмена данных** — Изменение сообщений в пути
- **SQL-инъекция** — Внедрение кода в запросы к базе

💡 Шаг 08:

Миссия 8

Расположите шаги установки SSL-соединения с PostgreSQL клиентом:

1. Настройка sslmode на клиенте
2. Импорт сертификата CA
3. Указание пути к клиентскому сертификату и ключу
4. Подключение к серверу

Подсказка: Сначала проверяем доверие, затем настраиваем сертификаты и подключаемся.

Ответ:

- Настройка sslmode на клиенте
- Импорт сертификата CA
- Указание пути к клиентскому сертификату и ключу
- Подключение к серверу

💡 Шаг 09:

Миссия 9

Расположите шаги проверки SSL-соединения клиента с сервером PostgreSQL:

1. Установить соединение через psql с опцией sslmode
2. Проверить, что соединение использует SSL
3. Проверить сертификат сервера
4. Проверить правильность имени хоста

Подсказка: Сначала подключаемся, потом проверяем используемые сертификаты и имя хоста.

Ответ:

- Установить соединение через psql с опцией sslmode
- Проверить, что соединение использует SSL
- Проверить сертификат сервера
- Проверить правильность имени хоста

Шаг 10:

Миссия 10

Расположите действия по безопасной передаче данных через SSL:

1. Шифрование данных на клиенте
2. Передача через TLS канал
3. Расшифровка данных на сервере
4. Проверка целостности и подлинности данных

Подсказка: Сначала шифруем, потом передаем и проверяем целостность.

Ответ:

- Шифрование данных на клиенте
- Передача через TLS канал
- Расшифровка данных на сервере
- Проверка целостности и подлинности данных

Шаг 11:

Миссия 11

Расположите шаги обновления SSL-сертификатов на сервере PostgreSQL:

1. Сгенерировать новый сертификат и ключ
2. Заменить старый сертификат и ключ на сервере
3. Перезапустить PostgreSQL
4. Проверить соединение с клиентами

Подсказка: Новый сертификат сначала создаем, затем меняем, перезапускаем сервер и тестируем подключение.

Ответ:

- Сгенерировать новый сертификат и ключ
- Заменить старый сертификат и ключ на сервере
- Перезапустить PostgreSQL
- Проверить соединение с клиентами

Шаг 12:

Миссия 12

Сергей хочет убедиться, что клиент не подключается к фальшивому серверу. Какой режим sslmode стоит выбрать?

Подсказка: Нужно полное доверие к сертификату и проверка имени хоста.

Ответ:

- **sslmode=verify-full**
- sslmode=require

- sslmode=disable
- sslmode=prefer

Шаг 13:

Миссия 13

Ирина тестирует соединение. Какие данные важно проверять при установке SSL?

Подсказка: Думайте о подлинности и целостности канала.

Ответ:

- **Проверка сертификата сервера**
- **Проверка имени хоста**
- Проверка пароля в открытом виде
- Проверка соединения без шифрования

Шаг 14:

Миссия 14

 Сопоставьте инструменты для проверки SSL с их функцией:

Ответ:

- OpenSSL
 - psql
 - pgAdmin
 - Wireshark
1. Тестирование сертификатов и шифрования
 2. Проверка SSL-соединения с сервером
 3. Мониторинг и настройка SSL в графическом интерфейсе
 4. Анализ пакетов сети для проверки шифрования

Подсказка: Различайте проверку сертификатов, интерфейс управления и анализ пакетов.

- **OpenSSL** — Тестирование сертификатов и шифрования
- **psql** — Проверка SSL-соединения с сервером
- **pgAdmin** — Мониторинг и настройка SSL в графическом интерфейсе
- **Wireshark** — Анализ пакетов сети для проверки шифрования

Шаг 15:

Миссия 15

Расположите шаги конфигурации клиентского SSL для PostgreSQL:

1. Установка клиентского сертификата
2. Импорт корневого CA
3. Настройка sslmode и путей к файлам
4. Подключение к серверу
5. Проверка успешного шифрованного соединения

Подсказка: Сначала доверие (CA), затем сертификаты, настройки и подключение.

Ответ:

- Установка клиентского сертификата
- Импорт корневого CA

- Настройка sslmode и путей к файлам
- Подключение к серверу
- Проверка успешного зашифрованного соединения

Шаг 16:

Миссия 16

Расположите шаги тестирования SSL-соединения после изменения сертификата:

1. Перезапустить сервер PostgreSQL
2. Перезапустить клиентское приложение
3. Подключиться к серверу через SSL
4. Проверить корректность сертификата и имя хоста
5. Убедиться, что данные передаются зашифрованными

Подсказка: Сначала перезапуск, затем подключение и проверка шифрования.

Ответ:

- Перезапустить сервер PostgreSQL
- Перезапустить клиентское приложение
- Подключиться к серверу через SSL
- Проверить корректность сертификата и имя хоста
- Убедиться, что данные передаются зашифрованными

Урок 4 - Аудит данных и мониторинг безопасности

Шаг 01:

Миссия 1

Анна хочет включить аудит действий пользователей в PostgreSQL. Какой параметр нужно активировать?

Подсказка: Подумайте, что отвечает за логирование действий в базе.

Ответ:

- **pgaudit.enabled**
- logging_collector
- sslmode
- autovacuum

Шаг 02:

Миссия 2

Какая цель аудита данных в крупных системах?

Подсказка: Думайте о безопасности и контроле действий пользователей.

Ответ:

- **Отслеживание изменений данных**
- **Контроль действий пользователей**
- Ускорение выполнения запросов
- Автоматическая оптимизация индексов

Шаг 03:

Миссия 3

 Сопоставьте тип события аудита с его описанием:

Ответ:

- DDL-событие
 - DML-событие
 - Аутентификация
 - Ошибки доступа
1. Изменение структуры таблиц или базы
 2. Вставка, обновление, удаление данных
 3. Вход пользователя в систему
 4. Попытка выполнить запрещённое действие

Подсказка: Разделите события по типу действий: структура, данные, вход, ошибки.

- **DDL-событие** — Изменение структуры таблиц или базы
- **DML-событие** — Вставка, обновление, удаление данных
- **Аутентификация** — Вход пользователя в систему
- **Ошибки доступа** — Попытка выполнить запрещённое действие

 **Шаг 04:**

Миссия 4

Какой инструмент помогает анализировать журналы PostgreSQL для безопасности?

Подсказка: Подумайте о программах для разбора логов.

Ответ:

- **pgBadger**
- psql
- pgAdmin
- pg_dump

 **Шаг 05:**

Миссия 5

Алексей хочет мониторить активность пользователей в реальном времени. Что лучше использовать?

Подсказка: Ищите инструмент для потокового анализа событий.

Ответ:

- **pgAudit**
- **log_statement = 'all'**
- autovacuum
- vacuumdb

 **Шаг 06:**

Миссия 6

 Сопоставьте тип угроз с примером аудита:

Ответ:

- Несанкционированный доступ
- Ошибка пользователя
- Внутреннее мошенничество

- Сбой системы

1. Попытка входа с чужим паролем
2. Удаление важной таблицы по ошибке
3. Подмена данных сотрудником
4. Сбой сервера и потеря соединений

Подсказка: Думайте, кто или что является источником проблемы.

- **Несанкционированный доступ** — Попытка входа с чужим паролем
- **Ошибка пользователя** — Удаление важной таблицы по ошибке
- **Внутреннее мошенничество** — Подмена данных сотрудником
- **Сбой системы** — Сбой сервера и потеря соединений

Шаг 07:

Миссия 7

Расположите шаги настройки аудита PostgreSQL:

1. Установить расширение pgAudit
2. Настроить логирование команд и пользователей
3. Перезапустить сервер PostgreSQL
4. Проверить журналы и корректность записей

Подсказка: Сначала устанавливаем, потом конфигурируем, затем тестируем.

Ответ:

- Установить расширение pgAudit
- Настроить логирование команд и пользователей
- Перезапустить сервер PostgreSQL
- Проверить журналы и корректность записей

Шаг 08:

Миссия 8

Расположите шаги анализа журнала безопасности:

1. Собрать логи за нужный период
2. Отфильтровать по критическим событиям
3. Составить отчёт по пользователям и действиям
4. Выявить подозрительные активности

Подсказка: Логично сначала собрать, потом фильтровать и анализировать данные.

Ответ:

- Собрать логи за нужный период
- Отфильтровать по критическим событиям
- Составить отчёт по пользователям и действиям
- Выявить подозрительные активности

Шаг 09:

Миссия 9

Расположите шаги настройки оповещений о подозрительной активности:

1. Настроить триггеры на критические таблицы

2. Записать события в лог
3. Настроить уведомления на email или систему мониторинга
4. Проверить корректность работы уведомлений

Подсказка: Сначала триггер, потом запись, уведомления и тест.

Ответ:

- Настроить триггеры на критические таблицы
- Записать события в лог
- Настроить уведомления на email или систему мониторинга
- Проверить корректность работы уведомлений

Шаг 10:

Миссия 10

Какие события стоит логировать для мониторинга безопасности?

Подсказка: Подумайте о действиях, которые могут повлиять на целостность данных.

Ответ:

- **Попытки неудачной аутентификации**
- **Изменения структуры таблиц**
- **Вставка/удаление конфиденциальных данных**
- Создание индексов для оптимизации
- Автоматическая очистка кеша

Шаг 11:

Миссия 11

 Сопоставьте тип логов с их назначением:

Ответ:

- ERROR
 - WARNING
 - NOTICE
 - INFO
1. Критические ошибки, влияющие на работу сервера
 2. Предупреждения о потенциальных проблемах
 3. Информационные сообщения о выполнении команд
 4. Обычные уведомления о действиях пользователя

Подсказка: ERROR=критическое, WARNING=предупреждение, NOTICE=уведомление, INFO=информация.

- **ERROR** — Критические ошибки, влияющие на работу сервера
- **WARNING** — Предупреждения о потенциальных проблемах
- **NOTICE** — Обычные уведомления о действиях пользователя
- **INFO** — Информационные сообщения о выполнении команд

Шаг 12:

Миссия 12

Что поможет уменьшить объем логов без потери информации о безопасности?

Подсказка: Используйте фильтры и уровни логирования.

Ответ:

- Настроить **log_min_messages** на **WARNING** или **ERROR**
- Логировать только критические таблицы и действия
- Отключить pgAudit
- Удалять логи каждый час без анализа

Шаг 13:

Миссия 13

Расположите этапы реагирования на выявленные угрозы:

1. Определить источник угрозы
2. Изолировать затронутые данные или пользователей
3. Сообщить команде безопасности
4. Принять меры для устранения угрозы
5. Документировать событие

Подсказка: Сначала идентификация, затем изоляция, уведомление, устранение, документирование.

Ответ:

- Определить источник угрозы
- Изолировать затронутые данные или пользователей
- Сообщить команде безопасности
- Принять меры для устранения угрозы
- Документировать событие

Шаг 14:

Миссия 14

 Сопоставьте инструмент мониторинга с его функцией:

Ответ:

- pg_stat_activity
 - pg_stat_user_tables
 - pg_stat_replication
 - pg_stat_database
1. Показывает текущие подключения и выполняемые запросы
 2. Содержит информацию о таблицах и их активности
 3. Мониторинг состояния репликации
 4. Статистика по базе данных в целом

Подсказка: Сопоставьте название с типом собираемых данных.

- **pg_stat_activity** — Показывает текущие подключения и выполняемые запросы
- **pg_stat_user_tables** — Содержит информацию о таблицах и их активности
- **pg_stat_replication** — Мониторинг состояния репликации
- **pg_stat_database** — Статистика по базе данных в целом

Шаг 15:

Миссия 15

Какие метрики наиболее важны для мониторинга безопасности базы данных?

Подсказка: Думайте о подозрительных действиях и ошибках.

Ответ:

- **Количество неудачных попыток входа**
- **Изменения в критических таблицах**
- **Аномальная активность пользователей**
- Время отклика сервера
- Размер базы данных

 **Шаг 16:**

Миссия 16

Расположите действия администратора после обнаружения аномальной активности:

1. Проверить журналы и детали активности
2. Отключить подозрительного пользователя
3. Уведомить команду безопасности
4. Проанализировать источник аномалии
5. Принять меры по устранению угрозы

Подсказка: Логично сначала проверка, затем изоляция, уведомление и анализ.

Ответ:

- Проверить журналы и детали активности
- Отключить подозрительного пользователя
- Уведомить команду безопасности
- Проанализировать источник аномалии
- Принять меры по устранению угрозы

 **Шаг 17:**

Миссия 17

Какой подход лучше всего для регулярного аудита безопасности в крупной системе?

Подсказка: Подумайте о регулярности, автоматизации и полноте проверки.

Ответ:

- **Автоматизированный сбор и анализ логов**
- **Регулярная проверка ключевых таблиц и действий пользователей**
- Ручная проверка отдельных событий только при проблемах
- Игнорирование предупреждений в логах

Урок 5 - Реализация резервного копирования и восстановления

 **Шаг 01:**

Миссия 1

Что является основным назначением резервного копирования базы данных?

Подсказка: Думайте о том, что происходит при потере данных.

Ответ:

- **Восстановление данных в случае сбоя**
- Ускорение работы запросов
- Увеличение объема диска

- Обновление структуры таблиц

🔮 Шаг 02:

Миссия 2

Какой метод резервного копирования сохраняет все изменения после последнего полного бэкапа?

Подсказка: Подумайте о том, что сохраняется только после предыдущего полного копирования.

Ответ:

- **Инкрементное копирование**
- Полное копирование
- Дифференциальное копирование

🔮 Шаг 03:

Миссия 3

Какой способ хранения резервных копий наиболее безопасен от локальных сбоев?

Подсказка: Подумайте о внешнем или удаленном месте хранения.

Ответ:

- **Хранение на удаленном сервере или облаке**
- Хранение на том же диске, что и база
- Хранение в той же папке с логами

🔮 Шаг 04:

Миссия 4

📄 Сопоставьте тип резервного копирования с его характеристикой:

Ответ:

- Полное копирование
 - Инкрементное копирование
 - Дифференциальное копирование
1. Сохраняет всю базу полностью
 2. Сохраняет изменения после последнего полного бэкапа
 3. Сохраняет изменения с момента последнего полного копирования, но не предыдущих инкрементов

Подсказка: Смотрите на диапазон сохраненных данных.

- **Полное копирование** — Сохраняет всю базу полностью
- **Инкрементное копирование** — Сохраняет изменения после последнего полного бэкапа
- **Дифференциальное копирование** — Сохраняет изменения с момента последнего полного копирования, но не предыдущих инкрементов

🔮 Шаг 05:

Миссия 5

Что необходимо проверить перед восстановлением базы из резервной копии?

Подсказка: Подумайте о совместимости и целостности данных.

Ответ:

- **Совместимость версии PostgreSQL**

- **Целостность резервной копии**
- Количество индексов в базе
- Количество таблиц в логе

🔧 Шаг 06:

Миссия 6

📄 Сопоставьте утилиту PostgreSQL с её назначением:

Ответ:

- pg_dump
 - pg_restore
 - pg_basebackup
1. Создание резервной копии базы данных
 2. Восстановление базы из сохраненной копии
 3. Создание полной копии кластера PostgreSQL

Подсказка: Думайте о действиях «сохранение», «восстановление», «клонирование».

- **pg_dump** — Создание резервной копии базы данных
- **pg_restore** — Восстановление базы из сохраненной копии
- **pg_basebackup** — Создание полной копии кластера PostgreSQL

🔧 Шаг 07:

Миссия 7

Почему важно тестировать процесс восстановления базы?

Подсказка: Подумайте о готовности к реальной аварии.

Ответ:

- **Чтобы убедиться, что резервные копии действительно работают**
- Чтобы уменьшить размер резервной копии
- Чтобы ускорить работу базы

🔧 Шаг 08:

Миссия 8

Расположите шаги полного восстановления базы данных:

1. Остановить базу данных
2. Создать резервную копию текущего состояния
3. Восстановить данные из резервной копии
4. Запустить базу и проверить целостность данных

Подсказка: Сначала подготовка, потом восстановление, затем проверка.

Ответ:

- Остановить базу данных
- Создать резервную копию текущего состояния
- Восстановить данные из резервной копии
- Запустить базу и проверить целостность данных

Шаг 09:

Миссия 9

Что лучше всего использовать для автоматизации регулярного резервного копирования?

Подсказка: Подумайте о планировщике задач или cron.

Ответ:

- **Скрипты с cron или планировщиком задач**
- Ручной запуск каждый раз
- Использование SELECT без бэкапа

Шаг 10:

Миссия 10

Что лучше всего использовать для автоматизации регулярного резервного копирования?

Подсказка: Подумайте о планировщике задач.

Ответ:

- **Скрипты с cron или планировщиком задач**
- Ручной запуск каждый раз
- Использование SELECT без бэкапа

Шаг 11:

Миссия 11

 Сопоставьте возможные проблемы при восстановлении с их последствиями:

Ответ:

- Поврежденная резервная копия
- Несовпадение версии PostgreSQL
- Недостаток дискового пространства

1. Невозможно восстановить данные
2. Ошибка при запуске восстановления из-за несовместимости
3. Процесс восстановления прерывается

Подсказка: Подумайте о том, какие факторы мешают восстановлению.

- **Поврежденная резервная копия** — Невозможно восстановить данные
- **Несовпадение версии PostgreSQL** — Ошибка при запуске восстановления из-за несовместимости
- **Недостаток дискового пространства** — Процесс восстановления прерывается

Шаг 12:

Миссия 12

Какой метод восстановления позволяет откатиться к конкретной точке во времени?

Подсказка: Думайте о журнале транзакций.

Ответ:

- **Point-in-time recovery (PITR)**
- Полное восстановление
- Инкрементное восстановление

Шаг 13:

Миссия 13

Расположите этапы настройки PITR в правильном порядке:

1. Включить архивирование WAL
2. Сделать полную резервную копию базы
3. Настроить восстановление до нужной точки
4. Восстановить базу из полной копии и WAL

Подсказка: Сначала архивирование, потом полное копирование, далее восстановление.

Ответ:

- Включить архивирование WAL
- Сделать полную резервную копию базы
- Настроить восстановление до нужной точки
- Восстановить базу из полной копии и WAL

Шаг 14:

Миссия 14

Почему важно хранить резервные копии на разных носителях?

Подсказка: Подумайте о рисках аппаратных сбоев.

Ответ:

- Чтобы защититься от потери данных при выходе из строя носителя
- Чтобы ускорить восстановление
- Чтобы уменьшить размер резервной копии

Шаг 15:

Миссия 15

 Сопоставьте термин с его определением:

Ответ:

- Полная резервная копия
 - Инкрементная резервная копия
 - Дифференциальная резервная копия
 - PITR
1. Сохраняет все данные базы целиком
 2. Сохраняет изменения после последнего полного бэкапа
 3. Сохраняет изменения с момента последнего полного бэкапа, включая все предыдущие инкременты
 4. Восстановление до конкретного момента времени

Подсказка: Смотрите на количество данных и точку восстановления.

- **Полная резервная копия** — Сохраняет все данные базы целиком
- **Инкрементная резервная копия** — Сохраняет изменения после последнего полного бэкапа
- **Дифференциальная резервная копия** — Сохраняет изменения с момента последнего полного бэкапа, включая все предыдущие инкременты
- **PITR** — Восстановление до конкретного момента времени

Шаг 16:

Миссия 16

Какой тип резервного копирования экономит больше всего места на диске?

Подсказка: Думайте о сохранении только изменений.

Ответ:

- **Инкрементное копирование**
- Полное копирование
- Дифференциальное копирование

Шаг 17:

Миссия 17

Расположите этапы создания автоматизированного бэкапа:

1. Написание скрипта резервного копирования
2. Тестирование скрипта вручную
3. Добавление задания в cron или планировщик
4. Регулярная проверка восстановления резервных копий

Подсказка: Сначала подготовка, потом тестирование, далее автоматизация и проверка.

Ответ:

- Написание скрипта резервного копирования
- Тестирование скрипта вручную
- Добавление задания в cron или планировщик
- Регулярная проверка восстановления резервных копий

Шаг 18:

Миссия 18

Что необходимо делать после каждой процедуры восстановления для уверенности в целостности данных?

Подсказка: Подумайте о проверке функциональности.

Ответ:

- **Проверять корректность и целостность данных**
- Удалять резервные копии
- Обновлять сервер PostgreSQL

Модуль 7 - Итоговый проект

Урок 1 - Разработка ТЗ для интеграции PHP и PostgreSQL

Шаг 01:

Миссия 1

Программист Лена хочет подключить PostgreSQL к PHP-проекту. Какой метод подключения является наиболее стандартным и безопасным?

Подсказка: Используйте современные расширения, которые поддерживают подготовленные выражения.

Ответ:

- **Использование PDO (PHP Data Objects)**
- Использование устаревших функций pg_connect
- Прямое подключение через exec()
- Использование fopen() для базы

🔮 Шаг 02:

Миссия 2

Разработчик Алексей хочет защитить свои SQL-запросы от инъекций. Что ему следует использовать?

Подсказка: Думайте о безопасной подстановке переменных в запрос.

Ответ:

- **Подготовленные выражения через PDO**
- Объединение строк через конкатенацию
- Использование eval()
- Ввод данных напрямую в SQL

🔮 Шаг 03:

Миссия 3

Игорь работает с большими таблицами в PostgreSQL. Какой подход поможет уменьшить время выполнения сложного запроса?

Подсказка: Подумайте о структуре данных и индексах.

Ответ:

- **Создание индексов по часто используемым полям**
- Использование SELECT * без фильтров
- Повторное создание таблиц на лету
- Отключение кэширования

🔮 Шаг 04:

Миссия 4

Сопоставьте термин из PHP/PostgreSQL с его объяснением.

Понятия:

Ответ:

- PDO
- pg_query
- Подготовленные выражения
- Индекс

Определения:

1. Расширение PHP для безопасного подключения к базам данных
2. Функция для выполнения SQL-запроса
3. Метод защиты от SQL-инъекций
4. Структура для ускорения поиска данных

Подсказка: Подумайте о назначении каждого элемента: безопасность, скорость, выполнение.

- **PDO** — Расширение PHP для безопасного подключения к базам данных

- **pg_query** — Функция для выполнения SQL-запроса
- **Подготовленные выражения** — Метод защиты от SQL-инъекций
- **Индекс** — Структура для ускорения поиска данных

🔮 Шаг 05:

Миссия 5

Сопоставьте тип JOIN с его описанием в PostgreSQL.

Ответ:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN

1. Возвращает только совпадающие строки из обеих таблиц
2. Возвращает все строки из левой таблицы и совпадения из правой
3. Возвращает все строки из правой таблицы и совпадения из левой
4. Возвращает все строки из обеих таблиц, совпадения или NULL

Подсказка: Представьте таблицы как два списка и подумайте, какие строки останутся после объединения.

- **INNER JOIN** — Возвращает только совпадающие строки из обеих таблиц
- **LEFT JOIN** — Возвращает все строки из левой таблицы и совпадения из правой
- **RIGHT JOIN** — Возвращает все строки из правой таблицы и совпадения из левой
- **FULL OUTER JOIN** — Возвращает все строки из обеих таблиц, совпадения или NULL

🔮 Шаг 06:

Миссия 6

Сопоставьте тип данных PostgreSQL с его использованием.

Ответ:

- INTEGER
- TEXT
- BOOLEAN
- TIMESTAMP

1. Для хранения целых чисел
2. Для хранения строковых данных произвольной длины
3. Для хранения логических значений
4. Для хранения даты и времени

Подсказка: Обратите внимание на смысл каждого типа и реальные примеры использования в проектах.

- **INTEGER** — Для хранения целых чисел
- **TEXT** — Для хранения строковых данных произвольной длины
- **BOOLEAN** — Для хранения логических значений
- **TIMESTAMP** — Для хранения даты и времени

🔮 Шаг 07:

Миссия 7

Сопоставьте SQL-оператор с его действием.

Ответ:

- SELECT
- INSERT
- UPDATE
- DELETE

1. Выбирает данные из таблицы
2. Добавляет новые строки
3. Изменяет существующие строки
4. Удаляет строки

Подсказка: Подумайте о CRUD-операциях и соответствующих SQL-командах.

- **SELECT** — Выбирает данные из таблицы
- **INSERT** — Добавляет новые строки
- **UPDATE** — Изменяет существующие строки
- **DELETE** — Удаляет строки

Шаг 08:

Миссия 8

Сопоставьте термин PHP с его назначением в интеграции с PostgreSQL.

Ответ:

- fetch()
- prepare()
- execute()
- commit()

1. Получение результата запроса
2. Подготовка запроса для последующего выполнения
3. Выполнение подготовленного запроса
4. Подтверждение транзакции

Подсказка: Обратите внимание на последовательность работы с базой данных.

- **fetch()** — Получение результата запроса
- **prepare()** — Подготовка запроса для последующего выполнения
- **execute()** — Выполнение подготовленного запроса
- **commit()** — Подтверждение транзакции

Шаг 09:

Миссия 9

Расположите этапы подключения PHP к PostgreSQL в правильном порядке:

1. Создание объекта PDO
2. Настройка DSN и учетных данных
3. Подготовка и выполнение запроса
4. Обработка результатов запроса

Подсказка: Сначала нужно настроить подключение, затем работать с запросами и результатами.

Ответ:

- Настройка DSN и учетных данных
- Создание объекта PDO
- Подготовка и выполнение запроса
- Обработка результатов запроса

 Шаг 10:

Миссия 10

Расположите шаги безопасной работы с транзакциями в PostgreSQL через PHP:

1. Начало транзакции
2. Выполнение запросов
3. Commit или Rollback
4. Обработка ошибок

Подсказка: Обратите внимание на последовательность действий для надежности данных.

Ответ:

- Начало транзакции
- Выполнение запросов
- Обработка ошибок
- Commit или Rollback

 Шаг 11:

Миссия 11

Расположите шаги от создания структуры таблицы до использования данных в PHP:

1. Создание таблицы в PostgreSQL
2. Вставка тестовых данных
3. Подключение к базе через PDO
4. Выборка данных и отображение в приложении

Подсказка: Сначала создаем структуру, потом наполняем и используем.

Ответ:

- Создание таблицы в PostgreSQL
- Вставка тестовых данных
- Подключение к базе через PDO
- Выборка данных и отображение в приложении

 Шаг 12:

Миссия 12

Расположите этапы рефакторинга SQL-запроса в правильном порядке:

1. Определение проблемных мест (дублирование, сложные JOIN)
2. Оптимизация условий и индексов
3. Проверка на корректность и производительность
4. Документирование изменений

Подсказка: Сначала выявляем проблемы, затем исправляем, проверяем и фиксируем результат.

Ответ:

- Определение проблемных мест (дублирование, сложные JOIN)
- Оптимизация условий и индексов
- Проверка на корректность и производительность
- Документирование изменений

 Шаг 13:

Миссия 13

Расположите шаги интеграции сложного отчета PHP с PostgreSQL:

1. Анализ требований к отчету
2. Создание SQL-запросов с фильтрами и агрегатами
3. Подключение к базе через PDO
4. Вывод данных в веб-интерфейс

Подсказка: Сначала нужно понять, что требуется пользователю, потом получать и отображать данные.

Ответ:

- Анализ требований к отчету
- Создание SQL-запросов с фильтрами и агрегатами
- Подключение к базе через PDO
- Вывод данных в веб-интерфейс

 Шаг 14:

Миссия 14

Разработчик Катя обнаружила, что запрос к базе слишком медленный. Что она может сделать в первую очередь?

Подсказка: Сначала ищите узкие места в запросах и структуре таблиц.

Ответ:

- **Проверить наличие индексов на полях, используемых в WHERE и JOIN**
- Увеличить количество серверов без анализа запросов
- Переписать PHP без изменений SQL
- Удалить все временные таблицы

 Шаг 15:

Миссия 15

Программист Денис хочет логировать все запросы к базе для анализа. Что наиболее подходяще?

Подсказка: Подумайте о встроенных средствах PostgreSQL и удобстве интеграции с PHP.

Ответ:

- **Использовать pg_stat_statements и логи сервера**
- Логировать через echo в PHP без фильтров
- Записывать все данные в отдельную таблицу вручную
- Использовать клиентскую сторону браузера для логов

 Шаг 16:

Миссия 16

Расположите шаги при оптимизации большого отчета в PHP с PostgreSQL:

1. Анализ текущих запросов и времени выполнения
2. Создание индексов и оптимизация JOIN
3. Рефакторинг SQL-запросов и использование подготовленных выражений
4. Тестирование и сравнение времени выполнения

Подсказка: Сначала оцениваем проблему, потом улучшаем структуру и проверяем результат.

Ответ:

- Анализ текущих запросов и времени выполнения
- Создание индексов и оптимизация JOIN
- Рефакторинг SQL-запросов и использование подготовленных выражений
- Тестирование и сравнение времени выполнения

Шаг 17:

Миссия 17

Расположите этапы работы с транзакциями при массовом обновлении данных:

1. Начало транзакции
2. Выполнение обновлений и проверка корректности данных
3. Обработка ошибок и откат при необходимости
4. Подтверждение транзакции (commit)

Подсказка: Обратите внимание на порядок действий: сначала начало, затем изменения, потом проверка и фиксация.

Ответ:

- Начало транзакции
- Выполнение обновлений и проверка корректности данных
- Обработка ошибок и откат при необходимости
- Подтверждение транзакции (commit)

Шаг 18:

Миссия 18

Расположите шаги интеграции логирования запросов PHP с PostgreSQL:

1. Настройка расширения pg_stat_statements
2. Подключение к базе через PDO
3. Включение логирования и сохранение статистики
4. Анализ собранных данных для оптимизации

Подсказка: Сначала включаем сбор информации, потом подключаемся и анализируем результаты.

Ответ:

- Настройка расширения pg_stat_statements
- Подключение к базе через PDO
- Включение логирования и сохранение статистики
- Анализ собранных данных для оптимизации

Урок 2 - Оптимизация запросов и структуры данных

Шаг 01:

Миссия 1

Программист Лена анализирует производительность базы PostgreSQL. Какой способ поможет ей определить, какие запросы выполняются дольше всего?

Подсказка: Используйте встроенные средства PostgreSQL для мониторинга.

Ответ:

- **Включение `pg_stat_statements` для анализа запросов**
- Ручная проверка всех SELECT запросов
- Переписывание всех процедур без анализа
- Использование `echo` для замеров в PHP

Шаг 02:

Миссия 2

Алексей хочет ускорить выборку данных с большой таблицы. Что будет наиболее эффективным?

Подсказка: Подумайте о том, как СУБД ищет строки и как ускорить поиск.

Ответ:

- **Создание индексов на часто используемых полях**
- Удаление WHERE из запросов
- Использование SELECT * без фильтров
- Повторное копирование таблицы для ускорения

Шаг 03:

Миссия 3

Разработчик Катя хочет уменьшить нагрузку на сервер при сложных JOIN. Что стоит сделать в первую очередь?

Подсказка: Подумайте, что делает JOIN медленным на больших таблицах.

Ответ:

- **Добавить индексы на поля, участвующие в соединении**
- Разделить JOIN на несколько отдельных запросов без анализа
- Удалить все условия ON
- Использовать курсоры для каждой строки вручную

Шаг 04:

Миссия 4

Сопоставьте элементы PostgreSQL с их назначением при оптимизации запросов.

Ответ:

- Индекс
 - VACUUM
 - EXPLAIN
 - CTE
1. Оптимизация поиска строк по полям
 2. Очистка и реорганизация таблиц для повышения производительности

3. Анализ плана выполнения запроса
4. Создание временного результата для упрощения сложных запросов

Подсказка: Подумайте, как СУБД использует каждый инструмент для ускорения работы.

- **Индекс** — Оптимизация поиска строк по полям
- **VACUUM** — Очистка и реорганизация таблиц для повышения производительности
- **EXPLAIN** — Анализ плана выполнения запроса
- **CTE** — Создание временного результата для упрощения сложных запросов

Шаг 05:

Миссия 5

Сопоставьте тип JOIN с его особенностью при больших таблицах.

Ответ:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN

1. Возвращает только совпадающие строки
2. Возвращает все строки левой таблицы и совпадения правой
3. Возвращает все строки правой таблицы и совпадения левой
4. Возвращает все строки обеих таблиц, включая NULL для несоответствий

Подсказка: Представьте, какие строки останутся после объединения двух больших таблиц.

- **INNER JOIN** — Возвращает только совпадающие строки
- **LEFT JOIN** — Возвращает все строки левой таблицы и совпадения правой
- **RIGHT JOIN** — Возвращает все строки правой таблицы и совпадения левой
- **FULL OUTER JOIN** — Возвращает все строки обеих таблиц, включая NULL для несоответствий

Шаг 06:

Миссия 6

Сопоставьте SQL-оператор с его ролью в оптимизации запросов.

Ответ:

- SELECT
- INSERT
- UPDATE
- DELETE

1. Выборка данных из таблиц для анализа
2. Добавление новых строк с учетом индексов
3. Изменение существующих строк эффективно с использованием WHERE
4. Удаление строк с минимальной нагрузкой на базу

Подсказка: Подумайте о CRUD и как каждое действие влияет на производительность.

- **SELECT** — Выборка данных из таблиц для анализа
- **INSERT** — Добавление новых строк с учетом индексов
- **UPDATE** — Изменение существующих строк эффективно с использованием WHERE

- **DELETE** — Удаление строк с минимальной нагрузкой на базу

Шаг 07:

Миссия 7

Сопоставьте тип данных PostgreSQL с его применением при оптимизации хранения.

Ответ:

- **INTEGER**
- **TEXT**
- **BOOLEAN**
- **TIMESTAMP**

1. Для числовых идентификаторов и счетчиков
2. Для текстовой информации и описаний
3. Для логических значений и флагов
4. Для даты и времени событий

Подсказка: Подумайте, как правильный тип данных уменьшает размер таблицы и ускоряет запросы.

- **INTEGER** — Для числовых идентификаторов и счетчиков
- **TEXT** — Для текстовой информации и описаний
- **BOOLEAN** — Для логических значений и флагов
- **TIMESTAMP** — Для даты и времени событий

Шаг 08:

Миссия 8

Сопоставьте функции PHP/PDO с их задачами при оптимизации запросов.

Ответ:

- **prepare()**
- **execute()**
- **fetch()**
- **commit()**

1. Подготовка запроса для безопасного выполнения
2. Выполнение подготовленного запроса
3. Получение результатов запроса
4. Фиксация изменений после транзакции

Подсказка: Подумайте о последовательности работы с базой.

- **prepare()** — Подготовка запроса для безопасного выполнения
- **execute()** — Выполнение подготовленного запроса
- **fetch()** — Получение результатов запроса
- **commit()** — Фиксация изменений после транзакции

Шаг 09:

Миссия 9

Расположите шаги оптимизации SELECT-запроса на большой таблице:

1. Анализ текущего запроса через EXPLAIN

2. Добавление необходимых индексов
3. Рефакторинг запроса и использование JOIN/CTE
4. Тестирование производительности после изменений

Подсказка: Сначала выясните узкие места, потом оптимизируйте и проверьте результат.

Ответ:

- Анализ текущего запроса через EXPLAIN
- Добавление необходимых индексов
- Рефакторинг запроса и использование JOIN/CTE
- Тестирование производительности после изменений

Шаг 10:

Миссия 10

Расположите шаги при работе с транзакциями для массового обновления:

1. Начало транзакции
2. Выполнение обновлений с проверкой условий
3. Обработка ошибок и возможный откат
4. Фиксация изменений с commit

Подсказка: Последовательность действий важна для сохранности данных.

Ответ:

- Начало транзакции
- Выполнение обновлений с проверкой условий
- Обработка ошибок и возможный откат
- Фиксация изменений с commit

Шаг 11:

Миссия 11

Расположите шаги оптимизации JOIN-запроса:

1. Определение узких мест через EXPLAIN
2. Добавление индексов на поля соединения
3. Переписывание запроса с учетом индексов и CTE
4. Тестирование и анализ времени выполнения

Подсказка: Подумайте о том, как СУБД обрабатывает JOIN.

Ответ:

- Определение узких мест через EXPLAIN
- Добавление индексов на поля соединения
- Переписывание запроса с учетом индексов и CTE
- Тестирование и анализ времени выполнения

Шаг 12:

Миссия 12

Расположите шаги реорганизации таблиц для ускорения работы:

1. VACUUM и ANALYZE для очистки и статистики
2. Добавление/оптимизация индексов

3. Рефакторинг структуры таблицы при необходимости
4. Тестирование запросов после изменений

Подсказка: Сначала очищаем, потом структурируем, затем проверяем.

Ответ:

- VACUUM и ANALYZE для очистки и статистики
- Добавление/оптимизация индексов
- Рефакторинг структуры таблицы при необходимости
- Тестирование запросов после изменений

Шаг 13:

Миссия 13

Расположите этапы анализа медленного SELECT-запроса:

1. Сбор данных о времени выполнения
2. Определение проблемных JOIN и фильтров
3. Использование EXPLAIN для проверки плана выполнения
4. Внедрение индексов и переписывание запроса
5. Тестирование после оптимизации

Подсказка: Сначала диагностируем, потом исправляем и проверяем.

Ответ:

- Сбор данных о времени выполнения
- Определение проблемных JOIN и фильтров
- Использование EXPLAIN для проверки плана выполнения
- Внедрение индексов и переписывание запроса
- Тестирование после оптимизации

Шаг 14:

Миссия 14

Расположите шаги создания отчета с агрегатами для большой базы:

1. Определение требований к отчету
2. Создание SQL-запросов с группировками и фильтрами
3. Использование индексов для ускорения выборки
4. Подключение через PDO и вывод результата

Подсказка: Сначала понимаем, что нужно пользователю, потом строим запрос, оптимизируем и выводим.

Ответ:

- Определение требований к отчету
- Создание SQL-запросов с группировками и фильтрами
- Использование индексов для ускорения выборки
- Подключение через PDO и вывод результата

Шаг 15:

Миссия 15

Расположите шаги оптимизации массовой вставки данных:

1. Использование COPY вместо INSERT для больших объемов
2. Отключение ненужных индексов на время загрузки
3. Вставка данных пакетами
4. Включение индексов и тестирование производительности

Подсказка: Большие объемы вставки требуют пакетной работы и минимизации лишних операций.

Ответ:

- Использование COPY вместо INSERT для больших объемов
- Отключение ненужных индексов на время загрузки
- Вставка данных пакетами
- Включение индексов и тестирование производительности

Шаг 16:

Миссия 16

Расположите шаги интеграции логирования запросов для оптимизации:

1. Включение pg_stat_statements
2. Подключение PHP к базе
3. Сбор и сохранение статистики выполнения
4. Анализ логов и оптимизация медленных запросов

Подсказка: Сначала собираем данные, потом анализируем и исправляем медленные места.

Ответ:

- Включение pg_stat_statements
- Подключение PHP к базе
- Сбор и сохранение статистики выполнения
- Анализ логов и оптимизация медленных запросов

Шаг 17:

Миссия 17

Расположите шаги комплексной оптимизации структуры базы:

1. Анализ больших таблиц и узких мест
2. Добавление индексов и переработка схемы
3. Очистка таблиц через VACUUM
4. Тестирование производительности всех запросов

Подсказка: Сначала анализируем, потом оптимизируем структуру и проверяем результаты.

Ответ:

- Анализ больших таблиц и узких мест
- Добавление индексов и переработка схемы
- Очистка таблиц через VACUUM
- Тестирование производительности всех запросов

Урок 3 - Масштабирование PostgreSQL для обработки большого объема данных

Шаг 01:

Миссия 1

Программист Игорь хочет понять, как лучше масштабировать PostgreSQL при росте объема данных. Что следует проверить в первую очередь?

Подсказка: Начните с анализа текущих узких мест в производительности.

Ответ:

- **Анализ загрузки сервера и времени выполнения запросов**
- Сразу добавлять новые серверы без анализа
- Переписывать все запросы вручную
- Удалять данные из таблиц, чтобы уменьшить нагрузку

Шаг 02:

Миссия 2

Разработчица Ольга хочет ускорить выборку данных из большой таблицы. Что будет наиболее эффективным?

Подсказка: Подумайте о том, как СУБД ищет строки в таблице.

Ответ:

- **Создание индексов на часто используемых колонках**
- Удаление WHERE из запросов
- Использование SELECT * без фильтров
- Повторная выгрузка таблицы в новый формат

Шаг 03:

Миссия 3

Инженер Пётр хочет уменьшить нагрузку на сервер при сложных JOIN. Что стоит сделать в первую очередь?

Подсказка: Подумайте, что делает JOIN медленным на больших таблицах.

Ответ:

- **Добавить индексы на поля, участвующие в соединении**
- Разделить JOIN на несколько отдельных запросов без анализа
- Удалить все условия ON
- Использовать курсоры для каждой строки вручную

Шаг 04:

Миссия 4

Сопоставьте понятия PostgreSQL с их ролью при масштабировании больших данных.

Ответ:

- Шардинг
- Репликация
- Индексирование

- VACUUM

1. Разделение данных на несколько серверов для распределения нагрузки
2. Создание копий базы для отказоустойчивости и балансировки чтения
3. Ускорение поиска по большим таблицам
4. Очистка и реорганизация таблиц для улучшения производительности

Подсказка: Подумайте, какой механизм влияет на скорость, надежность и структуру данных.

- **Шардинг** — Разделение данных на несколько серверов для распределения нагрузки
- **Репликация** — Создание копий базы для отказоустойчивости и балансировки чтения
- **Индексирование** — Ускорение поиска по большим таблицам
- **VACUUM** — Очистка и реорганизация таблиц для улучшения производительности

Шаг 05:

Миссия 5

Сопоставьте тип репликации с его особенностью.

Ответ:

- Синхронная репликация
 - Асинхронная репликация
 - Логическая репликация
 - Физическая репликация
1. Данные подтверждаются на всех репликах одновременно
 2. Данные на реплике могут отставать от мастера
 3. Репликация на уровне таблиц и данных с возможностью фильтрации
 4. Дублирование всей физической базы без фильтров

Подсказка: Подумайте о скорости и точности копирования данных.

- **Синхронная репликация** — Данные подтверждаются на всех репликах одновременно
- **Асинхронная репликация** — Данные на реплике могут отставать от мастера
- **Логическая репликация** — Репликация на уровне таблиц и данных с возможностью фильтрации
- **Физическая репликация** — Дублирование всей физической базы без фильтров

Шаг 06:

Миссия 6

Сопоставьте методы оптимизации запросов с их эффектом при больших объемах данных.

Ответ:

- EXPLAIN
 - Partitioning
 - Materialized Views
 - Connection Pooling
1. Анализ плана выполнения запроса
 2. Разделение таблиц на части для ускорения выборки
 3. Предварительный расчет и хранение агрегированных данных
 4. Повышение количества одновременных подключений без перегрузки сервера

Подсказка: Подумайте о том, что влияет на скорость и нагрузку базы.

- **EXPLAIN** — Анализ плана выполнения запроса
- **Partitioning** — Разделение таблиц на части для ускорения выборки
- **Materialized Views** — Предварительный расчет и хранение агрегированных данных
- **Connection Pooling** — Повышение количества одновременных подключений без перегрузки сервера

Шаг 07:

Миссия 7

Расположите шаги масштабирования базы с помощью шардинга:

1. Определение ключа шардинга
2. Разделение таблиц на несколько серверов
3. Перенос данных на новые шарды
4. Тестирование корректности распределения данных

Подсказка: Сначала определяем стратегию, потом распределяем данные и проверяем.

Ответ:

- Определение ключа шардинга
- Разделение таблиц на несколько серверов
- Перенос данных на новые шарды
- Тестирование корректности распределения данных

Шаг 08:

Миссия 8

Расположите этапы настройки репликации для увеличения отказоустойчивости:

1. Выбор типа репликации (синхронная или асинхронная)
2. Настройка реплик и их подключения к мастеру
3. Тестирование работы репликации
4. Мониторинг задержки и корректности данных

Подсказка: Настройка репликации требует последовательного выполнения всех шагов.

Ответ:

- Выбор типа репликации (синхронная или асинхронная)
- Настройка реплик и их подключения к мастеру
- Тестирование работы репликации
- Мониторинг задержки и корректности данных

Шаг 09:

Миссия 9

Расположите шаги оптимизации индексов при больших таблицах:

1. Анализ текущих запросов с помощью EXPLAIN
2. Определение колонок, где индексы дадут максимальный эффект
3. Создание новых индексов или изменение существующих
4. Тестирование производительности после изменений

Подсказка: Индексы помогают только там, где они реально ускоряют поиск.

Ответ:

- Анализ текущих запросов с помощью EXPLAIN
- Определение колонок, где индексы дадут максимальный эффект
- Создание новых индексов или изменение существующих
- Тестирование производительности после изменений

 Шаг 10:

Миссия 10

Расположите шаги внедрения Materialized Views для ускорения отчетов:

1. Определение тяжелых запросов для отчета
2. Создание Materialized View с необходимыми агрегатами
3. Настройка регулярного обновления данных
4. Тестирование запросов к Materialized View

Подсказка: Сначала выбираем что ускорять, потом создаем и обновляем материализованные данные.

Ответ:

- Определение тяжелых запросов для отчета
- Создание Materialized View с необходимыми агрегатами
- Настройка регулярного обновления данных
- Тестирование запросов к Materialized View

 Шаг 11:

Миссия 11

Расположите шаги настройки Partitioning для больших таблиц:

1. Выбор типа партиционирования (range, list, hash)
2. Создание партиций и перенесение данных
3. Настройка индексов на каждой партиции
4. Тестирование запросов к партиционированной таблице

Подсказка: Партиционирование помогает ускорить выборку, но важно правильно выбрать тип и индексы.

Ответ:

- Выбор типа партиционирования (range, list, hash)
- Создание партиций и перенесение данных
- Настройка индексов на каждой партиции
- Тестирование запросов к партиционированной таблице

 Шаг 12:

Миссия 12

Расположите шаги настройки Connection Pooling для PHP и PostgreSQL:

1. Установка и настройка пула подключений
2. Подключение PHP через пул вместо прямого соединения
3. Мониторинг числа активных соединений
4. Тестирование производительности при высоких нагрузках

Подсказка: Connection Pooling помогает избежать переподключений и разгрузить сервер.

Ответ:

- Установка и настройка пула подключений
- Подключение PHP через пул вместо прямого соединения
- Мониторинг числа активных соединений
- Тестирование производительности при высоких нагрузках

Шаг 13:

Миссия 13

Расположите шаги настройки логирования медленных запросов:

1. Включение параметров log_min_duration_statement
2. Сбор и хранение логов
3. Анализ медленных запросов
4. Оптимизация медленных запросов (индексы, переписывание, партиционирование)

Подсказка: Сначала фиксируем проблему, потом анализируем и устраняем.

Ответ:

- Включение параметров log_min_duration_statement
- Сбор и хранение логов
- Анализ медленных запросов
- Оптимизация медленных запросов (индексы, переписывание, партиционирование)

Шаг 14:

Миссия 14

Расположите шаги настройки автоматического бэкапа большой базы данных:

1. Определение стратегии резервного копирования (полные, инкрементальные, логические)
2. Настройка расписания бэкапов и места хранения
3. Проверка работоспособности бэкапов
4. Регулярное тестирование восстановления данных

Подсказка: Надежный бэкап включает планирование, выполнение и проверку восстановления.

Ответ:

- Определение стратегии резервного копирования (полные, инкрементальные, логические)
- Настройка расписания бэкапов и места хранения
- Проверка работоспособности бэкапов
- Регулярное тестирование восстановления данных

Шаг 15:

Миссия 15

Расположите шаги для настройки мониторинга производительности PostgreSQL:

1. Выбор метрик для отслеживания (CPU, I/O, slow queries)
2. Настройка инструментов мониторинга и дашбордов
3. Регулярный сбор и анализ метрик
4. Принятие решений по оптимизации на основе данных

Подсказка: Сначала определяем что измерять, затем настраиваем сбор и делаем выводы для улучшения.

Ответ:

- Выбор метрик для отслеживания (CPU, I/O, slow queries)
- Настройка инструментов мониторинга и дашбордов
- Регулярный сбор и анализ метрик
- Принятие решений по оптимизации на основе данных

Шаг 16:

Миссия 16

Расположите шаги внедрения горизонтального масштабирования при большом количестве запросов:

1. Оценка текущей нагрузки и узких мест
2. Разделение данных на отдельные серверы (шардинг)
3. Настройка балансировки нагрузки между серверами
4. Тестирование производительности и отказоустойчивости

Подсказка: Горизонтальное масштабирование требует анализа, распределения данных, балансировки и проверки системы.

Ответ:

- Оценка текущей нагрузки и узких мест
- Разделение данных на отдельные серверы (шардинг)
- Настройка балансировки нагрузки между серверами
- Тестирование производительности и отказоустойчивости

Урок 4 - Обработка ошибок и создание надежных решений

Шаг 01:

Миссия 1

Вы разрабатываете систему учета заказов, и при подключении к PostgreSQL появляется ошибка соединения. Что нужно сделать в первую очередь?

Подсказка: Начните с проверки базовых настроек подключения, чтобы исключить простые ошибки.

Ответ:

- Проверить параметры подключения (хост, порт, логин, пароль)
- Сразу переписывать весь код работы с базой
- Игнорировать ошибку и продолжить работу
- Перезагрузить сервер PostgreSQL без анализа

Шаг 02:

Миссия 2

При работе с пользовательскими данными в PHP вы хотите предотвратить SQL-инъекции. Что наиболее эффективно?

Подсказка: Подумайте, как безопасно вставлять данные в запросы.

Ответ:

- Использовать подготовленные выражения (prepared statements)

- Просто экранировать кавычки
- Писать запросы через конкатенацию строк
- Не проверять данные, доверяя пользователю

🔧 Шаг 03:

Миссия 3

Ваш скрипт PHP иногда падает с ошибкой "Division by zero". Какое решение будет корректным?

Подсказка: Подумайте, как избежать ошибок до их возникновения.

Ответ:

- **Проверять делитель перед операцией деления**
- Отключить отображение ошибок
- Перехватывать все ошибки try-catch без условий
- Игнорировать ошибку и продолжать выполнение

🔧 Шаг 04:

Миссия 4

Вы строите логирование действий пользователей в PostgreSQL. Как лучше хранить время события?

Подсказка: Подумайте, как PostgreSQL работает с временными типами.

Ответ:

- **Использовать тип TIMESTAMP с часовым поясом**
- Сохранять дату как строку
- Хранить время в формате UNIX только как число
- Не хранить время, только действие

🔧 Шаг 05:

Миссия 5

📄 Задача: Сопоставление ошибок PHP и их причин

Формулировка: Сопоставьте тип ошибки с ее типичной причиной.

Понятия:

Ответ:

- Notice
- Warning
- Fatal Error
- Parse Error

Определения:

1. Использование неинициализированной переменной
2. Ошибка синтаксиса
3. Попытка подключить несуществующий файл
4. Попытка вызвать несуществующую функцию

🔧 Подсказка:

Обратите внимание на то, когда выполнение скрипта останавливается, а когда просто предупреждает.

- **Notice** — Использование неинициализированной переменной

- **Warning** — Попытка подключить несуществующий файл
- **Fatal Error** — Попытка вызвать несуществующую функцию
- **Parse Error** — Ошибка синтаксиса

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте типы транзакций PostgreSQL с их поведением:

Понятия:

Ответ:

- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE
- AUTOCOMMIT

Определения:

1. Каждый запрос видит только зафиксированные данные на момент начала запроса
2. Гарантирует полную изоляцию транзакций
3. Каждая транзакция видит данные, зафиксированные на момент начала транзакции
4. Автоматическое выполнение каждой инструкции как отдельной транзакции

🔧 Подсказка:

Помните: уровни изоляции управляют видимостью изменений других транзакций.

- **READ COMMITTED** — Каждый запрос видит только зафиксированные данные на момент начала запроса
- **REPEATABLE READ** — Каждая транзакция видит данные, зафиксированные на момент начала транзакции
- **SERIALIZABLE** — Гарантирует полную изоляцию транзакций
- **AUTOCOMMIT** — Автоматическое выполнение каждой инструкции как отдельной транзакции

🔧 Шаг 07:

Миссия 7

📖 Сопоставьте исключения PHP с их типичными источниками:

Понятия:

Ответ:

- PDOException
- InvalidArgumentException
- RuntimeException
- LogicException

Определения:

1. Ошибка работы с базой данных
2. Неверный аргумент функции
3. Ошибка выполнения программы в реальном времени
4. Ошибка логики кода

🔧 Подсказка:

Исключения помогают различать ошибки среды, аргументов и логики программы.

- **PDOException** — Ошибка работы с базой данных
- **InvalidArgumentException** — Неверный аргумент функции
- **RuntimeException** — Ошибка выполнения программы в реальном времени
- **LogicException** — Ошибка логики кода

Шаг 08:

Миссия 8

 Сопоставьте методы резервного копирования PostgreSQL с их описанием:

Понятия:

Ответ:

- `pg_dump`
- `pg_basebackup`
- Logical Replication
- Continuous Archiving

Определения:

1. Создание логической копии конкретной базы или таблицы
2. Полное физическое копирование всего кластера
3. Поддержание актуальной копии базы в реальном времени
4. Архивация WAL для восстановления после сбоев

 **Подсказка:**

Логическое и физическое резервное копирование имеют разные цели и стратегии.

- **`pg_dump`** — Создание логической копии конкретной базы или таблицы
- **`pg_basebackup`** — Полное физическое копирование всего кластера
- **Logical Replication** — Поддержание актуальной копии базы в реальном времени
- **Continuous Archiving** — Архивация WAL для восстановления после сбоев

Шаг 09:

Миссия 9

Расположите шаги обработки ошибок в правильном порядке при работе с PostgreSQL и PHP:

1. Проверка возвращаемого значения функции или запроса
2. Логирование ошибки для последующего анализа
3. Отправка пользователю дружелюбного сообщения
4. Перехват исключения через try-catch

 **Подсказка:**

Сначала ловим ошибку, затем фиксируем её, информируем пользователя и проверяем возвращаемые значения.

Ответ:

- Перехват исключения через try-catch
- Проверка возвращаемого значения функции или запроса
- Логирование ошибки для последующего анализа
- Отправка пользователю дружелюбного сообщения

Шаг 10:

Миссия 10

Расположите действия для безопасного использования транзакций в PostgreSQL:

1. Начало транзакции
2. Выполнение SQL-запросов
3. Фиксация или откат транзакции
4. Обработка ошибок и логирование

Подсказка:

Помните: транзакция начинается, выполняется, затем либо фиксируется, либо откатывается с учетом ошибок.

Ответ:

- Начало транзакции
- Выполнение SQL-запросов
- Обработка ошибок и логирование
- Фиксация или откат транзакции

Шаг 11:

Миссия 11

Расположите этапы миграции базы данных в правильном порядке:

1. Создание резервной копии существующей базы
2. Разработка скриптов миграции
3. Тестирование миграции на копии базы
4. Применение миграции к рабочей базе

Подсказка:

Начинайте с защиты данных, затем создавайте и проверяйте скрипты, и только после этого внедряйте изменения.

Ответ:

- Создание резервной копии существующей базы
- Разработка скриптов миграции
- Тестирование миграции на копии базы
- Применение миграции к рабочей базе

Шаг 12:

Миссия 12

Расположите шаги оптимизации запросов к PostgreSQL:

1. Анализ текущих запросов
2. Идентификация узких мест и индексов
3. Внесение улучшений и изменение запросов
4. Проверка результата и тестирование производительности

Подсказка:

Сначала изучаем запросы, затем ищем проблемы, исправляем и тестируем.

Ответ:

- Анализ текущих запросов

- Идентификация узких мест и индексов
- Внесение улучшений и изменение запросов
- Проверка результата и тестирование производительности

🔗 Шаг 13:

Миссия 13

Вы создаете форму регистрации пользователей. Какая практика наиболее важна для предотвращения ошибок при сохранении данных в PostgreSQL?

Подсказка: Подумайте о проверке данных до их записи в базу.

Ответ:

- **Валидация и фильтрация данных на стороне PHP**
- Записывать данные без проверки
- Полагаться на JavaScript без серверной проверки
- Игнорировать потенциальные ошибки и выводить пользователю raw SQL

🔗 Шаг 14:

Миссия 14

📖 Сопоставьте ошибки базы данных PostgreSQL с возможными причинами:

Понятия:

Ответ:

- Unique Violation
- Foreign Key Violation
- Deadlock Detected
- Division by Zero

Определения:

1. Попытка вставить дублирующее значение в столбец с уникальным ограничением
2. Попытка удалить запись, на которую ссылается другая таблица
3. Две транзакции блокируют друг друга
4. Попытка деления на ноль в запросе

🔗 Подсказка:

Подумайте, какие ошибки связаны с ограничениями целостности, а какие с математикой или блокировками.

- **Unique Violation** — Попытка вставить дублирующее значение в столбец с уникальным ограничением
- **Foreign Key Violation** — Попытка удалить запись, на которую ссылается другая таблица
- **Deadlock Detected** — Две транзакции блокируют друг друга
- **Division by Zero** — Попытка деления на ноль в запросе

🔗 Шаг 15:

Миссия 15

📖 Сопоставьте методы оптимизации PHP-кода с их эффектом:

Понятия:

Ответ:

- Использование подготовленных выражений

- Кэширование результатов запросов
- Минимизация количества запросов
- Использование генераторов вместо массивов

Определения:

1. Снижение нагрузки на базу и повышение скорости работы скрипта
2. Снижение использования памяти при работе с большими наборами данных
3. Уменьшение числа обращений к базе, ускорение обработки
4. Защита от SQL-инъекций и повышение безопасности

🔗 Подсказка:

Подумайте, какие подходы помогают и с безопасностью, и с производительностью.

- **Использование подготовленных выражений** — Защита от SQL-инъекций и повышение безопасности
- **Кэширование результатов запросов** — Снижение нагрузки на базу и повышение скорости работы скрипта
- **Минимизация количества запросов** — Уменьшение числа обращений к базе, ускорение обработки
- **Использование генераторов вместо массивов** — Снижение использования памяти при работе с большими наборами данных

🔗 Шаг 16:

Миссия 16

Расположите шаги деплоя PHP-приложения с PostgreSQL:

1. Настройка окружения и зависимостей
2. Резервное копирование текущей базы данных
3. Перенос нового кода на сервер
4. Миграция базы данных
5. Тестирование функциональности

🔗 Подсказка:

Начинайте с подготовки окружения и защиты данных, затем перенос кода и базы, и только после этого тестируйте.

Ответ:

- Настройка окружения и зависимостей
- Резервное копирование текущей базы данных
- Перенос нового кода на сервер
- Миграция базы данных
- Тестирование функциональности

🔗 Шаг 17:

Миссия 17

Расположите шаги диагностики медленных запросов PostgreSQL:

1. Сбор информации через EXPLAIN и EXPLAIN ANALYZE
2. Идентификация узких мест и индексов
3. Оптимизация SQL-запросов
4. Проверка производительности после изменений

Подсказка:

Сначала анализируем запрос, затем выявляем проблему, исправляем и проверяем эффективность.

Ответ:

- Сбор информации через EXPLAIN и EXPLAIN ANALYZE
- Идентификация узких мест и индексов
- Оптимизация SQL-запросов
- Проверка производительности после изменений

Урок 5 - Презентация и защита итогового проекта

Шаг 01:

Миссия 1

Вы готовите защиту проекта и хотите выбрать правильный способ демонстрации работы PHP и PostgreSQL. Что будет оптимальным первым шагом?

Подсказка: Начните с наглядного примера, который легко объяснить аудитории.

Ответ:

- **Подготовить короткий демонстрационный скрипт с реальными данными**
- Сразу показывать весь код проекта
- Давать ссылки на документацию без демонстрации
- Презентовать только диаграммы базы данных

Шаг 02:

Миссия 2

На защите проекта кто-то спрашивает, как вы обеспечили целостность данных в PostgreSQL. Как лучше ответить?

Подсказка: Подумайте о ключевых концепциях базы данных, а не о коде в целом.

Ответ:

- **Объяснить использование первичных и внешних ключей и транзакций**
- Сказать, что данные проверяются вручную
- Сказать, что код PHP делает всё автоматически
- Избегать ответа, чтобы не показывать недостатки

Шаг 03:

Миссия 3

Вы хотите подчеркнуть важность обработки ошибок в проекте. Что будет лучшей демонстрацией на защите?

Подсказка: Покажите пример, который легко увидеть и понять.

Ответ:

- **Показать сценарий с try-catch и обработкой ошибки подключения к базе**
- Просто упомянуть в словах, что ошибки обрабатываются
- Скрыть ошибки и показать только успешные действия
- Удалить часть кода с ошибками перед демонстрацией

🔗 Шаг 04:

Миссия 4

📄 Сопоставьте элементы защиты проекта с их целью:

Понятия:

Ответ:

- Демонстрация работы скрипта
- Описание структуры базы данных
- Пояснение алгоритмов РНР
- Обсуждение обработки ошибок

Определения:

1. Показать на практике, как проект выполняет задачи
2. Объяснить организацию таблиц и связей
3. Показать, как реализована бизнес-логика и функции
4. Показать надежность и устойчивость проекта

🔗 **Подсказка:**

Каждый элемент защиты помогает аудитории понять отдельный аспект проекта.

- **Демонстрация работы скрипта** — Показать на практике, как проект выполняет задачи
- **Описание структуры базы данных** — Объяснить организацию таблиц и связей
- **Пояснение алгоритмов РНР** — Показать, как реализована бизнес-логика и функции
- **Обсуждение обработки ошибок** — Показать надежность и устойчивость проекта

🔗 Шаг 05:

Миссия 5

📄 Сопоставьте методы презентации проекта с их эффектом:

Понятия:

Ответ:

- Скриншоты интерфейса
- Живой показ работы
- Диаграммы схемы базы данных
- Кодовые фрагменты с пояснениями

Определения:

1. Демонстрирует визуальный аспект и дизайн
2. Позволяет аудитории увидеть работу проекта в реальном времени
3. Объясняет структуру и связи данных
4. Показывает логику работы функций и алгоритмов

🔗 **Подсказка:**

Разные форматы помогают раскрыть проект с разных сторон.

- **Скриншоты интерфейса** — Демонстрирует визуальный аспект и дизайн
- **Живой показ работы** — Позволяет аудитории увидеть работу проекта в реальном времени
- **Диаграммы схемы базы данных** — Объясняет структуру и связи данных
- **Кодовые фрагменты с пояснениями** — Показывает логику работы функций и алгоритмов

Шаг 06:

Миссия 6

 Сопоставьте типы аудитории с подходом к презентации проекта:

Понятия:

Ответ:

- Технические специалисты
- Менеджеры
- Клиенты
- Студенты

Определения:

1. Подробные схемы и алгоритмы, работа с кодом
2. Фокус на результатах и бизнес-ценности
3. Показ преимуществ и удобства использования
4. Обучающий подход с объяснением принципов работы

 **Подсказка:**

Разная аудитория требует разного уровня детализации и акцентов.

- **Технические специалисты** — Подробные схемы и алгоритмы, работа с кодом
- **Менеджеры** — Фокус на результатах и бизнес-ценности
- **Клиенты** — Показ преимуществ и удобства использования
- **Студенты** — Обучающий подход с объяснением принципов работы

Шаг 07:

Миссия 7

 Сопоставьте типы материалов для презентации с их целью:

Понятия:

Ответ:

- Слайды
- Демонстрационный скрипт
- Документация
- Диаграммы базы данных

Определения:

1. Визуальное подкрепление основных идей
2. Показ реальной работы приложения
3. Справочные материалы и инструкции
4. Пояснение структуры данных и связей

 **Подсказка:**

Каждый тип материала усиливает определенный аспект презентации.

- **Слайды** — Визуальное подкрепление основных идей
- **Демонстрационный скрипт** — Показ реальной работы приложения
- **Документация** — Справочные материалы и инструкции
- **Диаграммы базы данных** — Пояснение структуры данных и связей

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте подходы к объяснению функциональности с их преимуществом:

Понятия:

Ответ:

- Пошаговая демонстрация
- Сравнение с аналогами
- Объяснение кода
- Визуальные схемы процессов

Определения:

1. Показывает последовательность действий и результат
2. Позволяет подчеркнуть уникальность проекта
3. Дает техническое понимание реализации
4. Облегчает восприятие сложных процессов

🔗 Подсказка:

Выбирайте подход в зависимости от того, что хотите показать аудитории.

- **Пошаговая демонстрация** — Показывает последовательность действий и результат
- **Сравнение с аналогами** — Позволяет подчеркнуть уникальность проекта
- **Объяснение кода** — Дает техническое понимание реализации
- **Визуальные схемы процессов** — Облегчает восприятие сложных процессов

🔗 Шаг 09:

Миссия 9

Вы хотите показать на защите проект с PostgreSQL, чтобы аудитория увидела, как обрабатываются транзакции. Что будет лучшей демонстрацией?

Подсказка: Выбирайте простой сценарий, который наглядно показывает работу транзакций.

Ответ:

- **Показать пример с двумя зависимыми операциями и откатом при ошибке**
- Только показать SQL-запросы без объяснения
- Обсудить транзакции в теории без примера
- Игнорировать откаты и показывать только успешные случаи

🔗 Шаг 10:

Миссия 10

При защите проекта вас спрашивают о масштабируемости PHP-приложения. Как лучше ответить?

Подсказка: Покажите практическое понимание возможностей вашего решения.

Ответ:

- **Объяснить использование кэширования, пулов соединений и оптимизированных запросов**
- Сказать, что сервер справится сам без оптимизаций
- Игнорировать вопрос и переходить к следующему слайду
- Утверждать, что масштабируемость не важна

🔮 Шаг 11:

Миссия 11

Расположите шаги подготовки к защите проекта с PostgreSQL в правильном порядке:

1. Подготовка демонстрационных данных
2. Создание слайдов и схем
3. Репетиция демонстрации и сценария защиты
4. Проверка работы всех функций на локальном сервере
5. Настройка окружения на презентационном компьютере

🔮 Подсказка:

Начните с данных, затем материалы, проверка и репетиция, и в конце настройка окружения.

Ответ:

- Подготовка демонстрационных данных
- Создание слайдов и схем
- Репетиция демонстрации и сценария защиты
- Проверка работы всех функций на локальном сервере
- Настройка окружения на презентационном компьютере

🔮 Шаг 12:

Миссия 12

Расположите этапы подготовки ответов на вопросы комиссии в правильном порядке:

1. Составление списка возможных вопросов
2. Подготовка кратких и точных ответов
3. Обсуждение вопросов с командой
4. Репетиция ответов вслух
5. Фокус на демонстрацию практических примеров во время ответа

🔮 Подсказка:

Сначала продумайте вопросы, затем ответы, обсудите с командой и потренируйтесь, добавляя примеры.

Ответ:

- Составление списка возможных вопросов
- Подготовка кратких и точных ответов
- Обсуждение вопросов с командой
- Репетиция ответов вслух
- Фокус на демонстрацию практических примеров во время ответа

🔮 Шаг 13:

Миссия 13

Вы хотите, чтобы комиссия оценила надежность проекта. Что лучше показать?

Подсказка: Демонстрация реальной обработки ошибок выглядит убедительнее, чем просто слова.

Ответ:

- Пример **try-catch** для ошибок подключения и работы с базой
- Сказать, что ошибок нет
- Удалить проблемные части кода и показать только успешные случаи

- Показать только лог-файлы без объяснения

Шаг 14:

Миссия 14

На защите вас спрашивают о безопасности данных пользователей. Как лучше ответить?

Подсказка: Покажите реальные меры, а не общие фразы.

Ответ:

- **Объяснить использование подготовленных выражений и шифрования паролей**
- Сказать, что безопасность обеспечивается сервером
- Не обсуждать этот аспект
- Показать только текстовые файлы без шифрования

Шаг 15:

Миссия 15

Расположите этапы финальной репетиции презентации проекта:

1. Прогон всей демонстрации с командой
2. Отработка ответов на сложные вопросы
3. Исправление ошибок и недочетов
4. Синхронизация с презентационными материалами
5. Оценка времени и корректировка темпа

Подсказка:

Сначала пробегаем демонстрацию, отрабатываем вопросы, исправляем ошибки, синхронизируем материалы и проверяем темп.

Ответ:

- Прогон всей демонстрации с командой
- Отработка ответов на сложные вопросы
- Исправление ошибок и недочетов
- Синхронизация с презентационными материалами
- Оценка времени и корректировка темпа

Шаг 16:

Миссия 16

Расположите последовательность действий на защите итогового проекта:

1. Приветствие и краткое введение
2. Демонстрация работы PHP-приложения с PostgreSQL
3. Объяснение структуры базы данных и логики кода
4. Показ обработки ошибок и надежности системы
5. Ответы на вопросы аудитории
6. Заключение и выводы

Подсказка:

Сначала вступление, затем демонстрация, объяснение, надежность, вопросы и финальные выводы.

Ответ:

- Приветствие и краткое введение

- Демонстрация работы PHP-приложения с PostgreSQL
- Объяснение структуры базы данных и логики кода
- Показ обработки ошибок и надежности системы
- Ответы на вопросы аудитории
- Заключение и выводы

Шаг 17:

Миссия 17

Расположите этапы подготовки ответов на вопросы комиссии в правильном порядке:

1. Составление списка возможных вопросов
2. Подготовка кратких и точных ответов
3. Обсуждение вопросов с командой
4. Репетиция ответов вслух
5. Фокус на демонстрацию практических примеров во время ответа

Подсказка:

Сначала продумайте вопросы, затем ответы, обсудите с командой и потренируйтесь, добавляя примеры.

Ответ:

- Составление списка возможных вопросов
- Подготовка кратких и точных ответов
- Обсуждение вопросов с командой
- Репетиция ответов вслух
- Фокус на демонстрацию практических примеров во время ответа

Шаг 18:

Миссия 18

Вы хотите, чтобы комиссия оценила надежность проекта. Что лучше показать?

Подсказка: Демонстрация реальной обработки ошибок выглядит убедительнее, чем просто слова.

Ответ:

- **Пример try-catch для ошибок подключения и работы с базой**
- Сказать, что ошибок нет
- Удалить проблемные части кода и показать только успешные случаи
- Показать только лог-файлы без объяснения

Шаг 19:

Миссия 19

На защите вас спрашивают о безопасности данных пользователей. Как лучше ответить?

Подсказка: Покажите реальные меры, а не общие фразы.

Ответ:

- **Объяснить использование подготовленных выражений и шифрования паролей**
- Сказать, что безопасность обеспечивается сервером
- Не обсуждать этот аспект
- Показать только текстовые файлы без шифрования